

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	DVT	Documentation quality	High	
Timeline	2026-02-16 through 2026-02-26	Test quality	High	
Language	Solidity	Total Findings	21	Fixed: 10 Acknowledged: 9 Mitigated: 2
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	2	Fixed: 2
Specification	Feature description ⓘ	Medium severity findings ⓘ	7	Fixed: 4 Acknowledged: 2 Mitigated: 1
Source Code	ssvlabs/ssv-network ⓘ #fcfde8c ⓘ	Low severity findings ⓘ	10	Fixed: 4 Acknowledged: 5 Mitigated: 1
Auditors	- Leonardo Passos Senior Research Engineer - Jennifer Wu Auditing Engineer - Cameron Biniamow Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	2	Acknowledged: 2

Summary of Findings

SSV Network is a decentralized Ethereum staking protocol that leverages Distributed Validator Technology (DVT) to split validator responsibilities among multiple independent operators, who operate inside a logical cluster on behalf of their associated validators.

The SSV Network is transitioning to ETH-denominated payments and effective balance accounting, fundamentally changing how fees, cluster runway, and liquidations are managed.

The upgrade introduces three key components: (1) ETH Payments, aligning protocol fees with validator rewards and simplifying operations; (2) Effective Balance (EB) Accounting, which ensures fees and risk calculations scale with actual validator stake rather than fixed assumptions; and (3) SSV Staking, enabling SSV holders to stake tokens, delegate voting power for oracle selection, and earn ETH-denominated rewards. Effective Balance Oracles will track validator balances and update protocol state, starting with a permissioned set and evolving toward a permissionless, stake-driven model. Existing SSV-based clusters must migrate to ETH payments to maintain full functionality, and new governance parameters will control economic and risk boundaries. This upgrade aims to improve economic alignment and operational simplicity. These changes were introduced in commit #0c772bf, the target commit for the initial audit.

The goal of this audit is to assess the security of the smart contracts implementing SSV's features for ETH-denominated payments and effective balance accounting. We initially identified 21 issues that could compromise the protocol's integrity, reliability, and usability. ****All issues were properly addressed or acknowledged during the fix review phase. All patches were merged.**

After the fix review, the SSV team conducted a thorough analysis of their implementation and independently identified new issues. Additionally, they refactored code, addressed quality concerns, added new features, adjusted parameters, and created new tests. A new fix-review phase followed, and no unintended side effects were identified. Tracking of these change batches can be [found here](#).

The final commit for mainnet launch is #fcfde8c.

We praise the project for the significant effort to improve protocol correctness and for pursuing independent bug findings after the initial report & fix phase. We also note that the team added 992 new tests (a 136% increase). Overall, the average statement coverage is +95%, on par with high-standard projects.

Disclaimer

Quantstamp also audited the smart contracts of the SSV protocol that relied solely on SSV tokens as the primary currency when collecting fees and paying rewards (no ETH support). Most of the issues captured by the [original report](#) were acknowledged by the SSV team, but not

necessarily fixed. This stems from the understanding that the reported issues are not considered a significant threat in the present setup of the network. These issues, however, still apply here. We resurface them only if a new perspective is raised.

ID	DESCRIPTION	SEVERITY	STATUS
SSV-1	Stale Memory Write-Back Locks Operators at Zero ETH Fee & Silently Reactivates Removed Operators	• High ⓘ	Fixed
SSV-2	Live cSSV Supply Used Per Vote in <code>commitRoot</code> Allows Supply Manipulation to Block or Bypass Oracle Quorum	• High ⓘ	Fixed
SSV-3	Validator Registration Can Leave Cluster Immediately Liquidatable	• Medium ⓘ	Fixed
SSV-4	Remove-And-ReRegister Can Reset Explicit EB to 32 ETH Baseline and Create a Temporary Fee Undercharge Window	• Medium ⓘ	Acknowledged
SSV-5	Operator Removal Can Compromise Clusters Fault Tolerance	• Medium ⓘ	Acknowledged
SSV-6	ETH Rewards Accrued During Zero cSSV Supply Become Unclaimable	• Medium ⓘ	Mitigated
SSV-7	EB Auto-Liquidation Can Leave <code>ethValidatorCount</code> Inflated and Inflate Future ETH Accrual	• Medium ⓘ	Fixed
SSV-8	Double-Accounting of EB Deviation Blocks Validator Removal From Liquidated Clusters	• Medium ⓘ	Fixed
SSV-9	Incorrect Oracle Weight May Lead to Reach to Premature Quorum	• Medium ⓘ	Fixed
SSV-10	Cluster Owners Can Avoid Liquidation By Removing All Validators Before Withdrawal	• Low ⓘ	Acknowledged
SSV-11	Legacy Fee Requests May Execute After Upgrade With Incompatible Fee Scale	• Low ⓘ	Mitigated
SSV-12	Liquidation Fallback Branch Incorrectly Adds <code>operatorEthVUnits</code> in Sub-Baseline Case	• Low ⓘ	Acknowledged
SSV-13	Operator Registration Can Be DoSed	• Low ⓘ	Acknowledged
SSV-14	Existing Registration Setup Enables Phantom Operators to Extract Fees and Degrade Cluster Fault Tolerance	• Low ⓘ	Acknowledged
SSV-15	Deferred Reward Accounting Exposes Stakers to Eth Price Volatility	• Low ⓘ	Fixed
SSV-16	Non-Standard ERC20 Tokens Trapped in <code>SSVstaking</code> Contract	• Low ⓘ	Fixed
SSV-17	Stale Cluster Effective Balance Updates	• Low ⓘ	Fixed
SSV-18	Direct Liquidations Do Not Consider Effective Balance Updates	• Low ⓘ	Acknowledged
SSV-19	<code>replaceOracle</code> Allows Out-of-Set Oracle IDs To Vote in Root Commitments	• Low ⓘ	Fixed
SSV-20	Duplicate BLS Key Registration Across Owners Exposes Validators to Beacon Chain Slashing	• Informational ⓘ	Acknowledged
SSV-21	Operator Onboarding May Lead to Increasing Centralization	• Informational ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.



Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/ssvlabs/ssv-network`

Included Paths: `contracts/`

Operational Considerations

- Effective Balance (EB) updates are permissionless and oracle-driven, so cluster owners do not control when an EB change is applied on-chain. Under Pectra-era consolidation and withdrawal activity, a cluster's real effective balance can change off-chain before state is refreshed on-chain. This creates timing risk: a cluster that appears sufficiently funded under stale on-chain `vUnits` can become liquidatable immediately when a newer EB proof is submitted.
- When EB updates are delayed, clusters continue accruing fees using previously stored `vUnits` . If real EB has increased during the delay window, operator and network fees are under-collected until the next update. Once refreshed, accounting catches up for future accrual and liquidation checks, which can create abrupt changes in the apparent runway.
- Temporary EB/accounting drift can also occur because the protocol combines oracle-driven EB snapshots with local state transitions (for example, validator add/remove flows). As a result, short-lived discrepancies can appear between expected and observed fee/runway values before the next EB refresh reconciles all components.
- The EB update rules enforce `effectiveBalance >= validatorCount * 32 ETH` , which embeds a 32 ETH minimum per validator in protocol accounting. If the validator effective balance drops below 32 ETH after penalties or slashing, the protocol continues to account from this floor. This can overstate the fee base and liquidation safety relative to actual validator health.
- Since SSV-based clusters become legacy and cannot be extended via additional SSV deposits after the upgrade, clusters with low remaining runway (or users who miss the top-up window) face a heightened risk of hard liquidation before migration is completed. This creates migration-timing risk for legacy users and can force unplanned cluster exits even when migration intent exists.

- Governance voting is performed off-chain, and it is outside the scope of this audit.
- Governance-approved changes are then reflected on-chain by means of updating various protocol parameters.
- The owner of the main contract of the protocol (`SSVNetwork.sol`) is the DAO multisig.
- Governance-defined parameters are rational, particularly around fees. Maximum operator and network fees are never higher than the expected rewards that a validator ought to receive from the beacon chain. Furthermore, no governance takeover scenario is assumed.
- Snapshot updates take at maximum a few days, in the worst-case scenario. It is never in the order of months or years. This eliminates overflow and truncation concerns regarding the computation of accumulated amounts (rewards, DAO balance, etc) that could otherwise impair the protocol functioning.
- The protocol relies on off-chain performance indices to detect and penalize non-performing operators. Continuous monitoring and timely updates to these indices are essential to prevent silent degradation of cluster fault tolerance.

Following the fix review,

- `updateClusterBalance` now requires the latest committed root, so non-latest committed roots can no longer be used for EB updates.
- Root-commit quorum now uses per-round frozen cSSV supply.
- Oracle replacement is constrained to the fixed 4 oracle slots.

Key Actors And Their Capabilities

SSV Network Contract Owner Set during initialization; governs global protocol parameters. Trusted to apply changes within reasonable bounds and with adequate notice. Module upgrades must be audited beforehand to prevent vulnerabilities and storage collisions.

- Set ETH and SSV network fees
- Withdraw accumulated network SSV earnings
- Configure operator fee constraints (increase limit, declare/execute periods, max/min fee)
- Configure liquidation parameters (threshold periods, minimum collateral) for ETH and SSV clusters
- Manage oracle registry (replace oracle addresses within the fixed 4-slot ID set, set quorum threshold)
- Set SSV staking un stake cooldown
- Recover non-SSV/non-cSSV tokens from the contract (`rescueERC20`)
- Upgrade module implementations

Oracles Compute validator effective balances off-chain and submit Merkle root commitments on-chain. Multiple oracles must agree on the same root commitment (same `blockNum` and Merkle root) to reach quorum. Colluding oracles could commit malicious roots and trigger improper liquidations.

- Submit effective balance Merkle root proposals (`commitRoot`): caller must be a registered oracle; commitment blocks must be strictly increasing; quorum weight is based on cSSV supply frozen at the first vote for each commitment round.

Operators Run SSV node infrastructure; collaboratively manage validators within their clusters (block proposals, attestations, sync committees). Each cluster requires a threshold of operators to produce a valid signature (e.g., 3-of-4). Trusted to maintain high availability and protect validator key shares.

- Register and remove themselves
- Manage ETH fees (declare, execute, cancel, reduce)
- Set privacy status and whitelist settings
- Withdraw ETH and SSV earnings

Validators (Cluster Owners) Create clusters by pairing validators with operators and depositing ETH to fund fees. Responsible for monitoring cluster health, operator performance, and maintaining sufficient balance to avoid liquidation.

- Register and remove validators (single and bulk)
- Initiate validator exit
- Reactivate a liquidated cluster
- Withdraw excess or full cluster funds
- Migrate an SSV-funded cluster to an ETH-funded cluster (`migrateClusterToETH`)

Liquidators Any on-chain actor may liquidate undercollateralized clusters. Successful liquidators claim the remaining cluster balance after all fees are settled.

- **Balance-based:** call `liquidate()` when a cluster's net balance falls below the threshold; fees settle immediately and the liquidator claims the remainder.
- **Effective-balance-based:** call `updateClusterBalance()` when a cluster has stale on-chain effective balances; triggers liquidation if the updated balance renders the cluster undercollateralized.

Findings

SSV-1

Stale Memory Write-Back Locks Operators at Zero ETH Fee & Silently Reactivates Removed Operators

• High ⓘ Fixed



Update

Addressed in: `bd973d47c7d689b270c2c5aa49a2501b53fec3c1` .
The client reordered `updateClusterOperatorsOnRegistration()` to initialize ETH defaults in storage before memory copy/write-back, and added `ensureOperatorExist` (snapshot-based) checks that block removed operators from being reintroduced during

registration. The client also validated this with tests for legacy ETH default initialization and removed-operator reverts in registration flows.

File(s) affected: `contracts/libraries/OperatorLib.sol` , `contracts/modules/SSVOperators.sol` , `contracts/modules/SSVValidators.sol`

Description: `OperatorLib.updateClusterOperatorsOnRegistration` takes a storage-to-memory copy of each `Operator` it iterates over, calls `ensureETHDefaults` on the **storage** reference, and finally writes the **memory** copy back to storage. Because the memory copy is taken before the storage mutation and written back after it, the two diverge.

This issue stems from the inconsistent use of memory and storage data in the library contracts. Some library functions update the memory data provided as a parameter, while others update the storage data directly. This deviation in data handling increases the risk of stale data in calculations and incorrect data in storage. Presently, two bugs arise:

Bug 1: `ethFee` permanently clobbered for SSV-only operators

When an SSV-only operator (one with `ethFee = 0` , `ethSnapshot.block = 0` , `fee != 0`) is first included in an ETH cluster, `ensureETHDefaults` writes `ethFee = defaultOperatorEthFee()` to storage. But the memory copy still holds `ethFee = 0` . The write-back at the end of the loop overwrites storage with the stale zero value:

```
// [1] Memory copy – ethFee=0 for SSV-only operators
ISSVNetworkCore.Operator memory operator = s.operators[operatorId];

// [2] Storage write – sets ethFee = defaultOperatorEthFee()
ensureETHDefaults(s.operators[operatorId]);

// [3] Snapshot computed on memory copy with ethFee=0
//     blockDiffEthFee = (currentBlock - 0) * 0 = 0
//     ethSnapshot.block set to currentBlock unconditionally
updateSnapshot(operator, operatorId);

// [4] Reads 0 instead of defaultOperatorEthFee()
cumulativeFee += PackedETH.unwrap(operator.ethFee);

// [5] Write-back clobbers the ethFee that ensureETHDefaults set
s.operators[operatorId] = operator;
```

The consequences are:

- The operator is left permanently with `ethFee = 0` . It accrues no ETH fees for any cluster it belongs to. The `cumulativeFee` returned to the caller is also understated, corrupting the cluster's burn-rate calculation and liquidation threshold.
- `declareOperatorFee` no longer re-applies defaults once `ethSnapshot.block != 0` .
- Fee increase checks are based on current `ethFee` ; when it is zero, `maxAllowedFee` is zero, so non-zero fee declarations revert (`FeeExceedsIncreaseLimit`).

Bug 2: removed operators are reactivated on validator registration

When `isExistingCluster = true` , the existence check only requires `operator.owner != address(0)` , which passes for removed operators (since `_resetOperatorState` preserves `owner`). The function then:

1. Calls `ensureETHDefaults` on storage — sees `ethSnapshot.block == 0` , sets it to `block.number`
2. Calls `updateSnapshot` on the memory copy — unconditionally sets `operator.ethSnapshot.block = currentBlock`
3. Increments `operator.ethValidatorCount += deltaValidatorCount` on the memory copy
4. Writes back the memory copy — persists `ethSnapshot.block = currentBlock` (non-zero) and the incremented `ethValidatorCount`

After the write-back, the removed operator has `ethSnapshot.block != 0` , which is the signal downstream code (`updateClusterOperators` , `updateSnapshotSt`) uses to identify active operators. The operator is effectively **reactivated**: it appears active to all snapshot logic and carries an incorrectly inflated `ethValidatorCount` . Its `ethFee` remains zero (since `_resetOperatorState` also zeroed `fee` , and `ensureETHDefaults` only sets `ethFee` when `fee != 0`), so no fees accrue — but the corrupted state persists indefinitely.

Recommendation:

1. In `updateClusterOperatorsOnRegistration` , use storage-only mutation (or reload from storage after `ensureETHDefaults`), so initialized ETH defaults cannot be overwritten.
2. Add regression tests for legacy operator state transitions and assert no zero-fee lock is possible.
3. Guard `ensureETHDefaults` so it is skipped for removed operators. In the `isExistingCluster` branch, removed operators are deliberately allowed through the existence check (to avoid bricking existing clusters), but they should not have their ETH state re-initialised.

SSV-2

Live cSSV Supply Used Per Vote in `commitRoot` Allows Supply Manipulation to Block or Bypass Oracle Quorum

• High ⓘ Fixed

✓ Update

Addressed in: `aa2fe7669cdee76b09e8e4cc97a57ccd2d838897` .

The client fixed the issue by freezing the cSSV voting supply on the first `commitRoot` vote for each (`blockNum` , `merkleRoot`) round and reusing that same frozen supply for all subsequent quorum and weight calculations in that round. The client also added regression tests to verify that mid-round cSSV supply increases or decreases do not change quorum outcomes within an ongoing commitment round.

File(s) affected: `contracts/modules/SSVDAO.sol` , `contracts/modules/SSVStaking.sol` , `contracts/libraries/storage/SSVStorageStaking.sol`

Description: `commitRoot` uses live cSSV total supply for every vote, but vote weight is accumulated across calls for the same `commitmentKey` . This mixes a single round across multiple supply states.

In the current implementation, each vote does all of the following at call time:

1. Computes added vote weight as `currentTotalSupply / defaultOracleIds.length` .
2. Computes quorum threshold as `(currentTotalSupply * quorumBps) / 10_000` .
3. Adds that vote to previously stored round weight in `rootCommitments[commitmentKey]` .

Because `currentTotalSupply` can change between votes (stake mints `cSSV` , unstake burns `cSSV`), the round is evaluated against a moving denominator:

1. If supply increases mid-round, quorum becomes harder to reach (liveness risk).
2. If supply decreases mid-round, quorum becomes easier to reach (safety risk).

Concrete example: 4 oracles, `quorumBps = 7500`

1. Vote #1 at supply `S` adds `S/4` .
2. Supply increases to `2S` ; votes #2 and #3 each add `S/2` .
3. Accumulated weight after 3 votes is `1.25S` .
4. Current threshold is `1.5S` .
5. Round fails despite `3/4` votes.

Recommendation: Freeze quorum parameters at round start and keep them immutable for that `commitmentKey` :

1. On first vote, persist `roundTotalSupply` , `roundVoteWeight` , and `roundThreshold` .
2. For every later vote in the same round, use only stored round parameters and never read live `totalSupply` .
3. Ensure any `cSSV` supply change affects only future rounds (new `commitmentKey`).
4. Add regression tests where supply changes between votes and assert the same outcome as a no-supply-change round.

Alternatively, make `cSSV` a votable token by inheriting OpenZeppelin `ERC20Votes` and pin each round to a fixed `snapshotBlock` ; then compute quorum/weight via `getPastTotalSupply(snapshotBlock)` (and `getPastVotes` if moving to account-level vote weights) so mint/burn events after the snapshot cannot change the active round.

SSV-3

Validator Registration Can Leave Cluster Immediately Liquidatable

• Medium ⓘ Fixed

✓ Update

Addressed in: `4d1689bcc88bdcef262c19c8152000d2570a89fb` .

The client fixed this by running the registration liquidation-safety check against projected post-registration `vUnits` (including newly added validators), not stale pre-registration `vUnits` . The client validated the fix by adding sanity tests that make `bulkRegisterValidator` revert when the balance only covers old `vUnits` and succeed when the balance covers post-registration `vUnits` .

File(s) affected: `contracts/modules/SSVValidators.sol`

Description: During validator registration, the liquidation-safety check is executed before the cluster's explicit EB units are updated for the newly added validators. For clusters already using explicit `clusterEB.vUnits` , this creates a pre-/post-state mismatch:

1. The check evaluates solvency using old (lower) `vUnits`.
2. Registration is accepted.
3. The transaction then increases `clusterEB.vUnits` for the added validators.

After the same transaction, the final cluster state can already be liquidatable even though registration succeeded. In practice, this can happen when an existing cluster near the liquidation threshold adds validators and immediately becomes eligible for liquidation.

Recommendation: Validate registration against projected post-registration units, not pre-update units. Concretely, either:

1. Apply `clusterEB.vUnits += validatorsAdded * VUNITS_PRECISION` before liquidation-safety validation, or
2. Keep write order but compute liquidation threshold using `projectedVUnits` in the check.

Add regression tests for explicit-EB clusters near threshold that add validators (including existing-cluster path), and assert registration reverts whenever final post-registration state would be liquidatable.

SSV-4

Remove-And-ReRegister Can Reset Explicit EB to 32 ETH Baseline and Create a Temporary Fee Undercharge Window

• Medium ⓘ Acknowledged

Update

The client acknowledged and provided the following explanation:

- Residual risk is accepted for v2.0.0 due to low expected likelihood under current operations:
- Oracles update roots 3 times per day.
- `updateClusterBalance` is permissionless.
- A company-run instance updates cluster balances when they change.
- The remove-then-re-register path also carries material practical disincentives:
- Additional gas costs (bulk remove and especially bulk re-register operations).
- Operational risk of missed beacon-chain duties while validators are removed from SSV node duties (removal is not validator exit, but SSV nodes stop performing duties during that period).
- No protocol code change is planned at this time.

File(s) affected: `contracts/modules/SSVValidators.sol`, `contracts/libraries/ClusterLib.sol`, `contracts/modules/SSVClusters.sol`

Description: A cluster that previously had explicit high effective-balance units can be reset to implicit baseline accounting through a remove-then-re-register sequence:

1. Remove validators until `cluster.validatorCount == 0`, which sets `clusterEB.vUnits = 0`.
2. Re-register validators into the same cluster while `clusterEB.vUnits == 0`.
3. Registration keeps explicit EB at zero (it only increments snapshot vUnits when `vUnits > 0`), so fee/liquidation math falls back to `validatorCount * VUNITS_PRECISION` baseline.

As a result, the cluster temporarily pays baseline-rate fees instead of true EB-weighted fees until a later `updateClusterBalance` restores explicit EB units. This can be repeated and is more impactful when EB is high (e.g., post-consolidation) and root/update cadence is slow.

Recommendation: Prevent EB-reset fee dodge on full validator removal:

1. Preserve/restitch explicit EB state across remove-to-zero and re-registration (do not allow implicit fallback immediately after explicit EB history), or force an EB refresh before allowing re-registration to reactivate charging.
2. Add regression tests for `explicit high EB -> remove all validators -> re-register`, asserting no baseline undercharge window.

SSV-5

Operator Removal Can Compromise Clusters Fault Tolerance

• Medium ⓘ Acknowledged

Update

The client acknowledged and provided the following explanation:

This is an intentional design decision, not a bug. The SSV Network protocol operates on the principle of delegated responsibility where:

1. Operators have autonomy - Operators are independent service providers and must be free to exit the network (similar to how Ethereum validators can voluntarily exit)
2. Cluster owners bear monitoring responsibility - It is the cluster owner's responsibility to:
 - Monitor the health and status of operators in their clusters
 - React to operator removal events by migrating validators to new operators
 - Maintain sufficient fault tolerance for their validator operations

Rationale:

 - No on-chain enforcement - The protocol cannot force operators to continue providing services against their will
 - Economic incentives - Operators have economic incentives (ongoing fees) to remain active, but these incentives cannot be guaranteed indefinitely
 - Off-chain monitoring - The SSV ecosystem provides off-chain monitoring tools and operator reputation systems to help cluster owners track operator status
 - Graceful degradation - Reduced operator sets (e.g., 3-of-4) can still operate validators, though with reduced fault tolerance. This gives cluster owners time to respond.

Existing Mitigations:

 1. Event emission - OperatorRemoved event is emitted when an operator exits, allowing off-chain monitoring systems to alert cluster owners
 2. Operator earnings withdrawal - Final earnings are paid out, ensuring no locked funds
 3. Cluster continues operating - Validators don't immediately fail; they continue with reduced operator coverage
 4. Reactivation support - Clusters can be reactivated after operator changes

File(s) affected: `contracts/modules/SSVOperators.sol`, `contracts/libraries/OperatorLib.sol`

Description: `SSVOperators.removeOperator` accepts any removal request from the operator owner without checking whether the operator still has active validators in running clusters. Nothing prevents the removal when `operator.validatorCount > 0` or `operator.ethValidatorCount > 0`.

SSV clusters rely on threshold BLS signatures distributed across their operator set. When an operator exits unilaterally:

- **Reduced fault tolerance:** a 4-operator cluster (3-of-4 threshold) immediately loses all fault tolerance; one additional departure silences the validators entirely.
- **Immutable operator set:** the cluster owner cannot substitute the departed operator in-place — doing so would change the cluster identity and invalidate the stored cluster hash. The only recovery path is to remove all validators from the affected cluster one by one, deploy a new cluster with a replacement operator, and re-register every validator — a costly, gas-intensive migration that imposes downtime.
- **No minimum active-operator enforcement:** neither `updateClusterOperators` nor any other cluster-level function checks that a minimum number of operators remain active before processing cluster state changes.

A malicious or negligent operator can therefore force expensive validator migrations or silently degrade the security guarantees of every cluster it belongs to, while incurring no on-chain penalty.

Recommendation: Add a guard in `removeOperator` that reverts if the operator still has active validators:

```
if (operator.validatorCount != 0 || operator.ethValidatorCount != 0) {
    revert OperatorHasActiveValidators();
}
```

Also, consider tracking the count of active (non-removed) operators per cluster and enforcing a minimum threshold during cluster operations.

SSV-6

ETH Rewards Accrued During Zero cSSV Supply Become Unclaimable

• Medium ⓘ Mitigated

Update

Addressed in: `b702fed19233a692b5b01a8b87005ea34bd4cfdd`.

The client mitigated the issue by enforcing a deployment-time bootstrap step in the DAO Safe upgrade batch: immediately `approve` and `stake` (1 SSV) after upgrade so `cSSV.totalSupply()` is non-zero before any fee accrual, preventing the zero-supply `_syncFees()` accounting gap.

The client provided the following explanation:

Mitigation Strategy: Deployment Procedure (Not Code Fix)

This issue is mitigated by deployment procedure rather than a code change. The DAO multisig (Safe) upgrade batch transaction includes an SSV `approve + stake(1 SSV)` after the upgrade of the contracts finishes. This ensures:

- `cSSV.totalSupply() > 0` before any network fees can accrue
- The zero-staked window is impossible in practice
- The 1 SSV stake goes to the DAO address (tokens not lost)

File(s) affected: `contracts/modules/SSVStaking.sol`, `contracts/libraries/ProtocolLib.sol`, `contracts/modules/SSVDAO.sol`

Description: `cSSV` is the staking share token minted to stakers, and ETH rewards are distributed by increasing `accEthPerShare` using `totalSupply(cSSV)` as the denominator. Staker claims are derived from `accrued + balance * (accEthPerShare - userIndex)`.

In `_syncFees`, the contract always updates `stakingEthPoolBalance` to the latest network ETH earnings (`current`), but it updates `accEthPerShare` only when `totalStaked != 0`. This creates a gap when `totalSupply(cSSV) == 0`:

1. ETH rewards accumulate while there are no `cSSV` holders (bootstrap period or after full exit).
2. `syncFees` is called during that zero-supply window.
3. `stakingEthPoolBalance` moves forward, but `accEthPerShare` does not.
4. The skipped delta is never attributed to any shareholder.

When supply later becomes non-zero, those rewards remain outside user-share accounting and cannot be claimed via normal reward settlement.

Recommendation: 1. Handle zero-supply ETH accrual with one of two policies:

- Carry-forward policy: track `undistributedEthRewards` whenever `totalSupply(cSSV) == 0` and fold it into the next non-zero-supply distribution.
- Sweep policy: expose an explicit governance withdrawal/sweep path for undistributed ETH (similar to the current SSV network earnings withdrawal).

1. Add regression tests for bootstrap zero-supply accrual, full-exit-to-zero accrual, and repeated `syncFees` calls during zero-supply windows.

SSV-7

EB Auto-Liquidation Can Leave ethValidatorCount Inflated and Inflate Future ETH Accrual

• Medium ⓘ Fixed

✓ Update

Addressed in `f54a514fbf2d005e836359d956264ace81069a46` and `b431079a8bc49faf7700b35e3bf839bc15f18230`.

The client fixed this by changing liquidation cleanup to decrement `ethValidatorCount` based on ETH snapshot activity only, and by blocking removed operators from being reintroduced via registration. The client validated the fix by adding regression tests that removed operators revert in new and existing cluster registrations, and that liquidation/accounting paths keep operator validator counts and vUnits consistent after liquidation.

File(s) affected: `contracts/modules/SSVClusters.sol`, `contracts/libraries/OperatorLib.sol`

Description: In `_liquidateAfterEBUpdateIfNeeded`, ETH validator cleanup is gated by both ETH and SSV snapshot state:

```
if (op.ethSnapshot.block != 0 && op.snapshot.block != 0) { op.ethValidatorCount -= cluster.validatorCount; }
```

For mixed-snapshot operators (`ethSnapshot.block != 0`, `snapshot.block == 0`), liquidation still executes, but the decrement is skipped. This can leave `ethValidatorCount` overstated after liquidation and distort later ETH accounting.

A reachable path for this mixed state is the separate issue, *Stale Memory Write-Back Locks Operators at Zero Eth Fee & Silently Reactivates Removed Operators*, where a removed operator can be reintroduced with ETH snapshot state reinitialized while SSV snapshot state remains zero.

Recommendation:

- In `_liquidateAfterEBUpdateIfNeeded`, decrement `ethValidatorCount` using ETH snapshot state only, e.g. `if (op.ethSnapshot.block != 0)`, and do not gate on `op.snapshot.block`.
- Fix this issue together with *Removed Operators Can Be Re-Added Through Existing-Cluster Registration*, and add an end-to-end regression that covers `removeOperator` → `existing-cluster registration` → `EB auto-liquidation`.

SSV-8

Double-Accounting of EB Deviation Blocks Validator Removal From Liquidated Clusters

• Medium ⓘ Fixed

✓ Update

Addressed in `7bbb22603b03c2b2b8a67bfc197bc82f470c23ea`.

The client fixed this by preventing a second deviation deduction when removing validators from already-liquidated clusters. The client validated the fix by testing both bulk and sequential validator removals after liquidation and confirming no underflow and no additional deviation/accounting deduction occurs.

File(s) affected: `contracts/modules/SSVClusters.sol`, `contracts/modules/SSVValidators.sol`

Description: The protocol intentionally preserves `clusterEB.vUnits` across `liquidate` → `reactivate` so reactivation can restore prior EB weighting. However, liquidation already settles cluster deviation from global accounting (`operatorEthVUnits` and `daoTotalEthVUnits`). Later, if the owner removes validators from that already-liquidated cluster until `validatorCount == 0`, the final cleanup path treats the remaining `clusterEB.vUnits` residue as fresh deviation and subtracts it again from global/operator deviation accounting. This creates a second deduction for the same deviation that was already settled at liquidation, causing either underflow reverts or understated accounting.

Example

Cluster with 1 validator, EB = 48 ETH → `vUnits = 15000`, deviation = `15000 - 10000 = 5000`.

During liquidation (`SSVClusters._executeLiquidation`)

- `sp.updateDAO(false, 1)` → `daoTotalEthVUnits -= 10000` (baseline via `updateDAO`)
- Deviation removal → `daoTotalEthVUnits -= 5000`, `operatorEthVUnits[op] -= 5000`
- `ebSnapshot.vUnits` is **never cleared** — still reads 15000

During validator removal (`SSVValidators._bulkRemoveValidator`)

- `cluster.active == false` → operator count and DAO updates are skipped (correct)
- `ebSnapshot.vUnits == 15000 > 0` → enters EB cleanup block
- `ebSnapshot.vUnits -= 10000` → 5000 remains
- `cluster.validatorCount == 0` → triggers final cleanup:
 - `operatorEthVUnits[op] -= 5000` → **underflow** (already 0 from liquidation) → **revert**
 - `daoTotalEthVUnits -= 5000` → **underflow** → **revert**

Recommendation:

- Keep snapshot persistence for reactivation, but make global deviation settlement one-time.
- In inactive-cluster validator removal, skip final global deviation subtraction when deviation was already settled at liquidation (or track an explicit `deviationSettled` state).

3. Add a regression test to close this lifecycle gap: `explicit deviation -> liquidate -> remove all validators`, including multi-cluster operator scenarios, and assert global/operator deviation is deducted exactly once.

SSV-9

Incorrect Oracle Weight May Lead to Reach to Premature Quorum

• Medium ⓘ Fixed

✓ Update

Addressed in: `58984fe4d647e7eb5a373c6e18b61e9869518aa0`.

The oracle set is hard-capped at 4 slots, so `replaceOracle()` can rotate oracle addresses but cannot change the oracle count. The client also explained that the weight is there to prepare for a permissionless voting upgrade. This is why cSSV total supply is used, even though there are only 4 oracles.

File(s) affected: `contracts/modules/SSVDAO.sol`

Description: The `commitRoot()` function calculates the oracle's weight as the cSSV total supply divided by the `s.defaultOracleIds.length`, a fixed array with a length of four. The `s.defaultOracleIds` array is not otherwise referenced in the `commitRoot()` function and may serve as a misleading number of total oracles. In reality, there can be any number of oracles, as long as the oracle was previously registered via `replaceOracle()`. Therefore, if the actual number of registered oracles deviates from the default number of oracles, a quorum may be reached prematurely or never be reached at all. While the SSV team has stated that they will initially use four oracles with a 3-of-4 threshold, the `commitRoot()` function is not future-proof if the SSV team adds or removes oracles.

Recommendation: Track the actual number of registered oracles in a global state variable. Update the `weight` calculation to use the actual number of registered oracles instead of the default number of oracles.

SSV-10

Cluster Owners Can Avoid Liquidation By Removing All Validators Before Withdrawal

• Low ⓘ Acknowledged

ⓘ Update

The client acknowledged and provided the following explanation:

Revised Conclusion:

The code path is real:

- an owner can front-run a same-block `liquidate()` call with `removeValidator()`
- the stale liquidation tx then reverts on outdated cluster state
- if enough unpaid usage has accumulated, the subsequent removal settlement can create an accounting gap against the shared ETH pool

However, this is not an "instant insolvency at first liquidation boundary" issue.

For clusters that become liquidatable via the normal burn-threshold branch, the liquidation threshold period itself acts as a buffer. Once a cluster first becomes liquidatable, it still has roughly one threshold-period worth of additional runway before settlement would drive balance all the way to zero. In practice, exploitability therefore depends on a prolonged failure of liquidation liveness, not just on mempool ordering.

Protocol Team / QA Assessment:

Liquidator bots are a critical protocol component, not an optional convenience. The protocol is intentionally designed around:

- permissionless liquidation
- active liquidator monitoring
- a liquidation threshold period that provides a safety margin when liquidators are delayed, miss a block, or encounter short-term operational issues

Under these assumptions:

- a liquidatable cluster should be picked up quickly by liquidators
- the threshold-period buffer should prevent the cluster from remaining underfunded long enough for `removeValidator()` to settle all the way to zero
- if liquidators fail for the entire buffer window, that represents a broader operational problem more severe than this individual finding

File(s) affected: `contracts/modules/SSVValidators.sol`, `contracts/modules/SSVClusters.sol`

Description: A cluster owner can avoid liquidation by removing validators before liquidation occurs.

Recommendation: If this remains accepted behavior, maintain active monitoring and alerting for this pattern in ETH payment flows.

SSV-11

Legacy Fee Requests May Execute After Upgrade With Incompatible Fee Scale

• Low ⓘ Mitigated

Update

Addressed in: `21b6519da3c9f87f79543e932a2dbb664effb3a5` . The client states that `UPGRADE_TIMESTAMP` will be `24727500` , but this value appears to be a block height rather than a timestamp. The client needs to ensure that `UPGRADE_TIMESTAMP` is set to the upgrade timestamp + `declareOperatorFeePeriod` as a timestamp (not a block height).

File(s) affected: `contracts/modules/SSVOperators.sol`

Description: `executeOperatorFee` tries to block legacy fee declarations, but the guard is based on `approvalBeginTime` , not on when the request was declared. This creates a timeline gap:

1. Operator declares the fee before the upgrade.
2. `declareOperatorFeePeriod` pushes `approvalBeginTime` to after `UPGRADE_TIMESTAMP` .
3. Request passes the legacy guard in `executeOperatorFee` because `approvalBeginTime > UPGRADE_TIMESTAMP` .

If that pre-upgrade request value was created under legacy fee-scale assumptions (SSV-style packing) and is later executed under post-upgrade ETH fee interpretation, the same stored integer can map to a different effective fee. This can apply unintended operator fees until manually corrected.

Recommendation: Gate execution with an upgrade buffer window:

1. Define a fixed legacy cutoff at upgrade time, e.g. `legacyCutoff = UPGRADE_TIMESTAMP + declareOperatorFeePeriodAtUpgrade` (captured once during upgrade).
2. In `executeOperatorFee` , reject requests with `approvalBeginTime <= legacyCutoff` so pre-upgrade declarations cannot execute post-upgrade through delayed approval windows.

SSV-12

Liquidation Fallback Branch Incorrectly Adds `operatorEthVUnits` in Sub-Baseline Case

• Low ⓘ Acknowledged

Update

The client acknowledged and provided the following explanation:

The auditor's finding is technically correct, but:

1. Current invariants prevent execution - Oracle validation guarantees `vUnitsCluster >= baselineVUnits`
2. Defensive code intent - The fallback exists for safety, not expected execution
3. No risk under current design - The branch is mathematically unreachable given EB constraints
4. Code clarity trade-off - Adding a revert or removing the branch adds complexity for a case that cannot occur

File(s) affected: `contracts/modules/SSVClusters.sol`

Description: In `_executeLiquidation` , the fallback branch for `vUnitsCluster < baselineVUnits` adds `deviation` to `operatorEthVUnits` :

```
if (moreThanBaseline) { operatorEthVUnits -= deviation; } else { operatorEthVUnits += deviation; }
```

`operatorEthVUnits` is intended to represent only deviation above baseline. Adding to it in a sub-baseline scenario creates artificial positive deviation for operators and can overstate future operator ETH accrual once operators participate in later active clusters.

Current invariants (`EBBelowMinimum`) make this branch unlikely in normal operation, but if the branch becomes reachable due to future parameter changes, migration edge cases, or storage corruption, operator accounting can be inflated.

Recommendation: Treat `vUnitsCluster < baselineVUnits` as unreachable for operator deviation accounting: either revert explicitly or skip operator deviation addition in that branch.

SSV-13 Operator Registration Can Be DoSed

• Low ⓘ Acknowledged

Update

The client acknowledged and provided the following explanation:

As stated in a previous audit, if it happens to one of the operators of the owner, she can make a new operator pub / private key and re-register, as the operator do not have clients yet to have to worry about trying to migrate them.

File(s) affected: `contracts/modules/SSVOperators.sol`

Description: The `SSVOperators.registerOperator(publicKey,...)` registers an operator, avoiding duplicates by checking `operatorsPKs[keccak256(publicKey)] != 0`. This control, however, is not enough.

A malicious actor can, for instance, front-run an honest operator registration, setting the public key of the latter to be the public key of the malicious operator. Although an operator could still register itself by simply issuing a new public key, if the number of operators becomes particularly small, it opens the room for a denial-of-service attack, preventing the SSV network to grow its set of operators. Depending on the number of operators, preventing registration could disrupt the entire network.g., if the number of operators drop below the minimum threshold to build the smallest possible cluster.

Recommendation: Although this issue was previously acknowledged by the SSV team in a past report, the risk of a potential denial-of-service attack vector is still a concern. We encourage reassessing risk. Similar to our past recommendation, we suggest ensuring that the operator's public key is validated to be the owner's (see scheme below). Only then proceed to allow its registration.

Scheme:

- PK(holder): public key
 - SK(holder): secret key
 - Sign(key, m): signs a message m using a specific method
 - Verify(key, m, s): checks if s is indeed signed by whoever is claiming so
- To prove the owner owns the BLS key:
- Owner creates two signatures
 - S1 = Sign(SK(validator), PK(owner))
 - S2 = Sign(SK(owner), PK(owner)).
 - Owner submits PK(owner), PK(validator), S1, and S2 to the off-chain component
 - Off-chain component checks that both conditions pass:
 - Verify(PK(validator), PK(owner), S1)
 - Verify(PK(owner), PK(owner), S2)
 - If so, off-chain component approves the operator at the smart-contract level.

SSV-14

Existing Registration Setup Enables Phantom Operators to Extract Fees and Degrade Cluster Fault Tolerance

• Low ⓘ Acknowledged

Update

The client acknowledged and provided the following explanation:

No bonding on operator registration by design.

File(s) affected: `modules/SSVOperators.sol` ,

Description: `registerOperator` imposes no bond, stake, or proof of operation — the only cost to an attacker is essentially a gas fee. An adversary can therefore register any number of *phantom* operators: accounts that appear as valid public operators on-chain but run no node infrastructure.

A phantom operator is indistinguishable from a legitimate one at both the contract level and the off-chain performance layer. A newly registered phantom and a newly registered honest operator share the same on-chain state and the same absence of performance history — there is no signal, on-chain or off, that separates them until the honest operator has accumulated a track record. This window of indistinguishability is precisely when a cluster owner is most vulnerable: attracted by a low or zero fee, they may include a phantom before any performance data exists to warn them. Once included, the cluster's threshold BLS signature scheme degrades silently: the phantom holds a key share but never participates, reducing effective fault tolerance without any on-chain signal.

Although the SSV protocol delegates long-term performance enforcement to an off-chain performance index, which acts as a partial countermeasure for established operators, three on-chain risks remain:

1. **No ejection mechanism:** the cluster owner has no on-chain way to force a phantom out of a cluster. A phantom that registered solely to harm a target has no incentive to leave, as remaining active continues to extract fees at zero cost. The only recourse is to remove every validator from the cluster to expel the phantom one.
2. **Cost asymmetry:** registering a phantom costs the attacker a fixed gas fee. Forcing the victim to migrate a cluster costs gas proportional to the number of validators.
3. **Direct fee extraction:** on-chain fee accounting in `updateClusterOperators` charges the cluster for every active operator based purely on `ethFee * blockDiff` , with no attestation proof or performance check. A phantom operator that registers with a non-zero fee and is included in a cluster accrues the exact same ETH fees as a legitimate operator running real infrastructure. The phantom owner can then call `withdrawOperatorEarnings` to collect, converting what would otherwise be a pure griefing attack into a profitable one — the attacker earns real ETH at zero operational cost.

Recommendation: The most effective on-chain mitigation is a **registration bond** — an ETH or SSV deposit required at `registerOperator` that is forfeited (or locked) until the operator is removed with a clean exit. A meaningful bond makes phantom registration economically irrational.

A complementary measure is an **operator ejection mechanism** that allows a cluster owner to remove a non-performing operator from their cluster without requiring the operator's cooperation, directly neutralising the cost-asymmetry risk.

SSV-15

Deferred Reward Accounting Exposes Stakers to Eth Price Volatility

• Low ⓘ Fixed

✓ Update

The client provided the following explanation:

INVALID - No code changes required. Confirmed as false positive.

The audit finding is based on incorrect execution order analysis. `_syncFees()` is called before balance decrements in `claimEthRewards()`, so no fees are deferred or lost.

This issue is a false positive and is kept in the report only to preserve the original issue order.

File(s) affected: `contracts/modules/SSVStaking.sol`

Description: The staking contract distributes ETH rewards using a cumulative index (`accEthPerShare`) updated in `_syncFees()`. However, the index only increases when the total staking pool balance exceeds its previous high-watermark. Otherwise, the function returns:

```
if (current.lte(previous)) {
    s.stakingEthPoolBalance = current;
    return;
}
```

If a large ETH claim reduces the pool balance, newly generated rewards first replenish the claim before they are reflected in `accEthPerShare`. During this window, users can still call `claimEthRewards()`, but the rewards generated in that period are not yet included in the claimable amount.

The rewards are not lost — they are deferred and will become claimable once the pool balance surpasses the previous watermark and `_syncFees()` updates the index.

Settlement timing depends on liquidity recovery rather than reward generation. This exposes users to ETH price volatility between reward generation and reward realization.

Recommendation: Do not lower `stakingEthPoolBalance` when `current <= previous`. Simply return without updating the watermark:

```
if (current.lte(previous)) {
    return; // don't lower the watermark
}
```

When `current` temporarily dips below `previous` (e.g., after a large claim), the watermark stays put. Once `current` recovers and exceeds `previous`, the full delta — including earnings generated during the recovery period — is distributed to stakers immediately. No new storage variables or structural changes are required.

SSV-16

Non-Standard ERC20 Tokens Trapped in `SSVStaking` Contract

• Low ⓘ Fixed

✓ Update

Addressed in: `583ea8b1e93c94f1d04bc774d9167a2552ad550a`.

The client fixed it by using `SafeERC20.safeTransfer` in `rescueERC20()`, which supports non-standard ERC20s that return no value, and validated it with tests covering non-standard token rescue behavior.

File(s) affected: `contracts/modules/SSVStaking.sol`

Description: The function `rescueERC20()` transfers tokens from the `SSVStaking` contract, assuming the ERC20 tokens adhere to the standard interface. The contract expects `IERC20.transfer()` to return `true` upon successful token transfer, otherwise the call will revert with the error `TokenTransferFailed`. However, in cases where the token returns no value (e.g., USDT), the token transfer will always fail.

```
// SSVStaking.sol#L150

function rescueERC20(address token, address to, uint256 amount) external nonReentrant {
    ...

    // ⚠ Non-standard ERC20 tokens that do not return
```

```
// a value during transfer() will always fail!
if (!IERC20(token).transfer(to, amount)) {
    revert TokenTransferFailed();
}

...
}
```

Recommendation: Replace `if(!IERC20(token).transfer(to, amount)) { ... }` with OpenZeppelin's `SafeERC20` library, which properly handles transfers of standard and non-standard ERC20 tokens.

SSV-17 Stale Cluster Effective Balance Updates

• Low ⓘ Fixed

✓ Update

Addressed in: `135c880b495463a4a0bd310b9eb3c1d57ccf5909`.

The client fixed it by requiring `updateClusterBalance()` to use the latest committed root (`MustUseLatestRoot`), so an older/stale root cannot be used to keep a lower effective balance and underpay fees. The client validated it with tests that stale-root updates revert, replayed roots revert, and only latest-root updates are accepted.

File(s) affected: `contracts/modules/SSVClusters.sol`

Description: If the effective balance of a cluster does not change between Merkle root commits, there is no incentive to update the cluster's effective balance using the latest Merkle root. However, if the cluster's unchanged effective balance is included in the following commits and eventually increased, the cluster validator could update the cluster's effective balance with stale Merkle roots (reflecting a lower effective balance) and pay lower operator and protocol fees. Currently, this issue is mitigated because the state variable `minimumBlocksBetweenUpdates` is never set, meaning the delay between cluster effective balances is 0 blocks. However, if `minimumBlocksBetweenUpdates` is ever set, effective balance updates could be continuously delayed with stale balances.

Exploit Scenario

1. Assume the global `minimumBlocksBetweenUpdates` is set at 1,000 blocks.
2. Merkle root is commit at block 1,000,000. The effective balance of cluster 5 can be proved as 120 ETH.
3. At block 1,000,005, `SSVClusters.updateClusterBalance()` is called, updating the cluster's effective balance on-chain.

```
seb.clusterEB[5] --> ClusterEBSnapshot {
    vUnits: ((120 * 10_000) - 1) / (32 + 1) = 33_363,
    lastRootBlockNum: 1_000_000,
    lastUpdateBlock: 1_000_005
}
```

4. Every day (7,200 blocks), oracles commit a new Merkle root for all of the clusters' effective balances.
5. Assume the effective balance of cluster 5 remains the same for 30 days. Therefore, there is no incentive to call `SSVClusters.updateClusterBalance()`.
6. On the 31st day, the effective balance of cluster 5 increases to 200 ETH. The oracles commit a Merkle root reflecting this increase at block 1,223,200.
7. At block 1,223,205, the validator calls `SSVClusters.updateClusterBalance()` to prove the cluster's effective balance at block 1,007,200, to avoid paying operators higher fees.

```
seb.clusterEB[5] --> ClusterEBSnapshot {
    vUnits: 33_363,
    lastRootBlockNum: 1_007_200,
    lastUpdateBlock: 1_223_205
}
```

8. Now the effective balance for cluster 5 cannot be updated again until after block $1,223,205 + 1,000 \rightarrow 1,224,205$ due to the check `_verifyEBUpdateFrequency()` in `updateClusterBalance()`.
9. At block 1,224,206, the validator calls `SSVClusters.updateClusterBalance()` to prove the cluster's effective balance at block 1,014,400. Again, the effective balance remains the same and is not updated to the current balance.

```
seb.clusterEB[5] --> ClusterEBSnapshot {
    vUnits: 33_363,
    lastRootBlockNum: 1_014_400,
    lastUpdateBlock: 1_224_206
}
```

10. At block 1,250,000, the cluster owner withdraws ETH from their validator, reducing the effective balance to 32 ETH. This effective balance is reflected in the Merkle root committed by the oracles in block 1,252,000.

11. The validator continues to update their cluster's effective balance using stale Merkle roots immediately after the `minimumBlocksBetweenUpdates` passes from the last update until the Merkle root committed right before their cluster's effective balance increases (block 1,216,000).

```
seb.clusterEB[5] --> ClusterEBSnapshot {
  vUnits: 33_363,
  lastRootBlockNum: 1_216_000,
  lastUpdateBlock: 1_253_235
}
```

12. At block 1,254,236, the validator calls `SSVClusters.updateClusterBalance()` to prove the cluster's effective balance at block 1,252,000, which decreases the clusters effective balance to 32 ETH.

```
seb.clusterEB[5] --> ClusterEBSnapshot {
  vUnits: ((32 * 10_000) - 1) / (32 + 1) = 9_696,
  lastRootBlockNum: 1_252_000,
  lastUpdateBlock: 1_254_236
}
```

13. The operators and protocol are never properly compensated for the cluster's actual effective balance (200 ETH) between Merkle root commits between blocks 1,223,200 and 1,244,800. Instead, their fees are calculated based on stale effective balances (120 ETH).

Recommendation: In `updateClusterBalance()`, validate the Merkle proof is verified using the latest Merkle root committed by the oracles. Alternatively, remove the `_verifyEBUpdateFrequency()` check from `updateClusterBalance()` to prevent delayed effective balance updates.

SSV-18

Direct Liquidations Do Not Consider Effective Balance Updates

Low ⓘ

Acknowledged

i

Update

The client acknowledged and provided the following explanation:

Accepted residual fee variance risk

- The current architecture keeps likelihood low:
- Oracles are permissioned.
- Root updates are expected 3 times per day.
- A company-run instance also updates cluster balances when they change.
- If this edge case occurs, the potential missed fees are treated as acceptable residual risk for v2.0.0.
- No protocol code change is planned at this time.

File(s) affected: `contracts/modules/SSVClusters.sol`

Description: Liquidations that occur via `SSVClusters.liquidate()` do not update the cluster's effective balance, which may be stale at the time of liquidation. If a cluster's effective balance stored in the `SSVNetwork` contract is lower than its effective balance reflected by the latest Merkle root committed by the oracles, cluster operators and the SSV protocol will lose out on potential fees as the incorrect effective balance is used during fee calculations. The lost fees to the operators and the protocol are redirected to the liquidator, who will receive the cluster's remaining balance.

Recommendation: Consider requiring liquidators to update clusters' effective balances when liquidating via `liquidate()`. Alternatively, document that operators should monitor the freshness of their cluster's effective balances to ensure they are properly compensated in the event of a liquidation.

SSV-19

replaceOracle Allows Out-of-Set Oracle IDs To Vote in Root Commitments

Low ⓘ

Fixed

✓

Update

Addressed in: `58984fe4d647e7eb5a373c6e18b61e9869518aa0` .
The oracle set is hard-capped to 4 slots, so `replaceOracle()` can rotate oracle addresses but cannot change oracle count.

File(s) affected: `contracts/modules/SSVDAO.sol`, `contracts/libraries/storage/SSVStorageStaking.sol`

Description: `replaceOracle` validates only that `oracleId != 0`; it does not verify that `oracleId` belongs to the configured default oracle set (`defaultOracleIds`). As a result, the owner can assign `newOracle` to an ID that is outside the intended active set. `commitRoot` then authorizes that oracle via `oracleIdOf[msg.sender] != 0`, so the out-of-set oracle can participate in voting. At the same time, vote weight is still derived from `defaultOracleIds.length`, creating a mismatch between who is authorized to vote and what set the quorum math assumes.

Recommendation: Enforce one canonical active-oracle set for both authorization and quorum math. Specifically:

1. In `replaceOracle`, require `oracleId` to be a member of `defaultOracleIds` (or an equivalent active-ID mapping).
2. In `commitRoot`, authorize only IDs that are members of that same active set.
3. Add invariants/tests ensuring any `oracleIdOf[addr] != 0` is always in the active set, and replacement with out-of-set IDs reverts.

SSV-20

Duplicate BLS Key Registration Across Owners Exposes Validators to Beacon Chain Slashing

• Informational ⓘ Acknowledged

Update

The client acknowledged and provided the following explanation:

It is up to the cluster owners to run alerting tools that can detect this situation.

File(s) affected: `contracts/libraries/ValidatorLib.sol`

Description: In `ValidatorLib.registerPublicKey`, the validator uniqueness check is keyed on `keccak256(publicKey, owner)`:

```
bytes32 hashedPk = keccak256(abi.encodePacked(publicKey, owner));
if (s.validatorPKs[hashedPk] != bytes32(0)) {
    revert ValidatorAlreadyExistsWithData(publicKey);
}
```

Because the key includes the `owner` address, the **same BLS public key** can be registered by two different addresses under two separate clusters, potentially with entirely different operator sets. On the Beacon Chain the BLS public key is seen as a single validator, but two clusters will attempt to produce attestations for it. Stated otherwise, if the same validator BLS key is simultaneously operated by two independent clusters with different operator sets, each cluster can independently reach quorum and produce attestations for the validator's assigned duty.

Because the clusters do not share consensus coordination, they may observe different chain states (e.g., due to network latency, client diversity, or temporary reorgs). In such a case, both clusters can finalize and sign distinct attestation data for the same target epoch.

Under Ethereum's PoS rules, submitting two distinct attestations for the same target epoch constitutes a double vote, which is a slashable offense. Therefore, concurrent operation of the same validator key across multiple independent clusters introduces a critical slashing risk.

Recommendation: Assuming that the identity of a validator requires hashing `publicKey+owner` to prevent one from frontrunning the registration of a BLS key, then the code is correct. We then recommend introducing off-chain monitoring to alert when the same BLS key appears in multiple clusters, in addition to educating users of the consequences of such a setup.

SSV-21

Operator Onboarding May Lead to Increasing Centralization

• Informational ⓘ Acknowledged

Update

The client acknowledged and provided the following explanation:

No bonding on operator registration by design.

File(s) affected: `contracts/modules/SSVOperators.sol`

Description: Operators are incentivized to behave honestly, otherwise, they are unlikely to be chosen as part of a cluster and earn fees. Stated otherwise, validators create clusters by selecting operators with a good **performance record**.

New public operators, however, lack any performance record. That forces them to charge lower fees to incentivize potential validators to select them as part of a cluster. The same tactic, however, can be used by malicious operators (see issue *Permissionless Registration Enables Phantom Operators to Extract Fees and Degrade Cluster Fault Tolerance*). Consequently, validators may increase their level of suspicion before adding a low-fee operator. Validators may then end up relying on operators with an existing track record or use verified ones.

In the described scenario, new operators struggle to participate in the network unless they become verified. As more operators join the network as verified ones, the level of centralization grows, and so does censorship.

Recommendation: Require a meaningful ETH/SSV deposit at `registerOperator` that is locked until the operator exits cleanly. This raises the cost of Sybil-style phantom registration and provides a credible commitment signal. New operators with skin in the game are immediately more trustworthy than anonymous zero-cost registrations. This also addresses the *Phantom Operators* finding.

Auditor Suggestions

S1 Improve Error Handling

Fixed

✓ Update

Addressed in: `33cf8429497bc88aaacdd16445d0782fc602178c` .

File(s) affected: `contracts/libraries/ValidatorLib.sol` , `contracts/modules/SSVClusters.sol`

Description: It is good practice to throw relevant and descriptive errors to improve code readability and debugging. The following list identifies areas of the code that could benefit from improved error handling:

- 1. **Fixed** `ValidatorLib.registerPublicKey()` : The error `ValidatorAlreadyExistsWithData` only includes the validator public key. Since the check looks up the hash of the validator's public key and the owner address, the error could be more descriptive by including the `owner` address.
- 2. **Fixed** `SSVClusters.bulkRemoveValidator()` , `SSVClusters.exitValidator()` , and `SSVClusters.bulkExitValidator()` : The validator data stored in `s.validatorPKs` is not directly validated to exist by checking if it is not `bytes32(0)` . In these functions, `ValidatorLib.validateCorrectState()` will throw an error, `IncorrectValidatorStateWithData` , which is misleading since the validator does not exist. This is inconsistent with the function `SSVClusters.removeValidator()` , which directly validates that the validator data is not `bytes32(0)` and throws the more precise error `ValidatorDoesNotExist` .

Recommendation: Consider improving error handling using the recommendations mentioned.

S2 Misleading Event Emission

Fixed

✓ Update

Addressed in: `0115cd8cae4f6520ccd56101493d5251190d4495` .

File(s) affected: `contracts/modules/SSVDAO.sol`

Description: The function `replaceOracle()` emits the event `OracleReplaced` when `oldOracle == newOracle` then returning without updating any state. Emitting the `OracleReplaced` event may be misleading, as the oracle was not replaced.

Recommendation: Consider either returning without emitting the `OracleReplaced` event when `oldOracle == newOracle` or throwing an error to indicate to the caller that the state is unchanged.

S3 Incorrect Code Comment

Fixed

✓ Update

Addressed in: `494abb8b769c25f8a43854c2fe5c1ebb38f7e19c` .

File(s) affected: `contracts/libraries/storage/SSVStorageReentrancy.sol` , `contracts/libraries/SSVReentrancyGuardLib.sol`

Description: In `SSVStorageReentrancy` , the code comment in the `StorageReentrancy` struct states "*The current reentrancy status (0 = non-entered, 1 = entered)*". However, in `SSVReentrancyGuardLib` , the actual values used are:

```
uint256 private constant NOT_ENTERED = 1;
uint256 private constant ENTERED = 2;
```

Recommendation: Update the code comment in `SSVStorageReentrancy` to use values consistent with `SSVReentrancyGuardLib` .

S4 Gas Savings

Fixed

✓ Update

Addressed in: `0ff85f289838e3d63fe35adc7f49d45ee8e05094` .

File(s) affected: `contracts/modules/SSVStaking.sol` , `contracts/modules/SSVClusters.sol` , `contracts/libraries/SSVReentrancyGuard.sol`

Description: Redundant checks and inefficient code can lead to excessive gas costs during execution. The following list identifies potential gas savings:

1. **Fixed** `SSVStaking.stake()` checks that the stake `amount` is non-zero and at least `MINIMAL_STAKING_AMOUNT` (1e9). Consider removing the redundant zero check since the `MINIMAL_STAKING_AMOUNT` check already covers the case where `amount` is zero.
2. **Fixed** In `SSVClusters._updateClusterBalanceInternal()` at L391-392, `seb.clusterEB[clusterId].vUnits` is read from storage, held in memory in `storedVUnits` , then held in memory again in `effectiveOldVUnits` . `storedVUnits` is never referenced again. Remove the line `uint64 effectiveOldVUnits = storedVUnits` and replace references to `effectiveOldVUnits` with `storedVUnits` .
3. **Fixed** To reduce the deployed contract size, remove the unused function `SSVReentrancyGuardLib._reentrancyGuardStorageSlot()` .

Recommendation: Consider implementing the gas savings.

S5 Missing Input Validation

Mitigated

✓ Update

Addressed in: `00859f1b00f9cf7b44d911b43cd056821b984953` .

File(s) affected: `contracts/libraries/CoreLib.sol`

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error. Specifically, the following function arguments could benefit from greater validation:

1. **Acknowledged** `SSVDAO.updateNetworkFee()` : Validate `fee` is below a reasonable upper-bound, or implement a similar process to operator fee increases that can only be done incrementally
2. **Acknowledged** `SSVDAO.updateNetworkFeeSSV()` : Validate `fee` is below a reasonable upper-bound or implement a similar process to operator fee increases that can only be done incrementally
3. **Fixed** `SSVDAO.updateOperatorFeeIncreaseLimit()` : Validate `percentage` is below a reasonable upper-bound.
4. **Acknowledged** `SSVDAO.updateDeclareOperatorFeePeriod()` : Validate `timeInSeconds` is within a reasonable range, long enough to allow validators to respond to fee changes, and short enough that operators can update their fees within a reasonable time.
5. **Acknowledged** `SSV.updateExecuteOperatorFeePeriod()` : Validate `timeInSeconds` is within a reasonable range.
6. **Acknowledged** `SSV.updateMinimumLiquidationCollateral()` : Validate `amount` is within a reasonable range. High enough that liquidators can act before clusters become insolvent, but low enough that clusters are not prematurely liquidated.
7. **Acknowledged** `SSV.updateMinimumLiquidationCollateralSSV()` : Validate `amount` is within a reasonable range. High enough that liquidators can act before clusters become insolvent, but low enough that clusters are not prematurely liquidated.
8. **Fixed** `SSV.updateMaximumOperatorFee()` : Validate `maxFee` is below a reasonable upper-bound and greater than `minimumOperatorEthFee` .
9. **Fixed** `SSV.updateMinimumOperatorEthFee()` : Validate `minFee` is above a reasonable lower-bound and less than `operatorMaxFee` .
10. **Acknowledged** `SSV.setUnstakeCooldownDuration()` : Validate `duration` is less than a reasonable upper-bound.
11. **Acknowledged** `CoreLib.transferBalance()` : Validate the `to` address is not `address(0)` .
12. **Acknowledged** `SSVClusters.deposit()` : Validate `amount` is non-zero.

Recommendation: We recommend adding the relevant checks.

S6 Unstake Cooldown Unit Mismatch (Docs Say Blocks, Code Uses Timestamps)

Mitigated

i Update

The client acknowledged and provided the following explanation:

Documents have been updated.

Although the documentation's initial value states 7 days, the description still references blocks.

File(s) affected: `contracts/modules/SSVStaking.sol`

Description: The SSV upgrade [proposal](#) defines `cooldownDuration` as block-based, but implementation is time-based:

1. `requestUnstake` sets `unlockTime = block.timestamp + cooldownDuration`.
2. `withdrawUnlocked` checks `unlockTime <= block.timestamp`.

If governance/configuration is set assuming block units (for example, `50190` for `7daysbyblockcount`), `therealcooldown` becomes `13.9` hours because the contract interprets the value as seconds.

Recommendation: If the intended unit is seconds, update documentation and parameter naming to make units explicit; if the intended unit is blocks, change implementation to block-based unlock logic.

S7 Code Quality Improvements

Mitigated



Update

Addressed in: `baf079d5474f142db57be4d87d832101201d8b3b`.

- S7.1. **Fixed** Although it might be better to keep the `cluster.validateClusterIsNotLiquidated()` check and remove the `cluster.active` checks later in the function. Throwing a `ClusterIsLiquidated` error seems clearer than an `InsufficientBalance` error since inactive clusters always have a `balance == 0`.
- S7.5. **Mitigated** Guard added to `reactivate` but not `migrateClusterToETH`.

File(s) affected: `contracts/libraries/ClusterLib.sol`, `contracts/modules/SSVClusters.sol`, `contracts/modules/SSVStaking.sol`, `contracts/SSVNetwork.sol`

Description: The following improvement opportunities can be consolidated into one code-quality workstream:

1. **Fixed** `getClusterData` infers version by storage-check order and does not explicitly reject the contradictory state where both cluster mappings are populated for the same `clusterId`.
2. **Fixed** `withdraw` contains redundant `cluster.active` conditionals after `validateClusterIsNotLiquidated()` already establishes the active precondition.
3. **Fixed** `stake` validates both `amount == 0` and `amount < MINIMAL_STAKING_AMOUNT`, even though the positive hardcoded minimum already covers zero.
4. **Fixed** `updateClusterBalance` mixes active and inactive cluster EB-update semantics in one flow. A dedicated inactive-cluster EB update path would make it explicit that deactivated clusters update only EB snapshot fields (no operator/DAO delta accounting, no liquidation checks).
5. **Mitigated** Reentrancy guard usage is inconsistent across state-heavy cluster flows (`liquidate`, `withdraw`, `updateClusterBalance` are guarded, while `reactivate` and `migrateClusterToETH` are not). Even if not currently exploitable, aligning guard policy improves readability and future-change safety.
6. **Acknowledged** `deposit` allows funding liquidated clusters, while `withdraw` blocks liquidated clusters. This is valid when deposit is intended as reactivation funding, but the behavior is implicit and can be confusing without explicit documentation.

Recommendation:

1. Add an explicit invariant check in `getClusterData` to revert when both mappings are non-zero for the same cluster.
2. Remove redundant `cluster.active` checks in `withdraw` while preserving necessary guards.
3. Keep a single minimum-amount validation path in `stake`.
4. Refactor `updateClusterBalance` into explicit active/inactive subpaths (or a dedicated `updateInactiveClusterEB` helper) so inactive EB refresh behavior is unambiguous and easier to test.
5. Define and apply a consistent `nonReentrant` policy for cluster lifecycle functions, and add tests that assert expected behavior for guarded vs unguarded paths.
6. Clarify liquidated-cluster deposit semantics in docs/events (or enforce guardrails) so users understand that deposited ETH is tied to reactivation flow and not directly withdrawable while liquidated.

S8 Remove Unchecked Arithmetic

Fixed



Update

Addressed in: `075d2084af9db9ee76f29fee58abae5ff590802a`.

File(s) affected: `contracts/libraries/OperatorLib.sol`, `contracts/modules/SSVClusters.sol`

Description: The codebase contains instances of unchecked arithmetic, specifically for loop counter increments. Since [Solidity version 0.8.22](#), the compiler automatically optimizes simple loop counter increments without requiring explicit `unchecked` blocks.

Recommendation: Remove all instances of `unchecked { i++; }` or similar constructs in loops throughout the codebase.

S9 `replaceOracle` No-Op Replacement Can Emit a Misleading Change Event

Fixed

✓ Update

Addressed in: 0115cd8cae4f6520ccd56101493d5251190d4495 .

File(s) affected: contracts/modules/SSVDAO.sol

Description: In `replaceOracle` , when `oldOracle == newOracle` , the function emits `OracleReplaced` and returns without changing state. This no-op path can create ambiguous governance/monitoring signals because the same event is used for both real replacements and no-op calls. It can also hide operator mistakes in runbooks by making a failed update look like a successful change.

Recommendation:

1. Either revert on no-op replacement (e.g., `SameOracleNotAllowed`) to enforce meaningful updates, or
2. Emit a dedicated no-op event (and avoid `OracleReplaced`) so off-chain systems can distinguish real config changes from no-op calls.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: <https://github.com/ssvlabs/ssv-network>

- 9cc...574 ./contracts/SSVNetwork.sol
- ce1...dbc ./contracts/SSVNetworkViews.sol
- 270...f9b ./contracts/SSVProxy.sol
- 296...c77 ./contracts/abstract/SSVReentrancyGuard.sol
- 0ba...2be ./contracts/deprecated/RegisterAuth.sol
- 3cd...b20 ./contracts/interfaces/ICSSVToken.sol
- c7a...305 ./contracts/interfaces/ISSVClusters.sol
- 6ff...edf ./contracts/interfaces/ISSVDAO.sol
- 55d...79b ./contracts/interfaces/ISSVNetwork.sol
- e40...90b ./contracts/interfaces/ISSVNetworkCore.sol
- 76d...2eb ./contracts/interfaces/ISSVOperators.sol
- 2ae...6b7 ./contracts/interfaces/ISSVOperatorsWhitelist.sol
- ea3...07b ./contracts/interfaces/ISSVStaking.sol
- fe8...533 ./contracts/interfaces/ISSVValidators.sol
- 8f7...a48 ./contracts/interfaces/ISSVViews.sol
- 39f...f20 ./contracts/interfaces/external/ISSVWhitelistingContract.sol
- a29...8a6 ./contracts/libraries/ClusterLib.sol

- bfb...8ae ./contracts/libraries/CoreLib.sol
- 4e7...231 ./contracts/libraries/OperatorLib.sol
- 169...5d3 ./contracts/libraries/ProtocolLib.sol
- 985...bae ./contracts/libraries/SSVCoreTypes.sol
- 1cb...b4e ./contracts/libraries/SSVPackedLib.sol
- 4ae...d97 ./contracts/libraries/SSVReentrancyGuardLib.sol
- 5c9...51f ./contracts/libraries/ValidatorLib.sol
- ab0...1a4 ./contracts/libraries/storage/SSVStorage.sol
- 162...7e7 ./contracts/libraries/storage/SSVStorageEB.sol
- 2b3...244 ./contracts/libraries/storage/SSVStorageProtocol.sol
- 96f...e07 ./contracts/libraries/storage/SSVStorageReentrancy.sol
- 3d4...c16 ./contracts/libraries/storage/SSVStorageStaking.sol
- 96f...590 ./contracts/modules/SSVClusters.sol
- 637...448 ./contracts/modules/SSVDAO.sol
- 6bc...782 ./contracts/modules/SSVOperators.sol
- cbb...475 ./contracts/modules/SSVOperatorsWhitelist.sol
- dbc...a2c ./contracts/modules/SSVStaking.sol
- 19f...954 ./contracts/modules/SSVValidators.sol
- aff...310 ./contracts/modules/SSVViews.sol
- d8f...3e4 ./contracts/token/CSSVToken.sol
- cf7...67b ./contracts/token/SSVToken.sol
- 9fe...37e ./contracts/whitelisting/BasicWhitelisting.sol

Test Suite Results

Conservation Law – Multi-Cluster ETH Balance Tracking

- ✓ Maintains ETH conservation across deposits, withdrawals, liquidations, and operator withdrawals (363ms)

ETH Cluster with Explicit EB

Fee Scaling With Explicit EB

- ✓ Fees use old vUnits before EB update and new vUnits after (296ms)

Migration With Explicit EB Deviation Sync

- ✓ migration syncs EB deviation to operators and DAO (391ms)

ETH Cluster Edge Cases

Withdraw From Empty Cluster (validatorCount == 0)

- ✓ Allows full withdrawal from cluster with 0 validators, skipping liquidation check (233ms)

Reactivation With Explicit EB – Deviation Properly Restored

- ✓ Restores EB deviation to operators and DAO on reactivation (276ms)

Withdraw – Operator Snapshots NOT Updated

- ✓ Correctly computes fees over two withdrawals without updating operator snapshots (242ms)

Packing Precision – ETH Values That Aren't Divisible By 100_000

- ✓ Reverts when setting operator ETH fee not divisible by ETH_DEDUCTED_DIGITS (201ms)
- ✓ Accepts operator ETH fee divisible by ETH_DEDUCTED_DIGITS
- ✓ Allows deposit/withdraw of amounts not divisible by ETH_DEDUCTED_DIGITS (53ms)

Liquidation Bounty Exactly Equals Post-Settlement Balance

- ✓ Bounty equals post-settlement balance, not original balance (236ms)

ETH Cluster Lifecycle

ETH Cluster Lifecycle

- ✓ Creates cluster, deposits, advances blocks, withdraws with correct fee deduction (223ms)
- ✓ Deposit at same block as registration – no fee settlement (edge)
- ✓ Multiple deposits accumulate without fee settlement (edge)

Withdraw Exactly To Liquidation Threshold

- ✓ Allows withdraw to exact threshold but rejects 1 more wei
- ✓ ValidatorCount == 0 allows full withdrawal (edge)

Third-Party Liquidation With Bounty

- ✓ Liquidates cluster after balance drops below threshold, liquidator receives bounty (40ms)
- ✓ Owner can always self-liquidate regardless of balance (edge)

Reactivation After Liquidation

- ✓ Full lifecycle: create → liquidate → reactivate → verify fee accrual from reactivation point
- Deposit Into Liquidated Cluster + Reactivation
- ✓ Deposits into liquidated cluster accumulate, reactivation uses sum

ETH Cluster Liquidation

- Cluster at exact threshold is NOT liquidatable by third party
- ✓ Balance == threshold is NOT liquidatable, balance < threshold IS (262ms)
- Liquidation With Explicit EB – Deviation Cleanup
- ✓ Liquidation reverses EB deviation from operators and DAO (41ms)
- Auto-Liquidation via updateClusterBalance
- ✓ EB increase triggers auto-liquidation, bounty goes to updater

Revert – Liquidate Cluster At Exact Threshold

- ✓ Third-party liquidation at exact threshold reverts with ClusterNotLiquidatable (258ms)
- ✓ Self-liquidation at exact threshold succeeds (owner bypass)
- ✓ Third-party liquidation at threshold – 1 wei succeeds

CM-17 & CM-25: SSV Cluster Fee Mechanics

- SSV Fee Accrual – Verify Exact SSV Deduction Over N Blocks
- ✓ Verifies exact SSV fee deduction after 500 blocks with 3 validators (358ms)
- updateClusterBalance on SSV Cluster – EB Snapshot Only
- ✓ Only updates EB snapshot on SSV cluster, no fee settlement (388ms)

SSV Cluster Legacy Operations

- SSV Cluster Self-Liquidation
- ✓ Self-liquidation returns correct SSV balance after fee deduction (333ms)
 - ✓ SSV cluster with near-zero balance – self-liquidation returns 0 SSV (edge)
 - ✓ Already liquidated SSV cluster reverts (edge)
- SSV Blocked Operations
- ✓ ETH operations revert with IncorrectClusterVersion on SSV cluster
 - ✓ migrateClusterToETH succeeds on SSV cluster

Cross-Cutting: Economics

- Full Economic Conservation Law
- ✓ conservation holds after every step (deposit, register, advance, withdraw, operator withdrawal) (240ms)
- Register → Advance → Verify Full Economics (Exact Numbers)
- ✓ Produces exact operator earnings, cluster balance, and DAO earnings after 100 blocks
- Operator Serving Multiple Clusters with Different EBs
- ✓ Correctly accumulates vUnit deviations and adjusts earnings after liquidation (94ms)

Cross-Cutting: Full System Lifecycle

- ✓ Exercises all modules through a complete **system** lifecycle (310ms)

Cross-Cutting: Multi-Step Flows

- Register → EB Update → Fee Change → Liquidation
- ✓ Correctly settles fees across EB update, fee change, and liquidation phases (293ms)
- Sequential Registration – Two Clusters, Same Operators
- ✓ Correctly tracks operator ETH state when two clusters register sequentially (59ms)
- Governance Parameter Change Mid-Operation
- Network Fee Update
- ✓ Correctly applies old fee for first half and new fee for second half (40ms)
- Liquidation Threshold Update
- ✓ Cluster becomes liquidatable when threshold increases (41ms)
- Fee declaration interleavings with EB updates
- ✓ declaring fee, then updating EB, then executing settles pre-exec blocks at old fee (100ms)
 - ✓ executeOperatorFee reverts after operator removal on explicit-EB cluster (61ms)

Cross-Cutting: Staking Integration

- Multi-Staker Revenue Distribution Through State Changes
- ✓ Correctly distributes rewards pro-rata across multiple stakers through EB changes (240ms)
- Staking Rewards Through Liquidation Event
- ✓ Correctly adjusts reward rate when cluster is liquidated (57ms)
- cSSV Transfer Mid-Revenue-Accrual
- ✓ Correctly settles rewards for both parties at pre-transfer balances (46ms)

Cross-Cutting: Validator Count Invariant

- Validator Count Through Liquidation Cycle
- ✓ maintains ethDaoValidatorCount == Σ(active clusters) through register → liquidate → reactivate (349ms)
 - ✓ prevents double-counting when operators are shared across clusters

EB Edge Cases

EB Limits Enforcement

- ✓ Reverts when effectiveBalance is below minimum ($< \text{validatorCount} * 32$) (273ms)
- ✓ Succeeds when effectiveBalance is exactly at minimum ($\text{validatorCount} * 32$) (262ms)
- ✓ Reverts when effectiveBalance exceeds maximum ($> \text{validatorCount} * 2048$) (263ms)
- ✓ Succeeds when effectiveBalance is exactly at maximum ($\text{validatorCount} * 2048$) (264ms)

Merkle Proof Verification

- ✓ Accepts a valid merkle proof (281ms)
- ✓ Reverts with invalid proof path (267ms)
- ✓ Reverts when proof is for a different cluster (275ms)
- ✓ Reverts when EB value doesn't match the proof (263ms)

Update Frequency and Staleness

- ✓ Reverts when update is too frequent ($\text{minBlocksBetweenUpdates}$) (289ms)
- ✓ Succeeds when enough blocks have passed (282ms)
- ✓ First update always passes frequency check ($\text{lastUpdateBlock} == 0$) (259ms)
- ✓ Reverts with MustUseLatestRoot when a newer root has already been committed (292ms)
- ✓ Reverts with StaleUpdate when replaying the latest root after a successful update (271ms)
- ✓ First update always passes staleness check ($\text{lastRootBlockNum} == 0$) (276ms)
- ✓ Reverts with MustUseLatestRoot when trying to use an older root after two updates (304ms)
- ✓ RootNotFound: reverts when no root committed for blockNum (275ms)

Operator vUnit Tracking

Operator vUnit Tracking Across Multiple Clusters

- ✓ Accumulates vUnit deviations from multiple clusters for the same operator (344ms)

EB Updates

First EB Update – Implicit to Explicit (Same vUnits)

- ✓ Transitions from implicit to explicit vUnits with no deviation change (277ms)

EB Increase – Higher Fee Burn Rate

- ✓ Updates vUnits upward and increases the fee burn rate proportionally (285ms)

EB Decrease – Lower Fee Burn Rate

- ✓ Updates vUnits downward and decreases the fee burn rate (280ms)

Auto-Liquidation on EB Increase

- ✓ Auto-liquidates cluster when EB increase pushes balance below threshold (300ms)

Fee Settlement Uses OLD vUnits – No Gap

- ✓ Settles fees with old vUnits before applying new vUnits (292ms)

Oracle Commits

Single Oracle Commit – Below Quorum

- ✓ Stores weight but does not commit root when 1 of 4 oracles votes (221ms)

Quorum Reached – 3 of 4 Oracles Agree

- ✓ Commits root when 3 of 4 oracles vote for the same root
- ✓ Prevents Oracle4 from voting for same block after quorum (StaleBlockNumber)

Conflicting Roots – Separate Weight Tracking

- ✓ tracks weight separately for different roots at the same block

ES-4: Oracle Replacement Mid-Vote

- ✓ Replacement oracle inherits same oracleId and cannot re-vote
- ✓ Old vote's weight still counts toward quorum after replacement

Oracle Edge Cases – Reverts

Stale block number

- ✓ Reverts when blockNum equals latestCommittedBlock
- ✓ Reverts when blockNum is less than latestCommittedBlock
- ✓ Succeeds when blockNum is greater than latestCommittedBlock

Future block number

- ✓ Reverts when $\text{blockNum} > \text{block.number}$
- ✓ Succeeds when $\text{blockNum} == \text{block.number}$ (equality OK)

Zero cSSV supply

- ✓ Reverts when cSSV totalSupply is 0 (189ms)

Double vote

- ✓ Reverts when same oracle votes twice for same (root, blockNum)
- ✓ Allows same oracle to vote for different root at same block
- ✓ Reverts when non-oracle calls commitRoot

Explicit EB vUnits scenarios

- ✓ E-04: higher explicit-EB burn rate is applied after an EB=64 update (253ms)
- ✓ E-06: EB=64 → EB=128 settles at the old rate and then accrues at the higher rate (42ms)
- ✓ E-08: explicit EB=32 → EB=64 settles at baseline and then accrues at the higher rate (41ms)
- ✓ E-09: 3-validator cluster updated to total EB=96 keeps baseline vUnits and burn rate (48ms)
- ✓ E-11: registering a second validator after EB=64 adds one baseline validator worth of vUnits (39ms)
- ✓ E-13: withdrawing the maximum allowed amount at explicit EB=64 leaves the cluster exactly at the liquidation boundary (55ms)

- ✓ E-14: depositing into an explicit-EB=64 cluster preserves its effective balance and burn rate

- ✓ R-09: register validator (EB=64) → remove operator → deposit
- ✓ RI-02: register validator → remove operator → remove last validator
- ✓ RI-03: register validator → remove operator → withdraw
- ✓ RI-05: register validator → remove operator → reactivate (if liquidated)

Migration SSV → ETH

Basic Migration With SSV Refund

- ✓ Migrates SSV cluster to ETH with correct SSV refund **and** ETH deposit (422ms)
- ✓ Migration with insufficient ETH reverts (edge)

Migration of Liquidated SSV Cluster

- ✓ Migrates liquidated SSV cluster – no SSV refund, emits ClusterReactivated (335ms)

Migration With Mixed Operator ETH State

- ✓ Operators with different ETH fees produce correct cumulative index after migration (337ms)
- ✓ Migration succeeds with default ETH fees (auto-assigned on migration)

Post-Migration ETH Fee Accrual

- ✓ ETH fees accrue correctly after migration, **not** SSV fees (351ms)

Migration Regression: removed operator SSV settlement

- ✓ Baseline: all operators active uses exact SSV refund formula (82ms)
- ✓ Includes removed operator frozen snapshot.index in migration SSV settlement
- ✓ Liquidated cluster migration with removed operator preserves SSV counts **and** skips removed ETH setup
- ✓ Accounts two removed operators with different removal times via frozen indices
- ✓ Removed operator with zero SSV fee creates zero refund delta vs buggy path
- ✓ Removed operator frozen ETH index is **not** charged again on first post-migration settlement (50ms)
- ✓ Assigns default ETH fee on migration when legacy operator had ethFee explicitly reset to zero

Migration Edge Cases

Migration – SSV Refund Is Exactly Correct After Extended Fee Accrual

- ✓ SSV refund matches independent fee calculation after 1000 blocks (340ms)

Migration of Cluster Where Some Operators Were Removed

- ✓ Migration succeeds when Op1 is removed – removed operator is skipped (321ms)

DAO Earnings Settlement During Migration

- ✓ DAO earnings for both SSV **and** ETH are settled during migration (345ms)

Multiple Migrations – Same Operators, Different Clusters

- ✓ Two clusters with same operators migrate correctly without index corruption (372ms)

Revert – Migrate With Insufficient ETH For Liquidation Check

- ✓ Reverts when ETH deposit is below liquidation threshold (330ms)
- ✓ Reverts when ETH deposit is 1 wei below threshold
- ✓ Reverts when ETH deposit is 0

Full End-to-End – SSV Cluster Creation → Fee Accrual → Migration → ETH Fee Accrual → Withdraw → Verify All Balances

- ✓ Verifies complete economic correctness across full lifecycle (365ms)

Operator Economics

Operator Earnings Accumulation **and** Withdrawal

- ✓ Verifies exact earnings math with partial **and** full withdrawal (312ms)

Fee Change During Active Cluster

- ✓ Verifies continuous fee accrual across fee change boundary (63ms)

Multi-Cluster Operator – Earnings From Multiple Clusters

- ✓ Operator earns from two clusters, correct accounting on partial removal (74ms)

Operator Removal After All Validators Removed

- ✓ Removes validators then operator, verifies **final** earnings withdrawal (67ms)

withdrawAllVersionOperatorEarnings – Combined ETH + SSV

- ✓ Withdraws both ETH **and** SSV earnings in single call
- ✓ Only ETH earnings, no SSV – SSV transfer skipped
- ✓ Zero earnings in both versions – no reverts, no transfers

Operator Edge Cases

Operator removal – state verification after removal

- ✓ Removed operator preserves owner but zeros ethSnapshot.block (200ms)

ensureETHDefaults with zero SSV fee – default fee NOT assigned

- ✓ Zero-fee operator stays at zero fee after ETH cluster interaction
- ✓ Zero-fee operator can never increase fee

Precision loss in operator earnings – vUnits division truncation

- ✓ Operator earnings are exact with standard vUnits (no truncation)
- ✓ Precision is exact with standard vUnits regardless of fee magnitude

Operator index frozen after removal – cluster still functions

- ✓ Cluster can remove validators after one operator is removed (45ms)

Concurrent fee changes on multiple operators in same cluster

- ✓ Cluster pays correct blended rate after operator fee changes (45ms)

Operator Lifecycle

Register Operator (Public, Non-Zero Fee)

- ✓ Registers **public** operator with non-zero fee **and** verifies initial state (233ms)
- ✓ Register with fee=0 succeeds, operator is free forever
- ✓ Register with setPrivate=true sets whitelisted flag
- ✓ Register with same pubkey again reverts OperatorAlreadyExists
- ✓ Register with fee **not** divisible by ETH_DEDUCTED_DIGITS reverts MaxPrecisionExceeded

Register Operator (Private, Zero Fee)

- ✓ Registers **private** zero-fee operator **and** verifies fee immutability

ensureETHDefaults – Default Fee Assignment

- ✓ Operator registered with non-zero fee gets correct ethFee
- ✓ Operator registered with fee=0 **and** SSV fee=0 stays free

Operator Fee Declaration -> Wait -> Execution

- ✓ Declares fee, waits, **and** executes within approval window
- ✓ Execute after approval window expires reverts
- ✓ Edge: cancel declared fee clears the request
- ✓ Fee increase exceeding limit reverts FeeExceedsIncreaseLimit

Operator Fee Reduction (Immediate, No Timelock)

- ✓ Reduces fee immediately, preserving earnings at old fee
- ✓ Reduce to exactly current fee reverts FeeIncreaseNotAllowed
- ✓ Reduce to higher fee reverts FeeIncreaseNotAllowed
- ✓ Reducing fee clears pending fee change request

Operator Earnings Accumulation **and** Withdrawal

- ✓ Accumulates earnings **and** supports partial + full withdrawal

Operator Fee Reduction – Legacy SSV Operator Edge Cases

- ✓ Legacy SSV operator can reduce ethFee to 0 without getting default on migration
- ✓ Operator reduces fee immediately after registration (ethSnapshot already initialized)
- ✓ Operator can reduce to 0 then cannot increase via declareOperatorFee

Remove Operator – Full Cleanup **and** Final Withdrawal

- ✓ Removes operator with earnings, transfers funds, **and** cleans up state (86ms)
- ✓ Remove operator with 0 earnings in both versions
- ✓ After removal, registering validator with removed operator reverts
- ✓ Double removal reverts OperatorDoesNotExist

Operator Reverts

Register Validator – Operator Revert Cases

- ✓ Reverts with OperatorAlreadyExists when registering operator with same pubkey (211ms)
- ✓ Reverts with OperatorDoesNotExist when registering validator with removed operator
- ✓ Reverts with CallerNotWhitelistedWithData when registering on **private** operator without whitelist

Operator Remove Revert Cases

- ✓ Reverts with OperatorDoesNotExist when removing non-existent operator
- ✓ Reverts with CallerNotOwnerWithData when non-owner tries to remove operator
- ✓ Reverts with OperatorDoesNotExist when removing already-removed operator

E2E Smoke Test

- ✓ Deploys, registers, mines blocks, **and** computes fees correctly (219ms)

E2E Staking Edge Cases

Accumulator Edge Cases

- ✓ Zero cSSV supply – fees are unclaimable (231ms)
- ✓ accEthPerShare monotonicity – never decreases (57ms)
- ✓ Dust accumulation – dust eventually claimable (50ms)

MAX_PENDING_REQUESTS (2000)

- ✓ Should allow exactly 2000 pending requests **and** revert on 2001 (6203ms)
- ✓ Withdrawing unlocked requests frees slots for **new** requests (6516ms)

MINIMAL_STAKING_AMOUNT

- ✓ Should revert with StakeTooLow for stake(0)
- ✓ Should revert with StakeTooLow for amount below minimum
- ✓ Should succeed at exactly MINIMAL_STAKING_AMOUNT

syncFees() Public Function

- ✓ Should update accEthPerShare without settling any user's rewards (40ms)
- ✓ Anyone can call syncFees (not restricted to stakers)

requestUnstake Followed by Immediate Claim

- ✓ Both requestUnstake **and** claimEthRewards can be called in same block context (42ms)
- ✓ Claiming twice in the same block only pays once (41ms)

E2E Staking Lifecycle

Basic Stake → Earn → Claim Cycle

- ✓ Should allow a user to stake SSV, earn network fee revenue, **and** claim ETH rewards (237ms)
- ✓ Pre-stake fees when cSSV supply is zero are permanently locked

Multiple Stakers – Pro-Rata Distribution

- ✓ Should distribute rewards proportionally: A gets 25%, B gets 75% with 10:30 ratio (47ms)

- ✓ Three stakers split rewards correctly when one unstakes mid-period (64ms)

Stake Timing Matters – Late Joiner

- ✓ Late joiner B does NOT capture fees from before they staked (48ms)

Unstake Request → Cooldown → Withdraw

- ✓ Should lock SSV during cooldown **and** allow withdrawal after (44ms)
- ✓ Rewards are settled with pre-burn balance during requestUnstake (58ms)
- ✓ Cooldown changes do **not** alter reward accrual before **and** after requestUnstake (49ms)
- ✓ Burned cSSV stops earning rewards immediately (38ms)

E2E Staking Rewards

Network Fee Raise staking Rewards

- ✓ Staking rewards increase after updateNetworkFee raises the ETH network fee (268ms)

Network Fee decrease staking Rewards

- ✓ Staking rewards decrease after updateNetworkFee lowers the ETH network fee (50ms)

Zero Network Fee do no generate **new** staking rewards

- ✓ Staking rewards stop after updateNetworkFee sets the ETH network fee to zero (55ms)

Multiple Network Fee Changes staking Rewards

- ✓ Staking rewards track multiple updateNetworkFee changes across consecutive periods (66ms)

Cooldown Increase → Same Staking Rewards

- ✓ Staking rewards stay unchanged after updateUnstakeCooldownDuration increases cooldown (53ms)

Cooldown Decrease → Same Staking Rewards

- ✓ Staking rewards stay unchanged after updateUnstakeCooldownDuration decreases cooldown (71ms)

EB Increase → Higher Network Fees → More Staking Rewards

- ✓ Staking rewards **double** after EB update doubles vUnits (77ms)

Auto-Liquidation Reduces Active Clusters → Less Staking Revenue

- ✓ Staking rewards decrease when a cluster is liquidated (59ms)

Full Staking Reward Math – Worked Example

- ✓ Exact reward calculation for 1 staker, 1 cluster, 1000 blocks

Staking Reward with Multiple Users **and** Precision

- ✓ Rewards split correctly with 3:7 ratio **and** no precision loss for clean division (48ms)
- ✓ Truncation dust is at most 1 wei equivalent per user when using odd supply

Staking with Existing Pre-Upgrade DAO Balance

- ✓ Pre-existing DAO revenue is **not** distributed to first staker (57ms)

EB Update Followed by syncFees – Full Chain

- ✓ Full chain trace: EB update → DAO vUnit change → higher earnings → syncFees → claim (76ms)

E2E Staking Transfers

cSSV Transfer Settles Rewards

- ✓ Transfer settles both sender **and** receiver; pre-transfer revenue goes to sender only (257ms)
- ✓ Stake-transfer-stake cycle preserves reward boundaries across all phases (50ms)
- ✓ Receiver B's userIndex is set to accEthPerShare at transfer time (41ms)

cSSV Transfer – Mint/Burn Do NOT Trigger Hook

- ✓ Mint (via stake) does **not** trigger onCSSVTransfer – from == address(0)
- ✓ Burn (via requestUnstake) does **not** trigger onCSSVTransfer – to == address(0) (39ms)
- ✓ Self-transfer does **not** trigger onCSSVTransfer – from == to
- ✓ Self-transfer keeps reward accrual equal to uninterrupted staking (41ms)
- ✓ Zero-amount transfer does **not** trigger onCSSVTransfer – amount == 0
- ✓ Normal user-to-user transfer DOES trigger onCSSVTransfer (59ms)

Validator Edge Cases

Register Validator – Revert Cases

- ✓ Reverts with EmptyPublicKeysList on bulk register with empty **array** (221ms)
- ✓ Reverts with PublicKeysSharesLengthMismatch on mismatched key/share arrays
- ✓ Reverts with InvalidPublicKeyLength on short **public** key
- ✓ Reverts with InvalidOperatorIdsLength for < 4 operators
- ✓ Reverts with InvalidOperatorIdsLength for 5 operators (not 4,7,10,13)
- ✓ Reverts with UnsortedOperatorsList for unsorted operators
- ✓ Reverts with OperatorsListNotUnique for duplicate operators
- ✓ Reverts with ValidatorAlreadyRegistered when registering same validator twice
- ✓ Reverts with IncorrectClusterState when passing wrong cluster struct

Remove Validator – Revert Cases

- ✓ Reverts with ValidatorDoesNotExist for non-existent validator
- ✓ Reverts with IncorrectValidatorStateWithData when wrong owner removes
- ✓ Reverts with IncorrectClusterState with stale cluster struct
- ✓ Reverts with ValidatorDoesNotExist for bulk remove with empty **array**

Race condition – register **and** remove in same block

- ✓ Register then remove in same block – no **double-counting** (97ms)

Cluster balance underflow protection

- ✓ Cluster balance floors at 0 when fees exceed balance

Exit validator – signal only, no state change

- ✓ exitValidator emits event but makes no state change (38ms)
- ✓ exitValidator reverts with ValidatorDoesNotExist for non-existent validator

- ✓ exitValidator reverts with wrong operator IDs

DAO network fee earnings – consistency with cluster accounting

- ✓ DAO earnings match cluster network fee payments

Operator registration then immediate validator registration – same block

- ✓ Register operators **and** validator in same block – no error from zero blockDiff

Large number of operators (13) – gas **and** correctness

- ✓ Register validator with 13 operators – correct state **and** reasonable gas (75ms)

Validator registration with explicit EB (post-updateClusterBalance)

- ✓ Adding validator to explicit-EB cluster adds default vUnits baseline (67ms)

Validator removal with implicit/explicit EB – full cluster empty

- ✓ Remove last validator – cluster persists with remaining balance (43ms)

Bulk remove validators – multiple removals in one transaction

- ✓ Bulk remove 3 of 5 validators – correct state after (99ms)

Deposit **and** withdraw – no side effects on operator state

- ✓ Deposit **and** withdraw do **not** change operator validator counts (53ms)

Validator Lifecycle

Register Validator – New Cluster with 4 Public Operators

- ✓ Registers validator, verifies default ETH fee applied, fees accrue correctly (263ms)
- ✓ Register on operators with fee=0 – zero fee accrual

Register Validator – Existing Cluster with Fee Settlement

- ✓ Adds validator to existing cluster, settles fees from first period (44ms)

Register Validator on Private Operators

- ✓ Non-whitelisted caller reverts, whitelisted caller succeeds
- ✓ Mix of **public** **and** private operators in same cluster

Bulk Register Validators

- ✓ Bulk registers 3 validators, verifies counts **and** events
- ✓ Bulk register with 0 **public** keys reverts EmptyPublicKeysList
- ✓ Bulk register with mismatched lengths reverts
- ✓ Bulk register with duplicate key reverts

Remove Validator – Fee Settlement **and** Count Adjustment

- ✓ Removes validator from 2-validator cluster, settles fees correctly (84ms)
- ✓ Remove non-existent validator reverts with ValidatorDoesNotExist

Remove Last Validator – Cluster Balance Preservation

- ✓ Removes last validator, cluster persists with remaining balance (43ms)

Full Validator Lifecycle

- ✓ Register → advance → remove → advance → withdraw – verifies complete lifecycle (52ms)
- ✓ Verifies exact fee math with block-precise accounting

SSVNetwork full integration tests made on forked contract

Constructor, initializer **and** upgrades

- Configures SSVNetwork correctly

Function 'registerOperator()'

- Creates **new** operator **and** emits correct event
- Is reverted with 'FeeTooLow' **if** the provided fee is less than minimal allowed
- Is reverted with 'FeeTooHigh' **if** the provided fee is higher than maximum allowed
- Is reverted with 'OperatorAlreadyExists' **if** the **public** key is already registered

Function 'removeOperator()'

- Deactivates the operator **and** emits correct event
- Is reverted with 'OperatorDoesNotExist' **if** operator with passed id is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner

Function 'setOperatorsWhitelists()'

- Whitelists addresses **and** emits correct event
- Whitelists multiple operators for multiple addresses
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'InvalidWhitelistAddressesLength' **if** the **array** of addresses is empty
- Is reverted with 'ZeroAddressNotAllowed' **if** one of addresses is zero address
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is the the operator owner
- Is reverted with 'OperatorsListNotUnique' **if** the **array** of operators has any duplicate
- Is reverted with 'UnsortedOperatorsList' **if** operators are **not** sorted in increasing order

Function 'removeOperatorsWhitelists()'

- Removes addresses from the whitelist **and** emits correct event
- Removes multiple operators for multiple addresses
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'InvalidWhitelistAddressesLength' **if** the **array** of addresses is empty
- Is reverted with 'ZeroAddressNotAllowed' **if** one of addresses is zero address
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is the the operator owner
- Is reverted with 'OperatorsListNotUnique' **if** the **array** of operators has any duplicate
- Is reverted with 'UnsortedOperatorsList' **if** operators are **not** sorted in increasing order

Function 'setOperatorsWhitelistingContract()'

- Registers whitelisting contract, emits correct event **and** allows to whitelist addresses via contract
- Updates whitelisting contract for operators
- Registers whitelisting contract for 10 operators
- Is reverted with 'InvalidWhitelistingContract' **if** the contract does **not** support required interface
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is the the operator owner

Function 'removeOperatorsWhitelistingContract()'

- Removes whitelisting address **and** emits correct event
- Removes whitelisting contract for 10 operators
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** one of operators is **not** registered

Function 'setOperatorsPrivateUnchecked()'

- Changes privacy status **and** emits correct event
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner

Function 'setOperatorsPublicUnchecked()'

- Changes privacy status **and** emits correct event
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner

Function 'declareOperatorFee()'

- Declares **new** fee **and** emits correct event
- Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- Is reverted with 'FeeTooLow' **if** the passed fee is less than minimal
- Is reverted with 'SameFeeChangeNotAllowed' **if** the passed value is the same as current one
- Is reverted with 'FeeTooHigh' **if** the **new** fee is higher than allowed
- Is reverted with 'FeeExceedsIncreaseLimit' **if** the **new** fee exceeds the allowed limit
- Is reverted with 'FeeIncreaseNotAllowed' **if** operators current fee is zero

Function 'cancelDeclaredOperatorFee()'

- Cancels declared fee **and** emits correct event
- Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- Is reverted with 'NoFeeDeclared' **if** no declarations were done before

Function 'executeOperatorFee()'

- Updates operator fee according to a declared one **and** emits the correct event
- Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- Is reverted with 'NoFeeDeclared' **if** no declarations were done before
- Is reverted with 'ApprovalNotWithinTimeframe' **if** execution period is **not** started **or** ended
- Is reverted with 'FeeTooHigh' **if** the maximum fee changed during the execution period

Function 'updateMaximumOperatorFee()'

- Updates maximum fee **and** emits correct event
- Is reverted with 'Ownable: caller is **not** the owner' **if** the caller is **not** the owner

Function 'reduceOperatorFee()'

- Decreases fee **and** emits the correct event
- Is reverted with 'OperatorDoesNotExist' **if** the operator is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- Is reverted with 'FeeTooLow' **if** the passed fee is less than minimum allowed
- Is reverted with 'FeeIncreaseNotAllowed' **if** caller is trying to increase the fee

Function 'withdrawOperatorEarnings()'

- Withdraws operators earnings, update balances **and** emits correct event
- Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** caller is **not** the operator owner
- Is reverted with 'InsufficientBalance' **if** the amount is less than operator earnings

Function 'withdrawAllOperatorEarnings()'

- Withdraws all operators earnings, update balances **and** emits correct event
- Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** caller is **not** the operator owner

Function 'withdrawAllVersionOperatorEarnings()'

- Withdraws all operators earnings **and** emits correct events
- Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** caller is **not** the operator owner

Function 'setFeeRecipientAddress()'

- Emits the correct event with the correct input data

Function 'updateOperatorFeeIncreaseLimit()'

- Changes fee increase limit **and** emits the correct event

```

- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
Function 'updateDeclareOperatorFeePeriod()'
- Changes the fee declare period and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
Function 'updateExecuteOperatorFeePeriod()'
- Changes the fee execute period and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
Function 'updateLiquidationThresholdPeriod()'
- Changes the period and emits correct event
- Is reverted 'NewBlockPeriodIsBelowMinimum' if the passed threshold is less than minimum allowed
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
Function 'updateLiquidationThresholdPeriodSSV()'
- Changes the period and emits correct event
- Is reverted 'NewBlockPeriodIsBelowMinimum' if the passed threshold is less than minimum allowed
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
Function 'updateMinimumLiquidationCollateral()'
- Changes collateral and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
Function 'updateMinimumLiquidationCollateralSSV()'
- Changes collateral and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
Function 'registerValidator()'
- For a new cluster, creates it with a passed validator and emits correct event
- Registers a validator for a new ETH cluster using whitelisting contract
- Registers a validator for a new ETH cluster with one whitelisted operator
- Registers a validator for a new ETH cluster with four whitelisted operators
- Registers a validator into an existing ETH cluster with four whitelisted operators
- Registers a validator into an existing ETH cluster
- Registers a validator into a prefunded ETH cluster with zero additional deposit
- Is reverted with 'InvalidOperatorIdsLength' if the amount of operators is not the allowed one
- Is reverted with 'InvalidPublicKeyLength' if the public key is not 48 bytes
- Is reverted with 'ValidatorAlreadyExistsWithData' if the public key is already registered
- Is reverted with 'IncorrectClusterState' for the new cluster is the cluster data is not
consisting from zeroes
- Is reverted with 'UnsortedOperatorsList' if operators are not sorted in increasing order
- Is reverted with 'OperatorsListNotUnique' if the array of operators has any duplicates
- Is reverted with 'CallerNotWhitelistedWithData' if one of operators did not whitelist the caller
- Is reverted with 'ExceedValidatorLimitWithData' if one of operators will exceed the network limit
- Is reverted with 'InsufficientBalance' if msg value is not enough to cover the validator
Function bulkRegisterValidator()
- Registers bulk of validators, creates a new cluster with the expected data and emits correct
events
- Registers bulk of validators into an existing cluster with one whitelisting contract operator
- Is reverted with 'InvalidOperatorIdsLength' if the amount of operators is not the allowed one
- Is reverted with 'InvalidPublicKeyLength' if one of public keys is not 48 bytes
- Is reverted with 'ValidatorAlreadyExistsWithData' if one of public keys is already registered
- Is reverted with 'IncorrectClusterState' for the new cluster is the cluster data is not
consisting from zeroes
- Is reverted with 'UnsortedOperatorsList' if operators are not sorted in increasing order
- Is reverted with 'OperatorsListNotUnique' if the array of operators has any duplicates
- Is reverted with 'CallerNotWhitelistedWithData' if one of operators did not whitelist the caller
- Is reverted with 'ExceedValidatorLimitWithData' if one of operators will exceed the network limit
- Is reverted with 'InsufficientBalance' if msg value is not enough to cover new validators
- Is reverted with 'EmptyPublicKeysList' if the array of public keys is empty
- Is reverted with 'PublicKeysSharesLengthMismatch' if the array of keys and array of shares have
different length
Function 'removeValidator()'
- Removes validator and emits correct event
- Is reverted with 'ClusterDoesNotExists' if the cluster with this owner and operators does not
exist
- Is reverted with 'IncorrectClusterState' if the cluster data is incorrect
- Is reverted with 'ValidatorDoesNotExist' if the validator was never registered
- Is reverted with 'ValidatorDoesNotExist' if validator is already removed
Function 'bulkRemoveValidator()'
- Is reverted with 'ClusterDoesNotExists' if the cluster with this owner and operators does not
exist
- Is reverted with 'IncorrectClusterState' if the cluster data is incorrect
- Is reverted with 'ValidatorDoesNotExist' if the validator was never registered
- Is reverted with 'ValidatorDoesNotExist' if validator is already removed
Function stake()
- Stakes SSV, mints CSSV to the staker and creates delegation weight
- Is reverted with 'StakeTooLow' if the amount to stake is smaller than minimum allowed

```

- Is reverted with 'StakeTooLow' when caller is trying to stake 0 SSV

Function requestUnstake()

- For full amount, creates unstake request, burns CSSV and removes delegation
- For partial amount, creates unstake request, burns CSSV and removes delegation
- Is reverted with 'MaxRequestsAmountReached' if more than 2000 pending requests
- Is reverted with 'ZeroAmount' if caller is trying to request 0 SSV
- Is reverted with 'UnstakeAmountExceedsBalance' if caller is trying to request more SSV than they

staked

Function 'withdrawUnlocked()'

- Withdraws SSV and emits correct event

Function 'claimEthRewards()'

- Processes only the first claim when two claims are mined in the same block

SSVNetwork Integration – Clusters (Enhanced)

Balance Delta Assertions

- ✓ deposit: verifies exact ETH transfer from depositor to contract (261ms)
- ✓ withdraw: verifies exact ETH transfer from contract to owner (43ms)
- ✓ liquidate: liquidator receives remaining cluster balance (56ms)

Multi-Block Simulation – Cluster Balance Burn

- ✓ Cluster balance decreases exactly by burn rate per block
- ✓ Burn rate scales with validator count (57ms)
- ✓ removeValidator settles exact fee deduction from cluster balance (45ms)

Invariant Checks – Balance Conservation

- ✓ Invariant: Deposited = ClusterBalance + OperatorEarnings + NetworkEarnings (49ms)
- ✓ Invariant: Withdrawal reduces cluster balance exactly
- ✓ Invariant: Deposit increases cluster balance exactly

Liquidation Boundary Tests

- ✓ Cluster is not liquidatable just above threshold
- ✓ Owner can self-liquidate even when not underfunded (38ms)
- ✓ Reactivation requires sufficient balance to avoid immediate liquidation (57ms)

Combined Scenarios – Full Lifecycle Economics

- ✓ Full lifecycle: register → operate → withdraw → deposit → liquidate → reactivate (111ms)
- ✓ Third-party deposit doesn't affect owner's ability to withdraw (40ms)

Edge Cases and Error Conditions

- ✓ Cannot withdraw more than available balance
- ✓ Cannot withdraw if it would make cluster liquidatable
- ✓ Deposit with stale cluster state reverts
- ✓ Cannot reactivate an already active cluster (47ms)
- ✓ Cannot operate on non-existent cluster
- ✓ updateClusterBalance succeeds on a liquidated cluster, emits ClusterBalanceUpdated with cluster

still inactive (67ms)

- ✓ Is reverted with 'InsufficientBalance' when withdrawing from a liquidated cluster with zero balance (45ms)

- ✓ Allows deposit to liquidated cluster and subsequent withdrawal without reactivation (69ms)

- ✓ Reverts withdraw from liquidated cluster when using stale pre-deposit cluster state (54ms)

existing earnings (52ms)

- ✓ Maintains global ETH accounting invariant after liquidated cluster withdrawal (79ms)

- ✓ Allows withdrawal from liquidated cluster even if one operator was removed (65ms)

- ✓ Allows reactivation after partial withdrawal from liquidated cluster (59ms)

ITEST-1 Integration: commitRoot -> updateClusterBalance E2E

✓ 3 oracles commit root, then updateClusterBalance applies EB=64 and doubles post-update fee accrual (266ms)

- ✓ Two clusters update from the same committed root and settle independently per-cluster (78ms)

SSVNetwork Integration – DAO Oracle Quorum

Oracle Quorum – 100% threshold (quorumBps = 10000)

- ✓ First three oracle votes emit WeightedRootProposed; fourth vote commits the root (242ms)

Oracle Quorum – 1 bps minimum threshold (quorumBps = 1)

- ✓ Single oracle vote immediately commits root when quorumBps is 1

Oracle Quorum – oracle replaced between votes

✓ Pre-replacement vote still counts; old oracle loses rights, new oracle gets AlreadyVoted for reused slot

Oracle Quorum – completely new address replaces oracle slot

- ✓ New oracle is blocked from the in-flight vote but has full slot ownership for subsequent blocks

Oracle Quorum – quorumBps changed between votes

- ✓ Lowering quorumBps between two votes causes the second vote to cross the new, lower threshold

Oracle Quorum – conflicting roots for same block

- ✓ First root to reach quorum is committed; further votes on the losing root revert with

StaleBlockNumber

TEST-6 Integration: EB decrease via oracle commitRoot pipeline

- ✓ EB update via oracle commitRoot: RootCommitted emitted, exact fees settled at baseline rate (263ms)
- ✓ EB decrease (64→32 ETH): fees for 14 blocks charged at **double** baseline rate (74ms)

SSVNetwork Integration tests – EB-Weighted Operator Earnings

- ✓ getOperatorEarnings reflects EB=64 uplift (2× vs baseline) after updateClusterBalance (269ms)
- ✓ getOperatorEarnings scales with combined vUnits from two clusters at EB=32 **and** EB=64 (80ms)
- ✓ withdrawAllOperatorEarnings transfers exact EB-weighted ETH after EB=64 accrual (70ms)
- ✓ withdrawOperatorEarnings on EB=64 cluster uses explicit-EB weighted accrual (63ms)
- ✓ withdrawOperatorEarnings reverts after removing operator from explicit-EB cluster (87ms)
- ✓ withdrawOperatorEarnings reflects higher post-update accrual at EB=128 (64ms)

SSVNetwork Integration – Legacy SSV Accounting

SSV vs ETH Cluster Differentiation

- ✓ ETH cluster has correct version **and** zero SSV balance/burn rate (252ms)
- ✓ Operator registered via ETH cluster has ETH fee but zero SSV fee
- ✓ ETH cluster operators have zero SSV earnings

Network Fee Earnings – SSV vs ETH Independence

- ✓ Initial network earnings are zero for both SSV **and** ETH
- ✓ Network fee is configured for both SSV **and** ETH
- ✓ ETH cluster activity increases ETH network earnings only, SSV unchanged
- ✓ updateNetworkFeeSSV changes SSV network fee independently of ETH fee
- ✓ updateNetworkFee changes ETH network fee independently of SSV fee

SSV-Specific DAO Functions

- ✓ withdrawNetworkSSVEarnings requires owner permission
- ✓ updateNetworkFeeSSV requires owner permission
- ✓ getMinimumLiquidationCollateralSSV is callable **and** returns a value

SSV Operator Earnings Functions

- ✓ withdrawOperatorEarningsSSV reverts with InsufficientBalance when no SSV earnings
- ✓ withdrawAllOperatorEarningsSSV reverts with InsufficientBalance when no SSV earnings
- ✓ withdrawOperatorEarningsSSV requires operator ownership

Liquidation Version Checks

- ✓ liquidateSSV reverts for ETH clusters with IncorrectClusterVersion

ITEST-2 Integration: migration with multiple EB updates

- ✓ Migrate after multiple EB updates uses the latest EB snapshot (74ms)
- ✓ EB set then validators added: migration uses updated vUnits for **new** validator count

SSVNetwork Integration – Operators (Enhanced)

Balance Delta Assertions

- ✓ withdrawOperatorEarnings: verifies exact ETH transfer to operator owner (238ms)
- ✓ withdrawAllOperatorEarnings: verifies complete balance drain with exact amounts

Boundary Tests – Operator Fees

- ✓ registerOperator: succeeds at exact minimum fee
- ✓ registerOperator: reverts at just below minimum fee
- ✓ registerOperator: succeeds at exact maximum fee
- ✓ registerOperator: reverts at just above maximum fee
- ✓ registerOperator: succeeds with zero fee (special case)
- ✓ declareOperatorFee: succeeds at exact max allowed increase
- ✓ declareOperatorFee: reverts at just above max allowed increase
- ✓ reduceOperatorFee: succeeds reducing to exact minimum fee
- ✓ reduceOperatorFee: reverts when reducing below minimum fee

Multi-Block Simulation – Operator Earnings Accrual

- ✓ Operator earnings accrue correctly over multiple block periods
- ✓ All 4 operators earn equally from a single validator cluster
- ✓ Operator earnings scale with validator count (42ms)
- ✓ removeValidator triggers exact settlement of operator earnings (44ms)

Invariant Checks – Operator Balance Consistency

- ✓ Invariant: Total operator earnings ≤ Cluster balance drained
- ✓ Invariant: Withdrawing all operator earnings zeros out balance
- ✓ Invariant: Removing validator stops operator fee accrual (54ms)

Combined Scenarios – Full Fee Distribution

- ✓ Full accounting: cluster deposit → operator earnings → network fees
- ✓ Operator withdrawal doesn't affect other operators' balances
- ✓ Fee change via declare→execute workflow with precise timing

Edge Cases **and** Error Conditions

- ✓ Cannot withdraw more than available earnings
- ✓ Operator with zero fee earns nothing
- ✓ executeOperatorFee reverts before declare period ends
- ✓ executeOperatorFee reverts after **execute** period expires
- ✓ Cannot increase fee from zero (must use reduceOperatorFee)
- ✓ Removed operator cannot have earnings withdrawn

Known Issue Repro: removed operator bricks explicit-EB maintenance paths

- ✓ allows the owner to self-liquidate after an operator removal (293ms)
- ✓ allows an EB decrease after an operator removal (81ms)
- ✓ allows the last validator to be removed cleanly after an operator removal (72ms)
- ✓ allows third-party liquidation once the cluster becomes objectively liquidatable (82ms)

SSVNetwork Integration – Staking (Enhanced)

Balance Delta Assertions – Token Movements

- ✓ stake: SSV transferred from staker to contract, cSSV minted 1:1 (223ms)
- ✓ requestUnstake: cSSV burned, delegation removed
- ✓ withdrawUnlocked: SSV returned to staker after cooldown

Reward Accrual from Cluster ETH Inflow

- ✓ Network fees from validator registration flow to staking rewards pool
- ✓ Multiple cluster deposits increase reward pool proportionally (50ms)
- ✓ EB=64 cluster contributes exactly 2x network-fee rewards vs EB=32 (64ms)
- ✓ Multiple clusters with different EBs accrue cumulative EB-weighted staking fees (97ms)

Staking Rewards Distribution

- ✓ Multiple stakers share rewards proportionally (60ms)

Invariant Checks – Staking Consistency

- ✓ Invariant: cSSV totalSupply always equals total staked across all users
- ✓ Invariant: Sum of individual staked balances equals totalStaked
- ✓ Invariant: Unstake request + staked balance = original stake

Combined Scenarios – Full Staking Lifecycle

- ✓ Full lifecycle: stake → cluster activity → unstake → withdraw (40ms)
- ✓ Multiple unstake requests processed in order

Edge Cases and Error Conditions

- ✓ Cannot stake zero amount
- ✓ Cannot stake below minimum stake amount
- ✓ Cannot unstake more than staked balance
- ✓ Cannot unstake zero amount
- ✓ Withdraws full amount one year after maturity
- ✓ Does **not** change cSSV supply on withdrawal
- ✓ Cannot withdraw before cooldown expires
- ✓ Cannot withdraw with no pending requests
- ✓ Cannot exceed maximum unstake requests (10) (5691ms)
- ✓ Cannot unstake when caller has no cSSV
- ✓ Cooldown duration change only affects **new** unstake requests
- ✓ Cannot claim rewards when no rewards accrued

Explicit EB staking revenue checks

- ✓ liquidating an explicit EB=64 cluster stops further staking revenue accrual (90ms)
- ✓ staking revenue doubles when explicit EB increases from 64 to 128 (89ms)

Multisig Accounts

- ✓ Multisig contract stakes SSV tokens
- ✓ Multisig stakes multiple times

SSVNetwork full integration tests

- ✓ Is reverted with 'EmptyPublicKeysList' **if** the **array** of **public** keys is empty (230ms)
- ✓ Is reverted with 'PublicKeysSharesLengthMismatch' **if** the **array** of keys **and** array of shares have different length

Constructor, initializer and upgrades

- ✓ Configures SSVNetwork correctly
- ✓ Calling initializeSSVStaking again reverts with already-initialized error

Function 'registerOperator()'

- ✓ Creates **new** operator **and** emits correct event
- ✓ Is reverted with 'FeeTooLow' **if** the provided fee is less than minimal allowed
- ✓ Is reverted with 'FeeTooHigh' **if** the provided fee is higher than maximum allowed
- ✓ Is reverted with 'OperatorAlreadyExists' **if** the **public** key is already registered

Function 'removeOperator()'

- ✓ Deactivates the operator **and** emits correct event
- ✓ Is reverted with 'OperatorDoesNotExist' **if** operator with passed id is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner

Function 'setOperatorsWhitelists()'

- ✓ Whitelists addresses **and** emits correct event
- ✓ Whitelists multiple operators for multiple addresses (47ms)
- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- ✓ Is reverted with 'InvalidWhitelistAddressesLength' **if** the **array** of addresses is empty
- ✓ Is reverted with 'ZeroAddressNotAllowed' **if** one of addresses is zero address
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is the the operator owner
- ✓ Is reverted with 'OperatorsListNotUnique' **if** the **array** of operators has any duplicate
- ✓ Is reverted with 'UnsortedOperatorsList' **if** operators are **not** sorted in increasing order

Function 'removeOperatorsWhitelists()'

- ✓ Removes addresses from the whitelist **and** emits correct event
- ✓ Removes multiple operators for multiple addresses (58ms)
- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- ✓ Is reverted with 'InvalidWhitelistAddressesLength' **if** the **array** of addresses is empty
- ✓ Is reverted with 'ZeroAddressNotAllowed' **if** one of addresses is zero address
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is the the operator owner
- ✓ Is reverted with 'OperatorsListNotUnique' **if** the **array** of operators has any duplicate
- ✓ Is reverted with 'UnsortedOperatorsList' **if** operators are **not** sorted in increasing order

Function 'setOperatorsWhitelistingContract()'

✓ Registers whitelisting contract, emits correct event **and** allows to whitelist addresses via contract

- ✓ Updates whitelisting contract for operators
- ✓ Registers whitelisting contract for 10 operators (46ms)
- ✓ Is reverted with 'InvalidWhitelistingContract' **if** the contract does **not** support required interface

- ✓ Is reverted with 'InvalidOperatorIdsLength' is the **array** of operators is empty
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is the the operator owner

Function 'removeOperatorsWhitelistingContract()'

- ✓ Removes whitelisting address **and** emits correct event
- ✓ Removes whitelisting contract for 10 operators (50ms)
- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** one of operators is **not** registered

Function 'setOperatorsPrivateUnchecked()'

- ✓ Changes privacy status **and** emits correct event
- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner

Function 'setOperatorsPublicUnchecked()'

- ✓ Changes privacy status **and** emits correct event
- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner

Function 'declareOperatorFee()'

- ✓ Declares **new** fee **and** emits correct event
- ✓ Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- ✓ Is reverted with 'FeeTooLow' is the passed fee is less than minimal
- ✓ Is reverted with 'SameFeeChangeNotAllowed' is the passed value is the same as current one
- ✓ Is reverted with 'SameFeeChangeNotAllowed' is the passed value is the same as current one
- ✓ Is reverted with 'FeeTooHigh' **if** the **new** fee is higher than allowed
- ✓ Is reverted with 'FeeExceedsIncreaseLimit' **if** the **new** fee exceeds the allowed limit
- ✓ Is reverted with 'FeeIncreaseNotAllowed' **if** operators current fee is zero

Function 'cancelDeclaredOperatorFee()'

- ✓ Cancels declared fee **and** emits correct event
- ✓ Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- ✓ Is reverted with 'NoFeeDeclared' **if** no declarations were done before

Function 'executeOperatorFee()'

- ✓ Updates operator fee according to a declared one **and** emits the correct event
- ✓ Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- ✓ Is reverted with 'NoFeeDeclared' **if** no declarations were done before
- ✓ Is reverted with 'ApprovalNotWithinTimeframe' **if** execution period is **not** started **or** ended
- ✓ Is reverted with 'FeeTooHigh' **if** the maximum fee changed during the execution period

Function 'updateMaximumOperatorFee()'

- ✓ Updates maximum fee **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** the caller is **not** the owner
- ✓ Reverts when **new** maximum fee is below the configured minimum fee

Function 'updateMinimumOperatorEthFee()'

- ✓ Updates minimum fee **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** the caller is **not** the owner
- ✓ When minimum is raised, registerOperator with fee below minimum reverts with FeeTooLow
- ✓ Reverts when **new** minimum fee exceeds the configured maximum fee

Function 'updateUnstakeCooldownDuration()'

- ✓ Changes cooldown period **and** emits correct event
- ✓ Is reverted **if** the caller is **not** the error

Function 'updateMinBlocksBetweenUpdates()'

- ✓ Updates the EB update cooldown blocks **and** emits correct event

- ✓ Is reverted **if** the caller is **not** the owner

Function 'updateQuorumBps()'

- ✓ Changes quorum **and** emits correct event
- ✓ Is reverted **if** the caller is **not** the error

Function 'reduceOperatorFee()'

- ✓ Decreases fee **and** emits the correct event
- ✓ Is reverted with 'OperatorDoesNotExist' **if** the operator is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner
- ✓ Is reverted with 'FeeTooLow' **if** the passed fee is less than minimum allowed
- ✓ Is reverted with 'FeeIncreaseNotAllowed' **if** caller is trying to increase the fee

Function 'withdrawOperatorEarnings()'

- ✓ Withdraws operators earnings, update balances **and** emits correct event
- ✓ Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** caller is **not** the operator owner
- ✓ Is reverted with 'InsufficientBalance' **if** the amount is less than operator earnings

Function 'withdrawAllOperatorEarnings()'

- ✓ Withdraws all operators earnings, update balances **and** emits correct event
- ✓ Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** caller is **not** the operator owner

Function 'withdrawAllVersionOperatorEarnings()'

- ✓ Withdraws all operators earnings **and** emits correct events
- ✓ Is reverted with 'OperatorDoesNotExist' **if** operator is **not** registered
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** caller is **not** the operator owner

Function 'setFeeRecipientAddress()'

- ✓ Emits the correct event with the correct input data

Function 'updateOperatorFeeIncreaseLimit()'

- ✓ Changes fee increase limit **and** emits the correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner
- ✓ Reverts when fee increase limit exceeds 100%

Function 'updateDeclareOperatorFeePeriod()'

- ✓ Changes the fee declare period **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'updateExecuteOperatorFeePeriod()'

- ✓ Changes the fee **execute** period **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'updateLiquidationThresholdPeriod()'

- ✓ Changes the period **and** emits correct event
- ✓ Is reverted 'NewBlockPeriodIsBelowMinimum' **if** the passed threshold is less than minimum allowed
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'updateLiquidationThresholdPeriodSSV()'

- ✓ Changes the period **and** emits correct event
- ✓ Is reverted 'NewBlockPeriodIsBelowMinimum' **if** the passed threshold is less than minimum allowed
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'updateMinimumLiquidationCollateral()'

- ✓ Changes collateral **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'updateMinimumLiquidationCollateralSSV()'

- ✓ Changes collateral **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'updateNetworkFee()'

- ✓ Updates network fee **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'updateNetworkFeeSSV()'

- ✓ Updates network fee SSV **and** emits correct event
- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'withdrawNetworkSSVEarnings()'

- ✓ Is reverted with 'Ownable: caller is **not** the owner' **if** caller is **not** the owner

Function 'registerValidator()'

- ✓ For a **new** cluster, creates it with a passed validator **and** emits correct event (46ms)
- ✓ Registers a validator for a **new** ETH cluster using whitelisting contract (38ms)
- ✓ Registers a validator for a **new** ETH cluster with one whitelisted operator (38ms)
- ✓ Registers a validator for a **new** ETH cluster with four whitelisted operators
- ✓ Registers a validator into an existing ETH cluster with four whitelisted operators (42ms)
- ✓ Registers a validator for a **new** ETH cluster with one whitelisting contract operator (40ms)
- ✓ Registers a validator into an existing ETH cluster (42ms)
- ✓ Registers a validator into a prefunded ETH cluster with zero additional deposit (40ms)
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is removed
- ✓ Is reverted with 'OperatorDoesNotExist' **if** one of operators is removed for an existing cluster

(61ms)

- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the amount of operators is **not** the allowed one
- ✓ Is reverted with 'InvalidPublicKeyLength' **if** the **public** key is **not** 48 bytes
- ✓ Is reverted with 'ValidatorAlreadyRegistered' **if** the **public** key is already registered

```
    ✓ Is reverted with 'IncorrectClusterState' for the new cluster is the cluster data is not
consisting from zeroes
    ✓ Is reverted with 'UnsortedOperatorsList' if operators are not sorted in increasing order
    ✓ Is reverted with 'OperatorsListNotUnique' if the array of operators has any duplicates
    ✓ Is reverted with 'CallerNotWhitelistedWithData' if one of operators did not whitelist the caller
    ✓ Is reverted with 'ExceedValidatorLimitWithData' if one of operators will exceed the network limit
(39ms)
    ✓ Is reverted with 'InsufficientBalance' if msg value is not enough to cover the validator
Function bulkRegisterValidator()
    ✓ Registers bulk of validators, creates a new cluster with the expected data and emits correct
events (194ms)
    ✓ Registers bulk of validators into an existing cluster with one whitelisting contract operator
(65ms)
    ✓ Is reverted with 'InvalidOperatorIdsLength' if the amount of operators is not the allowed one
    ✓ Is reverted with 'InvalidPublicKeyLength' if one of public keys is not 48 bytes
    ✓ Is reverted with 'ValidatorAlreadyRegistered' if one of public keys is already registered
    ✓ Is reverted with 'IncorrectClusterState' for the new cluster is the cluster data is not
consisting from zeroes
    ✓ Is reverted with 'UnsortedOperatorsList' if operators are not sorted in increasing order
    ✓ Is reverted with 'OperatorsListNotUnique' if the array of operators has any duplicates
    ✓ Is reverted with 'CallerNotWhitelistedWithData' if one of operators did not whitelist the caller
    ✓ Is reverted with 'ExceedValidatorLimitWithData' if one of operators will exceed the network limit
(40ms)
    ✓ Is reverted with 'InsufficientBalance' if msg value is not enough to cover new validators
    ✓ Is reverted with 'OperatorDoesNotExist' if one of operators is removed
    ✓ Is reverted with 'OperatorDoesNotExist' if one of operators is removed for an existing cluster
Function 'removeValidator()'
    ✓ Removes validator and emits correct event (41ms)
    ✓ Is reverted with 'ClusterDoesNotExists' if the cluster with this owner and operators does not
exist
    ✓ Is reverted with 'IncorrectClusterState' if the cluster data is incorrect
    ✓ Is reverted with 'ValidatorDoesNotExist' if the validator was never registered
    ✓ Is reverted with 'ValidatorDoesNotExist' if validator is already removed (56ms)
Function 'bulkRemoveValidator()'
    ✓ Removes validators and emits correct event (68ms)
    ✓ Is reverted with 'ClusterDoesNotExists' if the cluster with this owner and operators does not
exist
    ✓ Is reverted with 'IncorrectClusterState' if the cluster data is incorrect
    ✓ Is reverted with 'ValidatorDoesNotExist' if the validator was never registered
    ✓ Is reverted with 'ValidatorDoesNotExist' if validator is already removed
Function 'liquidate()'
    ✓ If called by cluster owner – liquidates cluster, refunds caller and emits correct event (55ms)
    ✓ If called not by a cluster owner, liquidates cluster, refunds the caller and emits correct event
(44ms)
    ✓ Is reverted with 'ClusterDoesNotExists' if the cluster was not created yet
    ✓ Is reverted with 'IncorrectClusterState' if the provided cluster state is incorrect
    ✓ Is reverted with 'ClusterIsLiquidated' if cluster is already liquidated
    ✓ Is reverted with 'ClusterNotLiquidatable' if called not by cluster owner and cluster is not yet
liquidatable
Function 'reactivate()'
    ✓ Reactivates the cluster and emits correct event (45ms)
    ✓ Is reverted with 'ClusterDoesNotExists' if such cluster was not yet registered (51ms)
    ✓ Is reverted with 'IncorrectClusterState' if the provided cluster state is incorrect
    ✓ Is reverted with 'ClusterAlreadyEnabled' if cluster is not liquidated
    ✓ Is reverted with 'InsufficientBalance' if the amount is not enough to cover runway
Function 'deposit()'
    ✓ Increases cluster balance and emits correct event
    ✓ Is reverted with 'ClusterDoesNotExists' if such cluster was not yet registered
    ✓ Is reverted with 'IncorrectClusterState' if the provided cluster state is incorrect
Function 'withdraw()'
    ✓ Withdraws from cluster balance and emits correct event
    ✓ Is reverted with 'ClusterDoesNotExists' if such cluster was not yet registered (47ms)
    ✓ Is reverted with 'IncorrectClusterState' if the provided cluster state is incorrect
    ✓ Is reverted with 'InsufficientBalance' when withdrawing from a liquidated cluster with zero
balance
    ✓ Is reverted with 'InsufficientBalance' if the amount is bigger than cluster balance
Function 'exitValidator()'
    ✓ Emits correct event
    ✓ Is reverted with 'ValidatorDoesNotExist' if the key does not exist or belong to a caller
Function 'bulkExitValidator()'
    ✓ Emits correct event
    ✓ Is reverted with 'ValidatorDoesNotExist' if the key does not exist or belong to a caller
```

Function stake()

- ✓ Stakes SSV, mints CSSV to the staker and creates delegation weight
- ✓ Is reverted with 'StakeTooLow' if the amount to stake is smaller than minimum allowed
- ✓ Is reverted with 'StakeTooLow' if caller is trying to stake 0 SSV

Function requestUnstake()

- ✓ For full amount, creates unstake request, burns CSSV and removes delegation
- ✓ For partial amount, creates unstake request, burns CSSV and removes delegation
- ✓ Is reverted with 'MaxRequestsAmountReached' if more than 10 pending requests (5759ms)
- ✓ Is reverted with 'ZeroAmount' if caller is trying to request 0 SSV
- ✓ Is reverted with 'UnstakeAmountExceedsBalance' if caller is trying to request more SSV than they

staked

Function 'withdrawUnlocked()'

- ✓ Withdraws SSV and emits correct event
- ✓ Is reverted with 'NothingToWithdraw()' if nor rewards were unfrozen yet

Function 'claimEthRewards()'

- ✓ Claims ETH rewards and emits correct event (183ms)
- ✓ Is reverted with 'NothingToClaim()' if no rewards were accumulated yet

Function 'syncFees()'

- ✓ Syncs fees and emits correct event if the data is relevant (187ms)
- ✓ Does not emit event if no relevant changes happened

Function 'rescueERC20()'

- ✓ Withdraws tokens and emits correct event
- ✓ Withdraws non-standard ERC20 tokens that do not return a value
- ✓ Is reverted with 'Ownable: caller is not the owner' if the caller is not the owner
- ✓ Is reverted with 'ZeroAddress()' if token address is zero
- ✓ Is reverted with 'ZeroAddress()' if receiver address is zero
- ✓ Is reverted with 'InvalidToken()' if trying to rescue ssv token
- ✓ Is reverted with 'InvalidToken()' if trying to rescue cssv token
- ✓ Is reverted with 'ZeroAmount()' if trying to rescue zero tokens

Function 'onCSSVTransfer()'

- ✓ Syncs fees and emits correct events (207ms)
- ✓ Is reverted with 'NotCSSV()' if the caller is not CSSV token

Reentrancy Guard Tests

- ✓ Prevents reentrancy in 'liquidate()' (40ms)
- ✓ Prevents reentrancy in 'withdraw()'
- ✓ Prevents reentrancy in 'withdrawAllOperatorEarnings()' (38ms)
- ✓ Prevents reentrancy in 'MaliciousWithdrawAllVersionOperatorEarnings()'
- ✓ Prevents reentrancy in 'withdrawOperatorEarnings()'
- ✓ Prevents reentrancy in 'claimEthRewards()' (186ms)
- ✓ Prevents reentrancy in 'reactivate()' (44ms)

SSVNetwork full integration tests with performing an upgrade on a legacy artifact

Legacy setup configuration

- ✓ Configures SSVNetwork and SSVNetworkViews correctly (142ms)

Restrictions for legacy ssv clusters

✓ 'registerValidator()' is reverted with 'IncorrectClusterVersion' if trying to register validators to a legacy cluster (213ms)

✓ 'bulkRegisterValidator()' is reverted with 'IncorrectClusterVersion' if trying to register validators to a legacy cluster (209ms)

- ✓ 'removeValidator()' succeeds on legacy SSV clusters (BUG-12 fix) (215ms)

- ✓ 'bulkRemoveValidator()' succeeds on legacy SSV clusters (BUG-12 fix) (223ms)

✓ 'liquidate()' is reverted with 'IncorrectClusterVersion' if trying to liquidate legacy ssv cluster with an ETH-based function (204ms)

✓ 'reactivate()' is reverted with 'IncorrectClusterVersion' if trying to reactivate a legacy ssv cluster (208ms)

✓ withdraw() is reverted with 'IncorrectClusterVersion' is trying to withdraw from a legacy ssv cluster (208ms)

✓ deposit() is reverted with 'IncorrectClusterVersion' is trying to deposit to a legacy ssv cluster (204ms)

Function 'migrateClusterToETH'

- ✓ Executes as expected and emits correct event (223ms)
- ✓ Migrates a liquidated cluster, emits correct events and reactivates cluster (225ms)

Legacy operators migration

- ✓ Migrates operators to the eth version after migration of operator's cluster (219ms)

Sanity

- ✓ Migrates operators to ETH after registering an ETH validator and applies correct fees (211ms)
- ✓ Reactivates liquidated cluster during migration, emits event and applies correct fees (228ms)
- ✓ Allows to remove all validators from a liquidated cluster (227ms)

Effective Balance Roundtrip Tests

Roundtrip conversion

- ✓ 0 ETH (edge case)

- ✓ 1 ETH (minimum)
- ✓ 31 ETH (below 1 validator)
- ✓ 32 ETH (1 validator, exact)
- ✓ 33 ETH (ceiling)
- ✓ 63 ETH
- ✓ 64 ETH (2 validators, exact)
- ✓ 100 ETH
- ✓ 515 ETH (ceiling)
- ✓ 1000 ETH
- ✓ 2048 ETH (max per validator)

Migration with removed operator frozen ETH index

- ✓ Migration cluster.index includes removed operator's frozen ethSnapshot.index (61ms)
- ✓ No phantom fee charge on first post-migration operation
- ✓ Migration with multiple removed operators preserves all frozen indices
- ✓ Removed SSV-only operator (zero ETH index) causes no issue on migration (68ms)
- ✓ Liquidated SSV cluster migration with removed dual operator is correctly accounted (39ms)

Operator views consistency sanity tests

getOperatorById vs getOperatorByIdSSV isActive consistency

- ✓ ETH-only operator: both views report isActive = true (222ms)
- ✓ Removed operator: both views report isActive = false
- ✓ Operator used in ETH cluster: both views report isActive = true
- ✓ Removed operator with prior ETH cluster usage: both views report isActive = false

precision and governance boundaries

- ✓ explicit EB=32 with removed operator liquidates without underflow (64ms)
- ✓ EB=33 lifecycle keeps exact minimal non-default vUnits
- ✓ same-EB update has zero vUnit delta
- ✓ governance parameter changes move explicit-EB liquidation boundaries deterministically (105ms)
- ✓ maximum deviation decrease (2048 → 32) after operator removal is safe and exact
- ✓ non-round vUnits accrual is exact across EB 33 → 65 transition
- ✓ EB=225 with 7 validators yields exact per-operator rounded deviation (97ms)
- ✓ withdrawing exact max after settlement leaves zero residual dust

'removeOperator()' deletes operatorEthVUnits and does not affect clusters

with 4 operators

- ✓ liquidation does not revert after operator removal when cluster has EB deviation (76ms)
- ✓ 'updateClusterBalance()' with previous deviation and EB decrease does not revert after operator

removal

- ✓ 'bulkRemoveValidator()' (emptying cluster) does not revert after operator removal
- ✓ auto liquidation via 'updateClusterBalance()' does not revert after operator removal
- ✓ 'updateClusterBalance()' with EB increase does not re-add deviation to removed operator
- ✓ liquidating two clusters with common removed operator cleans up correctly and does not revert

(64ms)

- ✓ 'reactivate()' with EB deviation does not add deviation to a removed operator
- ✓ 'migrateClusterToETH()' with EB deviation does not write deviation to removed operator
- ✓ 'migrateClusterToETH()' with removed operator skips removed operator's snapshot and fee

accumulation

- ✓ R-11: liquidation does not revert after removing 2 operators from explicit EB cluster
- ✓ EC-03: maximum EB (2048) + removed operator + liquidate does not underflow
- ✓ RI-04: implicit EB cluster → remove operator → first oracle EB update skips dead operator
- ✓ R-13: withdraw succeeds after operator removal on explicit EB cluster (balance settlement

correctness)

- ✓ R-11 variant: removing 2 operators + EB decrease does not underflow (48ms)
- ✓ R-07: registerValidator reverts with OperatorDoesNotExist after operator removal on explicit EB

cluster

- ✓ R-10: explicit EB=32 (zero deviation) + removed operator + self-liquidate does not revert
- ✓ EC-05: all operators removed from explicit EB cluster → self-liquidate does not revert

with 7 operators

- ✓ liquidation does not revert after operator removal when cluster has EB deviation (90ms)
- ✓ 'updateClusterBalance()' with previous deviation and EB decrease does not revert after operator

removal

- ✓ 'bulkRemoveValidator()' (emptying cluster) does not revert after operator removal
- ✓ auto liquidation via 'updateClusterBalance()' does not revert after operator removal (41ms)
- ✓ 'updateClusterBalance()' with EB increase does not re-add deviation to removed operator (46ms)
- ✓ liquidating two clusters with common removed operator cleans up correctly and does not revert

(84ms)

- ✓ 'reactivate()' with EB deviation does not add deviation to a removed operator (80ms)
- ✓ 'migrateClusterToETH()' with EB deviation does not write deviation to removed operator
- ✓ 'migrateClusterToETH()' with removed operator skips removed operator's snapshot and fee

accumulation

- ✓ R-11: liquidation does **not** revert after removing 2 operators from explicit EB cluster (45ms)
- ✓ EC-03: maximum EB (2048) + removed operator + liquidate does **not** underflow (40ms)
- ✓ RI-04: implicit EB cluster → remove operator → first oracle EB update skips dead operator
- ✓ R-13: withdraw succeeds after operator removal on explicit EB cluster (balance settlement correctness)
- ✓ R-11 variant: removing 2 operators + EB decrease does **not** underflow (62ms)
- ✓ R-07: registerValidator reverts with OperatorDoesNotExist after operator removal on explicit EB cluster
- ✓ R-10: explicit EB=32 (zero deviation) + removed operator + self-liquidate does **not** revert (40ms)
- ✓ EC-05: all operators removed from explicit EB cluster → self-liquidate does **not** revert (68ms)
- with 10 operators
- ✓ liquidation does **not** revert after operator removal when cluster has EB deviation (115ms)
- ✓ 'updateClusterBalance()' with previous deviation **and** EB decrease does **not** revert after operator removal (45ms)
- ✓ 'bulkRemoveValidator()' (emptying cluster) does **not** revert after operator removal (40ms)
- ✓ auto liquidation via 'updateClusterBalance()' does **not** revert after operator removal (46ms)
- ✓ 'updateClusterBalance()' with EB increase does **not** re-add deviation to removed operator (40ms)
- ✓ liquidating two clusters with common removed operator cleans up correctly **and** does **not** revert (99ms)
- ✓ 'reactivate()' with EB deviation does **not** add deviation to a removed operator (61ms)
- ✓ 'migrateClusterToETH()' with EB deviation does **not** write deviation to removed operator
- ✓ 'migrateClusterToETH()' with removed operator skips removed operator's snapshot **and** fee accumulation
- ✓ R-11: liquidation does **not** revert after removing 2 operators from explicit EB cluster (53ms)
- ✓ EC-03: maximum EB (2048) + removed operator + liquidate does **not** underflow (71ms)
- ✓ RI-04: implicit EB cluster → remove operator → first oracle EB update skips dead operator (40ms)
- ✓ R-13: withdraw succeeds after operator removal on explicit EB cluster (balance settlement correctness)
- ✓ R-11 variant: removing 2 operators + EB decrease does **not** underflow (55ms)
- ✓ R-07: registerValidator reverts with OperatorDoesNotExist after operator removal on explicit EB cluster
- ✓ R-10: explicit EB=32 (zero deviation) + removed operator + self-liquidate does **not** revert (45ms)
- ✓ EC-05: all operators removed from explicit EB cluster → self-liquidate does **not** revert (93ms)
- with 13 operators
- ✓ liquidation does **not** revert after operator removal when cluster has EB deviation (106ms)
- ✓ 'updateClusterBalance()' with previous deviation **and** EB decrease does **not** revert after operator removal (54ms)
- ✓ 'bulkRemoveValidator()' (emptying cluster) does **not** revert after operator removal (42ms)
- ✓ auto liquidation via 'updateClusterBalance()' does **not** revert after operator removal (46ms)
- ✓ 'updateClusterBalance()' with EB increase does **not** re-add deviation to removed operator (47ms)
- ✓ liquidating two clusters with common removed operator cleans up correctly **and** does **not** revert (79ms)
- ✓ 'reactivate()' with EB deviation does **not** add deviation to a removed operator (110ms)
- ✓ 'migrateClusterToETH()' with EB deviation does **not** write deviation to removed operator
- ✓ 'migrateClusterToETH()' with removed operator skips removed operator's snapshot **and** fee accumulation
- ✓ R-11: liquidation does **not** revert after removing 2 operators from explicit EB cluster (57ms)
- ✓ EC-03: maximum EB (2048) + removed operator + liquidate does **not** underflow (69ms)
- ✓ RI-04: implicit EB cluster → remove operator → first oracle EB update skips dead operator (39ms)
- ✓ R-13: withdraw succeeds after operator removal on explicit EB cluster (balance settlement correctness)
- ✓ R-11 variant: removing 2 operators + EB decrease does **not** underflow (49ms)
- ✓ R-07: registerValidator reverts with OperatorDoesNotExist after operator removal on explicit EB cluster
- ✓ R-10: explicit EB=32 (zero deviation) + removed operator + self-liquidate does **not** revert (44ms)
- ✓ EC-05: all operators removed from explicit EB cluster → self-liquidate does **not** revert (116ms)
- Cross-cluster removed-operator propagation (4 operators)
- ✓ shared operator removal does **not** corrupt multi-cluster explicit-EB totals
- ✓ liquidating explicit cluster after shared removal preserves implicit-only accounting
- ✓ EB decrease on one explicit cluster after shared removal updates only surviving operator slots (46ms)
- ✓ removing the last validator on second explicit cluster after shared removal cleans only that cluster
- ✓ mixed EB increase/decrease after one shared-operator removal keeps per-operator totals exact (69ms)
- ✓ multiple explicit-EB clusters liquidated end at daoTotalEthVUnits == 0 (41ms)
- ✓ shared operators across explicit clusters accumulate **and** clean exactly (53ms)
- ✓ removeOperator clears operator slot but leaves DAO total unchanged through deposit
- Cluster with a removed operator sanity test
- ✓ Allows to liquidate cluster with a previously removed operator (259ms)

replaceOracle() – InvalidOracleId boundary sanity

- ✓ Accepts oracleId equal to MAX_DELEGATION_SLOTS (boundary)
- ✓ Is reverted with 'InvalidOracleId' when oracleId is MAX_DELEGATION_SLOTS + 1
- ✓ Is reverted with 'InvalidOracleId' when oracleId is far above MAX_DELEGATION_SLOTS

Dust **check** in the ssv staking module

- ✓ Should **not** leave any dust after all participants withdraw their funds (666ms)

SSV-2: commitRoot freezes cSSV supply on first vote

- ✓ Freezes supply at first vote **and** cleans up on commit (50ms)
- ✓ Supply increase between votes does **not** block quorum (liveness attack blocked)
- ✓ Supply decrease between votes does **not** bypass quorum (safety attack blocked)

SSV-3: bulkRegisterValidator uses post-registration vUnits for liquidation **check**

- ✓ Reverts with InsufficientBalance when deposit covers old vUnits but **not** post-registration vUnits (56ms)
- ✓ Succeeds when deposit is sufficient for post-registration vUnits
- ✓ Implicit EB clusters (vUnits == 0 in storage) are unaffected

stale snapshot replay matrix

- ✓ stale caller-supplied cluster is rejected on repeated updateClusterBalance (59ms)
- ✓ liquidation rejects stale pre-mutation cluster snapshot
- ✓ removeValidator rejects stale snapshot after a successful fresh removal
- ✓ reactivate rejects stale active snapshot after liquidation
- ✓ removeValidator rejects stale EB=64 snapshot after EB=128 update
- ✓ liquidate replay with stale pre-liquidation snapshot fails cleanly

vUnits cluster-size matrix coverage

- ✓ fee accrual scales with 7/10/13 operators (110ms)
- ✓ per-operator counts **and** EB=64 deviation distribution are exact for 7/13 operators (49ms)
- ✓ EB transitions on 7/13 operators apply exact per-operator deltas (97ms)
- ✓ explicit-EB liquidation/reactivation restores deviations for 7/13 operators (88ms)
- ✓ post-liquidation operator removals are skipped on reactivation for 7/13 operators (93ms)
- ✓ migration with explicit EB=64 works for 7/13 operators (43ms)
- ✓ mixed-size shared-operator interactions keep per-cluster accounting isolated (135ms)
- ✓ DAO invariant holds across 4/7/10/13 clusters with **and** without removals (192ms)

Monte Carlo Upgrade Simulation

- runs 30-day simulation without invariant violations

SSVNetwork full integration tests made on forked contract

- Is reverted with 'EmptyPublicKeysList' **if** the **array** of **public** keys is empty
- Is reverted with 'PublicKeysSharesLengthMismatch' **if** the **array** of keys **and** array of shares have different length

Constructor, initializer **and** upgrades

- Configures SSVNetwork correctly

Function 'registerOperator()'

- Creates **new** operator **and** emits correct event
- Is reverted with 'FeeTooLow' **if** the provided fee is less than minimal allowed
- Is reverted with 'FeeTooHigh' **if** the provided fee is higher than maximum allowed
- Is reverted with 'OperatorAlreadyExists' **if** the **public** key is already registered

Function 'removeOperator()'

- Deactivates the operator **and** emits correct event
- Is reverted with 'OperatorDoesNotExist' **if** operator with passed id is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is **not** the operator owner

Function 'setOperatorsWhitelists()'

- Whitelists addresses **and** emits correct event
- Whitelists multiple operators for multiple addresses
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'InvalidWhitelistAddressesLength' **if** the **array** of addresses is empty
- Is reverted with 'ZeroAddressNotAllowed' **if** one of addresses is zero address
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered
- Is reverted with 'CallerNotOwnerWithData' **if** the caller is the the operator owner
- Is reverted with 'OperatorsListNotUnique' **if** the **array** of operators has any duplicate
- Is reverted with 'UnsortedOperatorsList' **if** operators are **not** sorted in increasing order

Function 'removeOperatorsWhitelists()'

- Removes addresses from the whitelist **and** emits correct event
- Removes multiple operators for multiple addresses
- Is reverted with 'InvalidOperatorIdsLength' **if** the **array** of operators is empty
- Is reverted with 'InvalidWhitelistAddressesLength' **if** the **array** of addresses is empty
- Is reverted with 'ZeroAddressNotAllowed' **if** one of addresses is zero address
- Is reverted with 'OperatorDoesNotExist' **if** one of operators is **not** registered

- Is reverted With 'CallerNotOwnerWithData' if the caller is the the operator owner
- Is reverted with 'OperatorsListNotUnique' if the array of operators has any duplicate
- Is reverted with 'UnsortedOperatorsList' if operators are not sorted in increasing order

Function 'setOperatorsWhitelistingContract()'

- Registers whitelisting contract, emits correct event and allows to whitelist addresses via contract
- Updates whitelisting contract for operators
- Registers whitelisting contract for 10 operators
- Is reverted with 'InvalidWhitelistingContract' if the contract does not support required interface
- Is reverted with 'InvalidOperatorIdsLength' is the array of operators is empty
- Is reverted with 'OperatorDoesNotExist' if one of operators is not registered
- Is reverted with 'CallerNotOwnerWithData' if the caller is the the operator owner

Function 'removeOperatorsWhitelistingContract()'

- Removes whitelisting address and emits correct event
- Removes whitelisting contract for 10 operators
- Is reverted with 'InvalidOperatorIdsLength' if the array of operators is empty
- Is reverted with 'OperatorDoesNotExist' if one of operators is not registered
- Is reverted with 'CallerNotOwnerWithData' if one of operators is not registered

Function 'setOperatorsPrivateUnchecked()'

- Changes privacy status and emits correct event
- Is reverted with 'InvalidOperatorIdsLength' if the array of operators is empty
- Is reverted with 'OperatorDoesNotExist' if one of operators is not registered
- Is reverted with 'CallerNotOwnerWithData' if the caller is not the operator owner

Function 'setOperatorsPublicUnchecked()'

- Changes privacy status and emits correct event
- Is reverted with 'InvalidOperatorIdsLength' if the array of operators is empty
- Is reverted with 'OperatorDoesNotExist' if one of operators is not registered
- Is reverted with 'CallerNotOwnerWithData' if the caller is not the operator owner

Function 'declareOperatorFee()'

- Declares new fee and emits correct event
- Is reverted with 'OperatorDoesNotExist' if operator is not registered
- Is reverted with 'CallerNotOwnerWithData' if the caller is not the operator owner
- Is reverted with 'FeeTooLow' is the passed fee is less than minimal
- Is reverted with 'SameFeeChangeNotAllowed' is the passed value is the same as current one
- Is reverted with 'FeeTooHigh' if the new fee is higher than allowed
- Is reverted with 'FeeExceedsIncreaseLimit' if the new fee exceeds the allowed limit
- Is reverted with 'FeeIncreaseNotAllowed' if operators current fee is zero

Function 'cancelDeclaredOperatorFee()'

- Cancels declared fee and emits correct event
- Is reverted with 'OperatorDoesNotExist' if operator is not registered
- Is reverted with 'CallerNotOwnerWithData' if the caller is not the operator owner
- Is reverted with 'NoFeeDeclared' if no declarations were done before

Function 'executeOperatorFee()'

- Updates operator fee according to a declared one and emits the correct event
- Is reverted with 'OperatorDoesNotExist' if operator is not registered
- Is reverted with 'CallerNotOwnerWithData' if the caller is not the operator owner
- Is reverted with 'NoFeeDeclared' if no declarations were done before
- Is reverted with 'ApprovalNotWithinTimeframe' if execution period is not started or ended
- Is reverted with 'FeeTooHigh' if the maximum fee changed during the execution period

Function 'updateMaximumOperatorFee()'

- Updates maximum fee and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if the caller is not the owner
- Reverts when new maximum fee is below the configured minimum fee

Function 'reduceOperatorFee()'

- Decreases fee and emits the correct event
- Is reverted with 'OperatorDoesNotExist' if the operator is not registered
- Is reverted with 'CallerNotOwnerWithData' if the caller is not the operator owner
- Is reverted with 'FeeTooLow' if the passed fee is less than minimum allowed
- Is reverted with 'FeeIncreaseNotAllowed' if caller is trying to increase the fee

Function 'withdrawOperatorEarnings()'

- Withdraws operators earnings, update balances and emits correct event
- Is reverted with 'OperatorDoesNotExist' if operator is not registered
- Is reverted with 'CallerNotOwnerWithData' if caller is not the operator owner
- Is reverted with 'InsufficientBalance' if the amount is less than operator earnings

Function 'withdrawAllOperatorEarnings()'

- Withdraws all operators earnings, update balances and emits correct event
- Is reverted with 'OperatorDoesNotExist' if operator is not registered
- Is reverted with 'CallerNotOwnerWithData' if caller is not the operator owner

Function 'withdrawAllVersionOperatorEarnings()'

- Withdraws all operators earnings and emits correct events
- Is reverted with 'OperatorDoesNotExist' if operator is not registered

- Is reverted with 'CallerNotOwnerWithData' if caller is not the operator owner

Function 'setFeeRecipientAddress()'

- Emits the correct event with the correct input data

Function 'updateOperatorFeeIncreaseLimit()'

- Changes fee increase limit and emits the correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner
- Reverts when fee increase limit exceeds 100%

Function 'updateDeclareOperatorFeePeriod()'

- Changes the fee declare period and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner

Function 'updateExecuteOperatorFeePeriod()'

- Changes the fee execute period and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner

Function 'updateLiquidationThresholdPeriod()'

- Changes the period and emits correct event
- Is reverted 'NewBlockPeriodIsBelowMinimum' if the passed threshold is less than minimum allowed
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner

Function 'updateLiquidationThresholdPeriodSSV()'

- Changes the period and emits correct event
- Is reverted 'NewBlockPeriodIsBelowMinimum' if the passed threshold is less than minimum allowed
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner

Function 'updateMinimumLiquidationCollateral()'

- Changes collateral and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner

Function 'updateMinimumLiquidationCollateralSSV()'

- Changes collateral and emits correct event
- Is reverted with 'Ownable: caller is not the owner' if caller is not the owner

Function 'registerValidator()'

- For a new cluster, creates it with a passed validator and emits correct event
- Registers a validator for a new ETH cluster using whitelisting contract
- Registers a validator for a new ETH cluster with one whitelisted operator
- Registers a validator for a new ETH cluster with four whitelisted operators
- Registers a validator into an existing ETH cluster with four whitelisted operators
- Registers a validator into an existing ETH cluster
- Registers a validator into a prefunded ETH cluster with zero additional deposit
- Is reverted with 'InvalidOperatorIdsLength' if the amount of operators is not the allowed one
- Is reverted with 'InvalidPublicKeyLength' if the public key is not 48 bytes
- Is reverted with 'ValidatorAlreadyRegistered' if the public key is already registered
- Is reverted with 'IncorrectClusterState' for the new cluster is the cluster data is not consisting from zeroes
- Is reverted with 'UnsortedOperatorsList' if operators are not sorted in increasing order
- Is reverted with 'OperatorsListNotUnique' if the array of operators has any duplicates
- Is reverted with 'CallerNotWhitelistedWithData' if one of operators did not whitelist the caller
- Is reverted with 'ExceedValidatorLimitWithData' if one of operators will exceed the network limit
- Is reverted with 'InsufficientBalance' if msg value is not enough to cover the validator

Function bulkRegisterValidator()

- Registers bulk of validators, creates a new cluster with the expected data and emits correct events
- Registers bulk of validators into an existing cluster with one whitelisting contract operator
- Is reverted with 'InvalidOperatorIdsLength' if the amount of operators is not the allowed one
- Is reverted with 'InvalidPublicKeyLength' if one of public keys is not 48 bytes
- Is reverted with 'ValidatorAlreadyRegistered' if one of public keys is already registered
- Is reverted with 'IncorrectClusterState' for the new cluster is the cluster data is not consisting from zeroes
- Is reverted with 'UnsortedOperatorsList' if operators are not sorted in increasing order
- Is reverted with 'OperatorsListNotUnique' if the array of operators has any duplicates
- Is reverted with 'CallerNotWhitelistedWithData' if one of operators did not whitelist the caller
- Is reverted with 'ExceedValidatorLimitWithData' if one of operators will exceed the network limit
- Is reverted with 'InsufficientBalance' if msg value is not enough to cover new validators

Function 'removeValidator()'

- Removes validator and emits correct event
- Is reverted with 'ClusterDoesNotExists' if the cluster with this owner and operators does not exist
- Is reverted with 'IncorrectClusterState' if the cluster data is incorrect
- Is reverted with 'ValidatorDoesNotExist' if the validator was never registered
- Is reverted with 'ValidatorDoesNotExist' if validator is already removed

Function 'bulkRemoveValidator()'

- Is reverted with 'ClusterDoesNotExists' if the cluster with this owner and operators does not exist
- Is reverted with 'IncorrectClusterState' if the cluster data is incorrect
- Is reverted with 'IncorrectValidatorStateWithData' if the validator was never registered
- Is reverted with 'ValidatorDoesNotExist' if validator is already removed

Function stake()

- Stakes SSV, mints CSSV to the staker and creates delegation weight
- Is reverted with 'StakeTooLow' if the amount to stake is smaller than minimum allowed
- Is reverted with 'StakeTooLow' if caller is trying to stake 0 SSV

Function requestUnstake()

- For full amount, creates unstake request, burns CSSV and removes delegation
- For partial amount, creates unstake request, burns CSSV and removes delegation
- Is reverted with 'MaxRequestsAmountReached' if more than 2000 pending requests
- Is reverted with 'ZeroAmount' if caller is trying to request 0 SSV
- Is reverted with 'UnstakeAmountExceedsBalance' if caller is trying to request more SSV than they

staked

Function 'withdrawUnlocked()'

- Withdraws SSV and emits correct event

SSVClusters function `deposit()`

- ✓ Deposits into an existing cluster, updates balance and emits correct event (93ms)
- ✓ Does not change operatorEthVUnits or stored cluster EB snapshot when depositing
- ✓ Allows a third party to deposit to an existing cluster
- ✓ Accumulates contract ETH balance by the sum of multiple deposits
- ✓ Is reverted with 'IncorrectClusterState' when provided cluster data is stale or mismatched
- ✓ Is reverted with 'ClusterDoesNotExist' when attempting to deposit into a missing cluster
- ✓ Deposit into zero-validator cluster accrues no fees over elapsed blocks (45ms)
- ✓ Does not change contract balance when deposit reverts

EB auto-liquidation on updateClusterBalance

- ✓ Auto-liquidates cluster when EB increase makes it insolvent at new rate (69ms)
- ✓ Does NOT auto-liquidate when cluster is solvent at new EB rate
- ✓ Auto-liquidates when cluster is already insolvent at old rate
- ✓ Blocks reentrant guarded calls during updateClusterBalance auto-liquidation callback (98ms)

EB decrease scenarios

- ✓ EB decrease from 64 to 32 ETH reduces vUnits, clears deviation, settles fees at old rate (78ms)
- ✓ EB decrease below 32 ETH per validator reverts with EBBelowMinimum
- ✓ EB decrease auto-liquidates cluster when balance falls below new lower threshold
- ✓ EB decrease correctly decrements operator deviation and daoTotalEthVUnits

EB-aware fee settlement on registration and removal

- ✓ Registration settles fees using EB-weighted vUnits, not flat validatorCount (53ms)
- ✓ Removal settles fees using EB-weighted vUnits

Edge Cases for EB Settlement

- ✓ Uses baseline vUnits when EB = 0 (no EB set)
- ✓ Handles EB exactly at baseline (32 ETH)
- ✓ Handles very high EB values (stress test)
- ✓ Handles zero validator count edge case

EB-weighted operator earnings (Consolidated)

Accumulation

- ✓ operator earns proportionally from two clusters with EB=32 and EB=64 (80ms)
- ✓ earnings split correctly at fee change boundary with EB-weighted vUnits
- ✓ operator snapshot balance equals expected EB-weighted ETH after settlement

Edge Cases

- ✓ operator with zero fee earns nothing despite EB > 32 (48ms)
- ✓ operator earnings cap at maximum EB (2048 ETH per validator)
- ✓ operator earnings reflect multi-validator cluster with EB > 32
- ✓ operator earnings adjust correctly when EB decreases
- ✓ operator earns from mixed implicit and explicit EB clusters correctly

Operator fee change + EB burn rate interaction

- ✓ Operator fee increase on EB=64 cluster doubles burn rate (361ms)
- ✓ Operator fee reduction on EB-weighted cluster lowers burn proportionally (94ms)
- ✓ Fee execution boundary with EB=64 applies old and new rates to correct block ranges (64ms)

SSVClusters legacy SSV accounting

- ✓ removeValidator settles accrued legacy SSV operator fees before decrementing validator count (73ms)
- ✓ removeValidator settles legacy SSV fees identically when a pending ETH fee change request exists
- ✓ bulkRemoveValidator settles legacy SSV network fees on active clusters (52ms)

SSVClusters function `liquidate()`

- ✓ Allows the cluster owner to liquidate and emits correct event (62ms)
- ✓ Transfers remaining cluster ETH balance to the liquidator
- ✓ Transfers no ETH when cluster remaining balance is zero after fee accrual
- ✓ Self-liquidation returns remaining ETH balance to the cluster owner

- ✓ Updates operatorEthVUnits on liquidation even when cluster EB snapshot is **not** set
- ✓ Uses stored cluster EB snapshot vUnits when present when updating operatorEthVUnits on liquidation (43ms)
- ✓ Allows the cluster owner to liquidate with 7 operators (60ms)
- ✓ Allows the cluster owner to liquidate with 10 operators (62ms)
- ✓ Allows the cluster owner to liquidate with 13 operators (79ms)
- ✓ Allows a third party to liquidate when the cluster is liquidatable
- ✓ Allows a third party to liquidate when liquidation threshold units exceed uint64 (70ms)
- ✓ Is reverted with 'ClusterNotLiquidatable' when a third party tries to liquidate a healthy cluster
- ✓ Is reverted with 'ClusterIsLiquidated' when liquidating an already liquidated cluster
- ✓ Is reverted with 'IncorrectClusterState' when provided cluster data is stale **or** mismatched
- ✓ Is reverted with 'ClusterDoesNotExists' when attempting to liquidate a missing cluster

SSVClusters function `liquidateSSV()`

- ✓ Allows the cluster owner to liquidate SSV cluster **and** emits correct event (58ms)
- ✓ Transfers remaining SSV token balance in the cluster to the liquidator
- ✓ Transfers no SSV when cluster remaining balance is zero after fee accrual
- ✓ SSV self-liquidation returns remaining SSV balance to the cluster owner
- ✓ Does **not** change operatorEthVUnits **or** stored cluster EB snapshot when liquidating an SSV cluster
- ✓ Allows the cluster owner to liquidate SSV cluster with 7 operators (65ms)
- ✓ Allows the cluster owner to liquidate SSV cluster with 10 operators (68ms)
- ✓ Allows the cluster owner to liquidate SSV cluster with 13 operators (82ms)
- ✓ Allows a third party to liquidate SSV cluster when liquidatable
- ✓ Is reverted with 'ClusterNotLiquidatable' when a third party tries to liquidate a healthy SSV cluster
- ✓ Is reverted with 'ClusterIsLiquidated' when liquidating an already liquidated SSV cluster
- ✓ Is reverted with 'IncorrectClusterState' when provided cluster data is stale **or** mismatched
- ✓ Is reverted with 'ClusterDoesNotExists' when attempting to liquidate a missing SSV cluster

SSVClusters function `migrateClusterToETH()`

- ✓ Migrates an existing SSV cluster to ETH **and** emits the expected event (61ms)
- ✓ Emits OperatorFeeExecuted for each legacy SSV operator when migrating to ETH
- ✓ Does **not** emit duplicate OperatorFeeExecuted when operator already initialized with ETH defaults
- ✓ Refunds SSV token balance to the owner when migrating an active SSV cluster
- ✓ Uses stored EB snapshot vUnits during migration when present
- ✓ Is reverted with 'InsufficientBalance' when ETH top-up is too low
- ✓ Is reverted with 'IncorrectClusterVersion' when migrating an ETH cluster
- ✓ Is reverted with 'ClusterDoesNotExists' when migrating a missing cluster
- ✓ Validates full migration accounting correctness from SSV cluster to ETH cluster after time passes (59ms)
- ✓ Correctly updates SSV snapshot **and** settles fees for already-ETH operators during migration

- #### updateClusterOperatorsMigration specific tests
- ✓ Preserves SSV snapshot state before validator count reduction
 - ✓ Correctly handles mixed operator states during migration
 - ✓ Accumulates SSV indices correctly for all operators during migration
 - ✓ Handles liquidated cluster migration correctly
- #### Migration balance accounting verification
- ✓ Exact SSV refund after 1000 blocks – independently calculated
 - ✓ Migration with partial SSV balance remaining – exact token transfer (43ms)
 - ✓ Migration with dual SSV/ETH fees – ETH side correctly initialized (50ms)
 - ✓ Zero SSV balance migration – exact refund calculation
 - ✓ Liquidated cluster migration – exact zero refund verification
 - ✓ Maximum precision SSV balance – exact refund with non-round values
 - ✓ Fee integer truncation – totalUnpackedUsage is always a multiple of DEDUCTED_DIGITS

Post-migration EB updates

- ✓ first ETH-side updateClusterBalance after migration applies explicit EB
- ✓ removed operator remains skipped in first post-migration EB update
- ✓ migrateClusterToETH rejects stale caller-supplied SSV cluster state

Removed Operators Security Check

- ✓ Skips removed operators during migration without reviving them
- ✓ Handles migration with all operators removed gracefully
- ✓ Prevents silent revival of removed operators with zero fees
- ✓ Maintains operator count integrity with mixed valid/removed operators

Network fee update impact on active clusters

- ✓ Increase ETH network fee cluster burns faster (71ms)
- ✓ Decrease ETH network fee cluster burn rate decreases
- ✓ Network fee with EB-weighted cluster vUnit scaling applied

Operator fee change + EB burn rate interaction

- ✓ Fee increase with EB=64 cluster → burn rate doubles (77ms)
- ✓ Fee reduction with EB=128 cluster → savings reflected (62ms)

- ✓ Fee change boundary accounting – total burn = sum of both rate periods
- ✓ Fee change with EB=0 (implicit vUnits mode) settles with baseline vUnits
- ✓ Fee change with removed operators skips removed entries **and** settles active operators
- ✓ Fee change can make cluster immediately liquidatable at max EB
- ✓ Multiple fee changes in quick succession preserve exact accounting
- ✓ Fee change with max EB (2048 ETH/validator) uses capped vUnits in settlement
- ✓ Operator fee change with network fee accounting → both fees correctly deducted
- ✓ EB update between fee change execution updates vUnits for earnings calculation
- ✓ Fee change on 4-validator cluster with EB=128 (avg 32 ETH/validator)

SSVClusters function `reactivate()`

- ✓ Reactivates a liquidated cluster with sufficient balance **and** emits correct event (66ms)
- ✓ Keeps operator deviation at zero when reactivating without EB snapshot
- ✓ Is reverted with 'ClusterAlreadyEnabled' when trying to reactivate an active cluster
- ✓ Is reverted with 'ClusterDoesNotExist' when a non-owner tries to reactivate a cluster
- ✓ Is reverted with 'IncorrectClusterState' when provided cluster data is stale **or** mismatched
- ✓ Is reverted with 'InsufficientBalance' when reactivation deposit is too low
- ✓ Is reverted with 'InsufficientBalance' when reactivation deposit does **not** cover runway (49ms)
- ✓ Scales reactivation solvency threshold with EB=64 (2x baseline vUnits) (75ms)
- ✓ Enforces a much higher reactivation threshold when EB=2048 (61ms)
- ✓ Migrates a liquidated SSV cluster to ETH without requiring an EB snapshot
- ✓ Migrates a liquidated SSV cluster to ETH using the stored EB snapshot when present
- ✓ Maintains daoTotalEthVUnits consistency through liquidation/reactivation
- ✓ Maintains accounting consistency across multiple liquidation/reactivation cycles (80ms)
- ✓ Accrues operator earnings across cycles without **double-counting** (72ms)
- ✓ EB=64 liquidate/reactivate then EB=128 update uses reactivated snapshot safely (41ms)
- ✓ validator removal works after EB=64 liquidate/reactivate cycle
- ✓ deposit before reactivation on explicit-EB cluster preserves deposited balance
- ✓ daoTotalEthVUnits is decremented exactly on explicit-EB liquidation
- ✓ daoTotalEthVUnits is restored exactly on explicit-EB reactivation

Removed operator impact on active cluster accounting

- ✓ excludes removed operator fees from ETH cluster settlement **and** freezes removed operator ETH earnings (70ms)
- ✓ freezes removed operator SSV earnings while active operators continue earning on SSV cluster settlement (53ms)

SSVClusters function `updateClusterBalance()`

- ✓ Is reverted with 'RootNotFound' when EB root is missing for the provided block (50ms)
- ✓ Is reverted with 'MustUseLatestRoot' when provided root is **not** the latest committed root
- ✓ Updates cluster balance when proof is valid
- ✓ Updates operator ETH vUnits when effective balance changes
- ✓ Is reverted with 'InvalidProof' when merkle proof is invalid
- ✓ Is reverted with 'EBExceedsMaximum' when effective balance exceeds 2048 ETH per validator
- ✓ Accepts EB at exactly maximum (2048 ETH per 1 validator) **and** produces 640000 vUnits
- ✓ Accepts EB at exactly maximum for 2-validator cluster (4096 ETH) **and** produces 1,280,000 vUnits
- ✓ Is reverted with 'EBExceedsMaximum' when EB exceeds 2048 ETH per validator for a 2-validator cluster
- ✓ Is reverted with 'StaleUpdate' when blockNum is **not** increasing
- ✓ Is reverted with 'UpdateTooFrequent' when a second EB update is within the cooldown window
- ✓ Allows a second EB update after the cooldown window passes (38ms)
- ✓ Updates only EB snapshot for SSV clusters (no ETH operator vUnits accounting)
- ✓ Succeeds on a liquidated cluster: updates EB snapshot but skips fee settlement **and** vUnit updates
- ✓ EB update on insolvent liquidated cluster does **not** corrupt operator **or** DAO vUnit accounting
- ✓ Is reverted with 'EBBelowMinimum' when effective balance is below 32 ETH per validator
- ✓ Multi-validator liquidated cluster: EB update preserves per-validator vUnit accounting
- ✓ EB decrease on liquidated cluster: updates snapshot without corrupting accounting
- ✓ Liquidated cluster with implicit EB: first updateClusterBalance transitions to explicit tracking
- ✓ EB update with effectiveBalance = 0 on zero-validator cluster succeeds without modifying vUnit state
- ✓ Oracle EB report effectiveBalance = 0 on active zero-validator cluster resets explicit EB to implicit-EB sentinel (54ms)
- ✓ Updates vUnit accounting correctly for 13 operators at maximum EB (2048 ETH per validator) (86ms)
- ✓ Auto-liquidates cluster with 13 operators when EB increase to maximum makes it insolvent (113ms)

SSVClusters function `withdraw()`

- ✓ Withdraws from an existing cluster, updates balance **and** emits correct event (57ms)
- ✓ Is reverted with 'InsufficientBalance' when withdrawal would make the cluster liquidatable
- ✓ Settles full fees when usageUnits exceeds uint64 (43ms)
- ✓ Is reverted with 'IncorrectClusterVersion' when withdrawing from an SSV cluster
- ✓ Is reverted with 'InsufficientBalance' when withdrawing more than the cluster balance
- ✓ Is reverted with 'IncorrectClusterState' when provided cluster data is stale **or** mismatched

- ✓ Is reverted with 'ClusterDoesNotExist' when a non-owner tries to withdraw
- ✓ Withdraws deposited funds from a liquidated cluster without reactivating
- ✓ Withdraws full balance from a liquidated cluster that received multiple deposits
- ✓ Withdraws from a liquidated zero-validator cluster even with non-zero liquidation settings
- ✓ Cluster balance becomes 0 when accumulated fees exceed the remaining balance (no underflow)
- ✓ Withdraws from a liquidated cluster after explicit EB update
- ✓ Zero-validator cluster allows full balance withdrawal without fee deduction (45ms)

SSVDAO governance access control (via SSVNetwork)

- ✓ reverts for non-owner on updateNetworkFee() (202ms)
- ✓ reverts for non-owner on updateNetworkFeeSSV()
- ✓ reverts for non-owner on withdrawNetworkSSVEarnings()
- ✓ reverts for non-owner on updateOperatorFeeIncreaseLimit()
- ✓ reverts for non-owner on updateDeclareOperatorFeePeriod()
- ✓ reverts for non-owner on updateExecuteOperatorFeePeriod()
- ✓ reverts for non-owner on updateLiquidationThresholdPeriod()
- ✓ reverts for non-owner on updateLiquidationThresholdPeriodSSV()
- ✓ reverts for non-owner on updateMinimumLiquidationCollateral()
- ✓ reverts for non-owner on updateMinimumLiquidationCollateralSSV()
- ✓ reverts for non-owner on updateMaximumOperatorFee()
- ✓ reverts for non-owner on updateMinimumOperatorEthFee()
- ✓ reverts for non-owner on updateUnstakeCooldownDuration()
- ✓ reverts for non-owner on replaceOracle()
- ✓ reverts for non-owner on updateQuorumBps()
- ✓ reverts for non-owner on updateModule()
- ✓ reverts for non-owner on rescueERC20()

SSVDAO function `commitRoot()`

- ✓ Is reverted with 'NotOracle' when caller is **not** an oracle
- ✓ Is reverted with 'StaleBlockNumber' when block number is **not** greater than last committed
- ✓ Is reverted with 'FutureBlockNumber' when block number is in the future
- ✓ Is reverted with 'ZeroCSSVSupply' when no cSSV supply exists
- ✓ Is reverted with 'AlreadyVoted' when oracle tries to vote twice
- ✓ Emits WeightedRootProposed when quorum is **not** reached
- ✓ Emits WeightedRootProposed repeatedly **and** accumulates weight when quorum is still **not** reached
- ✓ Commits root **and** emits RootCommitted when quorum is reached
- ✓ Commits on the third vote at 75% quorum even when totalSupply is **not** divisible by 4
- ✓ Stores truncated frozen supply **and** emits quorum based on the stored voting supply
- ✓ Requires all 4 oracle votes at 100% quorum even when totalSupply is **not** divisible by 4
- ✓ Does **not** commit on the third vote at 80% quorum when totalSupply is **not** divisible by 4
- ✓ Commits root on the first vote when accumulated weight meets the quorum threshold
- ✓ Is reverted with 'StaleBlockNumber' when trying to propose the same block after it was committed
- ✓ Accumulates weight across multiple oracle votes
- ✓ Is reverted with 'InsufficientCSSVSupply' when totalSupply is below the oracle count
- ✓ Requires all 4 oracle votes when quorumBps is 10000 (100%)
- ✓ Single oracle vote commits root when quorumBps is 1 (1 bps = 0.01%)
- ✓ Oracle replaced mid-vote: old oracle loses voting rights, **new** oracle gets AlreadyVoted for reused

slot

- ✓ Oracle replaced with a completely **new** address: **new** oracle inherits the slot **and** can vote on

subsequent blocks

- ✓ Lowering quorumBps between votes causes the next vote to evaluate against the **new** threshold
- ✓ Raising quorumBps between votes requires additional votes to reach **new** threshold
- ✓ Conflicting roots for same block: first root to reach quorum is committed, further votes on the

losing root revert

SSVDAO function `replaceOracle()`

- ✓ Replaces an oracle **and** emits OracleReplaced event
- ✓ Updates oracle address in storage
- ✓ Updates reverse mapping (oracleIdOf) correctly
- ✓ Is reverted with 'InvalidOracleId' when oracle ID is zero
- ✓ Is reverted with 'ZeroAddress' when **new** oracle address is zero
- ✓ Is reverted with 'OracleAlreadyAssigned' when **new** oracle is already assigned to another ID
- ✓ Is reverted with 'SameOracleAddressNotAllowed' when replacing with same address
- ✓ Can replace an oracle with ID that had no previous address

SSVDAO function `updateMinBlocksBetweenUpdates()`

- ✓ Sets EB update cooldown blocks **and** emits MinBlocksBetweenUpdatesUpdated event
- ✓ Can update EB update cooldown blocks from one non-zero value to another **and** go back to zero

SSVDAO function `updateQuorumBps()`

- ✓ Sets quorum basis points **and** emits QuorumUpdated event
- ✓ Stores the **new** quorum in storage

- ✓ Can set quorum to 100% (10000 bps)
- ✓ Is reverted when quorum is 0
- ✓ Can set quorum to 1 bps (minimum)
- ✓ Is reverted when quorum exceeds 10000 bps
- ✓ Can update quorum from one value to another

SSVDAO function `updateUnstakeCooldownDuration()`

- ✓ Sets unstake cooldown duration **and** emits CooldownDurationUpdated event
- ✓ Stores the **new** cooldown duration in storage
- ✓ Can set cooldown duration to zero
- ✓ Can set high cooldown duration
- ✓ Can update cooldown duration from one value to another

SSVDAO function `updateDeclareOperatorFeePeriod()`

- ✓ Updates the declare operator fee period **and** emits event
- ✓ Stores the **new** declare operator fee period in storage
- ✓ Can set declare period to zero
- ✓ Can update from one period to another

SSVDAO function `updateExecuteOperatorFeePeriod()`

- ✓ Updates the **execute** operator fee period **and** emits event
- ✓ Stores the **new** execute operator fee period in storage
- ✓ Can set **execute** period to zero
- ✓ Can update from one period to another

SSVDAO function `updateLiquidationThresholdPeriod()`

- ✓ Updates the liquidation threshold period **and** emits event
- ✓ Stores the **new** liquidation threshold period in storage
- ✓ Is reverted with 'NewBlockPeriodIsBelowMinimum' when period is below minimum
- ✓ Accepts exactly the minimum threshold
- ✓ Can update from one period to another

SSVDAO function `updateLiquidationThresholdPeriodSSV()`

- ✓ Updates the SSV liquidation threshold period **and** emits event
- ✓ Stores the **new** SSV liquidation threshold period in storage
- ✓ Is reverted with 'NewBlockPeriodIsBelowMinimum' when period is below minimum

SSVDAO function `updateMaximumOperatorFee()`

- ✓ Updates the maximum operator fee **and** emits event
- ✓ Stores the **new** maximum operator fee in storage
- ✓ Can set maximum operator fee to zero
- ✓ Can update from one max fee to another
- ✓ Reverts when the **new** maximum fee is below the configured minimum fee

SSVDAO function `updateMinimumLiquidationCollateral()`

- ✓ Updates the minimum liquidation collateral **and** emits event
- ✓ Is reverted when collateral is **not** a multiple of 1e7 (shrink precision)
- ✓ Stores the **new** minimum liquidation collateral in storage
- ✓ Can set minimum liquidation collateral to zero
- ✓ Can update from one collateral to another

SSVDAO function `updateMinimumLiquidationCollateralSSV()`

- ✓ Updates the SSV minimum liquidation collateral **and** emits event
- ✓ Is reverted when SSV collateral is **not** a multiple of 1e7 (shrink precision)
- ✓ Stores the **new** SSV minimum liquidation collateral in storage
- ✓ Can set SSV minimum liquidation collateral to zero

SSVDAO function `updateMinimumOperatorEthFee()`

- ✓ Updates the minimum operator ETH fee **and** emits event
- ✓ Stores the **new** minimum operator ETH fee in storage
- ✓ Can set minimum operator ETH fee to zero
- ✓ Can update from one min fee to another
- ✓ Reverts when the **new** minimum fee exceeds the configured maximum fee

SSVDAO function `updateNetworkFee()`

- ✓ Updates the network fee **and** emits NetworkFeeUpdated event
- ✓ Is reverted when fee is **not** a multiple of 1e5 (shrink precision)
- ✓ Stores the **new** network fee in storage
- ✓ Updates the network fee from a non-zero value
- ✓ Can set network fee to zero

SSVDAO function `updateNetworkFeeSSV()`

- ✓ Updates the SSV network fee **and** emits NetworkFeeUpdated event
- ✓ Is reverted when fee is **not** a multiple of 1e7 (shrink precision)
- ✓ Stores the **new** SSV network fee in storage
- ✓ Updates the SSV network fee from a non-zero value
- ✓ Can set SSV network fee to zero

SSVDAO function `updateOperatorFeeIncreaseLimit()`

- ✓ Updates the operator fee increase limit **and** emits event
- ✓ Can update operator fee increase limit from one value to another
- ✓ Stores the **new** operator fee increase limit in storage
- ✓ Can set operator fee increase limit to zero
- ✓ Can set high operator fee increase limit
- ✓ Reverts when operator fee increase limit exceeds 100%

SSVDAO function `withdrawNetworkSSVEarnings()`

- ✓ Is reverted with 'InsufficientBalance' when trying to withdraw more than available
- ✓ Is reverted when amount is **not** a multiple of 1e7 (shrink precision)
- ✓ Withdraws network SSV earnings **and** emits NetworkEarningsWithdrawn event
- ✓ Updates DAO balance after withdrawal
- ✓ Can withdraw all available earnings

SSVOperators function `cancelDeclaredOperatorFee()`

- ✓ Cancels declared fee **and** emits expected event (41ms)
- ✓ Is reverted with 'NoFeeDeclared' when canceling without a declaration

SSVOperators function `declareOperatorFee()`

- ✓ Declares operator fee within allowed limits **and** emits event
- ✓ Is reverted with 'FeeTooLow' when declaring below minimal fee
- ✓ Is reverted with 'FeeTooHigh' when declaring above max fee
- ✓ Is reverted with 'FeeIncreaseNotAllowed' when starting from zero fee
- ✓ Is reverted with 'SameFeeChangeNotAllowed' when declaring same fee
- ✓ Is reverted with 'FeeExceedsIncreaseLimit' when increasing fee beyond allowed percentage
- ✓ Is reverted with 'MaxPrecisionExceeded' when declared fee is **not** aligned to ETH_DEDUCTED_DIGITS
- ✓ Emits OperatorFeeExecuted when defaulting legacy SSV operator to ETH fee on declare
- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to declare fee

SSVOperators function `executeOperatorFee()`

- ✓ Executes declared fee **and** emits event
- ✓ Is reverted with 'NoFeeDeclared' when executing without a declaration
- ✓ Is reverted with 'ApprovalNotWithinTimeframe' when executing too early **or** too late
- ✓ Updates operator fee **and** clears request after execution
- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to **execute** fee
- ✓ Is reverted with 'FeeTooHigh' **if** DAO lowers max fee below declared amount before execution
- ✓ Is reverted with 'LegacyOperatorFeeDeclarationInvalid' when executing a pre-upgrade fee declaration
- ✓ executeOperatorFee reverts when governance minimum rises above declared fee
- ✓ [F-08] executeOperatorFee succeeds when declared fee equals newly raised minimum

SSVOperators privacy helpers

- ✓ Updates privacy status via unchecked helpers (44ms)
- ✓ Updates privacy status for a batch of operators
- ✓ Is reverted with 'CallerNotOwnerWithData' **if** caller does **not** own all operators in batch

SSVOperators function `reduceOperatorFee()`

- ✓ Reduces operator fee **and** emits execution event
- ✓ Is reverted with 'FeeIncreaseNotAllowed' when reducing to the same **or** higher fee
- ✓ Clears pending fee declaration when reducing fee
- ✓ Is reverted with 'FeeTooLow' when reducing below minimal allowed fee
- ✓ Is reverted with 'MaxPrecisionExceeded' when reduced fee is **not** aligned to ETH_DEDUCTED_DIGITS
- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to reduce fee
- ✓ Initializes legacy ETH snapshot **and** reduces fee for SSV legacy operator
- ✓ Keeps explicit zero fee after legacy initialization marker is set

SSVOperators reentrancy guard

- ✓ Blocks reentrancy during ETH earnings withdrawal (46ms)
- ✓ Blocks reentrancy during SSV earnings withdrawal

SSVOperators function `registerOperator()`

- ✓ Registers an operator with valid params **and** emits expected events
- ✓ Is reverted with 'FeeTooLow' when provided fee is below minimal allowed
- ✓ Is reverted with 'FeeTooHigh' when provided fee exceeds max operator fee
- ✓ Is reverted with 'OperatorAlreadyExists' when registering duplicate **public** key
- ✓ Stores operator data in storage

- ✓ Registers an operator with 0 fee
- ✓ Registers an operator with exact max fee
- ✓ Registers a **public** operator (whitelisted = false)
- ✓ Is reverted with 'MaxPrecisionExceeded' when fee is **not** aligned to ETH_DEDUCTED_DIGITS
- ✓ Increments operator ID correctly for multiple registrations

SSVOperators function `removeOperator()`

- ✓ Removes operator successfully **and** emits expected event
- ✓ Removes operator with a balance **and** withdraws
- ✓ Removes operator with SSV balance **and** withdraws
- ✓ Removes a legacy SSV-only operator without initializing ETH state
- ✓ Verifies operator state after removal (fees reset, owner persists)
- ✓ Clears a pending fee change request when removing an operator
- ✓ Blocks executeOperatorFee with OperatorDoesNotExist after removal clears both snapshots
- ✓ Blocks cancelDeclaredOperatorFee with OperatorDoesNotExist after removal clears both snapshots
- ✓ Cannot register the same **public** key after removal
- ✓ Clears operatorEthVUnits when removing an operator
- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to remove operator

SSVOperators function `withdrawAllVersionOperatorEarnings()`

- ✓ Withdraws both ETH **and** SSV earnings **and** resets balances (48ms)
- ✓ Withdraws both ETH **and** SSV earnings when both have balances
- ✓ Succeeds when withdrawing with zero balances (no-op)
- ✓ Does **not** initialize ETH snapshot for a legacy SSV-only operator
- ✓ Pays out SSV balance for a legacy SSV-only operator without initializing ETH snapshot
- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to withdraw

SSVOperators ETH earnings withdrawals

- ✓ withdrawOperatorEarnings withdraws specific amount **and** emits event (41ms)
- ✓ Succeeds when withdrawing zero amount
- ✓ withdrawAllOperatorEarnings withdraws full balance **and** resets snapshot
- ✓ Is reverted with 'InsufficientBalance' when withdrawing more than ETH snapshot balance
- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to withdraw ETH earnings
- ✓ Is reverted with 'MaxPrecisionExceeded' when ETH withdrawal amount is **not** aligned to

ETH_DEDUCTED_DIGITS

- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to withdraw all ETH earnings
- ✓ Withdraws exactly 1 * ETH_DEDUCTED_DIGITS (minimum non-zero precision unit) **and** zeroes balance

SSVOperators SSV earnings withdrawals

- ✓ withdrawOperatorEarningsSSV withdraws specific amount **and** emits event (48ms)
- ✓ Succeeds when withdrawing zero amount
- ✓ withdrawAllOperatorEarningsSSV withdraws full balance **and** resets snapshot
- ✓ Is reverted with 'InsufficientBalance' when withdrawing more than SSV snapshot balance
- ✓ Is reverted with 'MaxPrecisionExceeded' when SSV withdrawal amount is **not** aligned to

DEDUCTED_DIGITS

- ✓ Is reverted with 'CallerNotOwnerWithData' when non-owner tries to withdraw SSV earnings

SSVStaking function `claimEthRewards()`

- ✓ Claims accrued ETH rewards **and** emits RewardsClaimed event with correct args (70ms)
- ✓ Keeps remainder in accrued when user still holds cSSV (precision handling)
- ✓ Zeros dust in accrued when user holds 0 cSSV after full transfer (SEC-16b) (44ms)
- ✓ Keeps remainder when user still holds cSSV despite dust (SEC-16b no false positive) (42ms)
- ✓ Is reverted with 'NothingToClaim' when there are no rewards
- ✓ Is reverted with 'NothingToClaim' when accrued amount is too small to payout
- ✓ Is reverted with 'InsufficientBalance' when staking pool has insufficient balance
- ✓ Syncs fees before claiming (47ms)
- ✓ Stores updated accrued balance in storage after claiming
- ✓ Is reverted with 'InsufficientBalance' when ethDaoBalance is insufficient
- ✓ Allows multiple claims as rewards continue to accrue (47ms)
- ✓ Processes only the first claim when two claims are mined in the same block
- ✓ Settles pending rewards before claiming (41ms)
- ✓ Does **not** affect other users' accrued balances (52ms)
- ✓ Zeros dust when user unstakes all cSSV then claims (SEC-16b unstake flow) (39ms)
- ✓ Zeros exactly 99,999 wei dust (max) when bal == 0 after full unstake (SEC-16b boundary) (38ms)
- ✓ Keeps remainder when user unstakes partial cSSV **and** claims (SEC-16b partial unstake) (42ms)
- ✓ Zeros dust after multiple partial cSSV transfers resulting in bal == 0 (SEC-16b multi-transfer)

(45ms)

- ✓ Zeros exactly 99,999 wei remainder when bal == 0 but keeps it when bal > 0 (SEC-16b max dust)

(49ms)

- ✓ Allows **new** accrual after dust was zeroed when user receives cSSV back (SEC-16b re-stake) (52ms)
- ✓ Storage state is clean after dust is zeroed (SEC-16b view consistency) (43ms)
- ✓ Handles exact ETH_DEDUCTED_DIGITS boundary (100,000 wei) correctly (SEC-16b boundary) (42ms)

SSVStaking function `onCSSVTransfer()`

- ✓ Is reverted with 'NotCSSV' when caller is **not** the cSSV token
- ✓ Settles rewards for sender **and** receiver **and** updates user indexes
- ✓ Distributes rewards proportionally across 3 stakers with different balances
- ✓ Allocates rewards by staking time (A early, B late)
- ✓ Settles A→B transfer rewards **and** applies higher future rate to receiver balance
- ✓ Accrues future rewards for a receiver holding exactly 1 wei cSSV
- ✓ Handles sequential transfer chain A→B→C with correct per-period reward accumulation
- ✓ Settles transfer after fee accrual for A=100, B=200 using pendingReward formula
- ✓ Handles multi-transfer sequence A→B→C with exact settlement at each phase (42ms)
- ✓ Settles both users' pending rewards before transfer **and** applies later rewards to post-transfer

balances

SSVStaking reentrancy guard

- ✓ Blocks reentrancy during ETH rewards claim (53ms)

SSVStaking function `requestUnstake()`

- ✓ Requests unstake, burns cSSV **and** emits UnstakeRequested event with correct args (58ms)
- ✓ Creates a withdrawal request with correct unlock time
- ✓ Is reverted with 'ZeroAmount' when requesting unstake of zero amount
- ✓ Is reverted with 'MaxRequestsAmountReached' when pending requests limit is reached (6153ms)
- ✓ Is reverted with 'UnstakeAmountExceedsBalance' when requesting more than balance
- ✓ Is reverted with 'UnstakeAmountExceedsBalance' when caller has no cSSV
- ✓ Allows unstaking full balance
- ✓ Stores withdrawal request in storage
- ✓ Allows a receiver to request unstake after receiving cSSV by transfer
- ✓ Allows multiple sequential unstake requests with different unlock times
- ✓ Uses block.timestamp (seconds) for unlockTime, **not** block.number
- ✓ Cooldown duration change only affects **new** requests, **not** existing ones
- ✓ Cooldown increase: old request keeps original unlock, **new** request uses increased cooldown
- ✓ Cooldown decrease: pending request **not** accelerated, **new** request uses shorter cooldown
- ✓ Settles pending rewards before unstaking when fees have accrued

SSVStaking function `rescueERC20()`

- ✓ Rescues accidentally sent ERC20 tokens **and** emits ERC20Rescued event (44ms)
- ✓ Transfers the correct amount to the recipient
- ✓ Rescues non-standard ERC20 tokens that do **not** return a value (57ms)
- ✓ Is reverted with 'ZeroAddress' when token address is zero
- ✓ Is reverted with 'ZeroAddress' when recipient address is zero
- ✓ Is reverted with 'InvalidToken' when trying to rescue SSV token
- ✓ Is reverted with 'InvalidToken' when trying to rescue cSSV token
- ✓ Is reverted with 'ZeroAmount' when amount is zero
- ✓ Allows partial rescue of tokens

SSVStaking solvency invariant (cSSV supply <= SSV backing)

- ✓ holds before **and** after stake/requestUnstake ordering for a single user (52ms)
- ✓ holds for multiple users with partial unstake requests (44ms)
- ✓ holds through full unstake plus withdraw cycle with mixed unlocked requests (48ms)
- ✓ holds through fee sync **and** ETH reward claims (41ms)
- ✓ holds during cSSV transfer settlement path
- ✓ holds when all users fully unstake **and** withdraw (45ms)

SSVStaking function `stake()`

- ✓ Stakes SSV tokens, mints cSSV **and** emits Staked event (53ms)
- ✓ Updates user index after staking
- ✓ Accepts exactly the minimum stake amount
- ✓ Accepts a stake amount exactly 1 above the minimum
- ✓ Is reverted with 'StakeTooLow' when staking zero amount
- ✓ Is reverted with 'StakeTooLow' when staking below minimum
- ✓ Is reverted when allowance is insufficient
- ✓ Is reverted when staking without approval
- ✓ Is reverted when token balance is insufficient
- ✓ Allows multiple stakes from the same user
- ✓ Settles pending rewards for existing stake when fees accrue before staking again
- ✓ Settles rewards correctly when staking again after a partial unstake
- ✓ Multisig contract stakes SSV tokens
- ✓ Multisig contract stakes multiple times
- ✓ Multisig earns rewards after staking
- ✓ Multisig claims ETH rewards
- ✓ Multisig claims with dust – remainder preserved in accrued
- ✓ Multisig transfers cSSV to EOA

- ✓ EOA transfers cSSV to multisig
- ✓ Multisig transfers cSSV to another multisig
- ✓ Multisig requests unstake
- ✓ Multisig creates multiple unstake requests
- ✓ Multisig requests unstake after earning rewards
- ✓ Multisig withdraws unlocked SSV
- ✓ Multisig withdraws multiple matured requests
- ✓ Multisig complete flow: stake -> earn -> claim -> unstake -> withdraw
- ✓ Mixed EOA and multisig: EOA stakes, transfers cSSV to multisig, multisig claims rewards
- ✓ Transfers SSV tokens to the staking contract

SSVStaking function `syncFees()`

- ✓ Updates staking pool balance and emits FeesSynced event (54ms)
- ✓ Updates accEthPerShare when new fees are available and total staked is non-zero
- ✓ Calculates and syncs fees based on network usage (natural accrual)
- ✓ Does not emit FeesSynced or update accEthPerShare when no new fees (current == previous)
- ✓ Updates pool balance without emitting FeesSynced when current < previous (inconsistent state)
- ✓ Does not change accEthPerShare but updates pool balance when total staked is zero
- ✓ Syncs DAO balance correctly
- ✓ Can be called multiple times
- ✓ Stores updated pool balance in storage
- ✓ Stores updated accEthPerShare in storage
- ✓ Produces non-zero accEthPerShare update with minimum possible fee (1 packed unit) and standard

stake

- ✓ Calling syncFees twice in the same block does not double-count fees

SSVStaking function `withdrawUnlocked()`

- ✓ Withdraws unlocked tokens after cooldown and emits UnstakedWithdrawn event (64ms)
- ✓ Clears the withdrawal request after successful withdrawal
- ✓ Is reverted with 'NothingToWithdraw' when there is no pending withdrawal
- ✓ Is reverted with 'NothingToWithdraw' when cooldown has not passed
- ✓ Is reverted with 'NothingToWithdraw' when partially through cooldown
- ✓ Allows withdrawal exactly at unlock time
- ✓ Clears withdrawal request from storage after withdrawal
- ✓ Withdraws multiple unlocked requests in a single call (49ms)
- ✓ Withdraws only unlocked requests, leaving locked ones pending (45ms)
- ✓ Allows second withdrawal after remaining requests unlock (45ms)
- ✓ Withdraws full amount one year after maturity
- ✓ Does not change cSSV supply on withdrawal
- ✓ Does not allow one user to withdraw another user's tokens

Cooldown duration changes and existing pending requests

- ✓ Does not unlock an existing request earlier when cooldown is reduced after request creation
- ✓ Uses the updated cooldown duration for new requests created after a cooldown change (57ms)
- ✓ Keeps original unlock time for existing request when cooldown is increased after request creation

BUG-4: Double deviation cleanup on liquidated cluster validator removal

- ✓ should not double-subtract deviation when removing all validators from a liquidated cluster with explicit EB (83ms)
- ✓ should not double-subtract deviation when removing validators one-by-one from a liquidated cluster (40ms)
- ✓ should still correctly clean up deviation when removing validators from an ACTIVE cluster

SSVClusters function `bulkExitValidator()`

- ✓ Exits multiple validators and emits events (55ms)
- ✓ Does not change operatorEthVUnits or stored cluster EB snapshot when bulk exiting
- ✓ Exits 10 validators with 4 operators
- ✓ Exits 10 validators with 7 operators (51ms)
- ✓ Exits 10 validators with 10 operators (61ms)
- ✓ Exits 10 validators with 13 operators (67ms)
- ✓ Is reverted with 'ValidatorDoesNotExist' when no public keys are provided
- ✓ Is reverted with 'ValidatorDoesNotExist' when any validator is not registered
- ✓ Is reverted with 'IncorrectValidatorStateWithData' when operator ids do not match stored validators

SSVClusters function `bulkRegisterValidator()`

- ✓ Registers multiple validators, creates new cluster with the expected data and emits correct events
- ✓ Updates operatorEthVUnits even when cluster EB snapshot is not set
- ✓ Increments stored EB snapshot vUnits when cluster EB snapshot is set
- ✓ Registers 10 validators into a new cluster with 4 operators
- ✓ Registers 10 validators into an existing cluster with 4 operators
- ✓ Registers 10 validators into a new cluster with 7 operators (51ms)
- ✓ Registers 10 validators into an existing cluster with 7 operators
- ✓ Registers 10 validators into a new cluster with 10 operators (54ms)

- ✓ Registers 10 validators into an existing cluster with 10 operators
- ✓ Registers 10 validators into a **new** cluster with 13 operators (66ms)
- ✓ Registers 10 validators into an existing cluster with 13 operators
- ✓ Is reverted with 'EmptyPublicKeysList' when no **public** keys are provided
- ✓ Is reverted with 'InvalidPublicKeyLength' when any **public** key is empty **or** has invalid length
- ✓ Is reverted with 'PublicKeysSharesLengthMismatch' **if** there is a mismatch between **public** keys **and**

shares

- ✓ Is reverted with 'ValidatorAlreadyRegistered' **if** trying to register already existing key
- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the length is **not** allowed one for clusters
- ✓ Is reverted with 'UnsortedOperatorsList' **if** the list of operator ids is **not** sorted
- ✓ Is reverted with 'OperatorsListNotUnique' **if** the list of operator ids has duplications
- ✓ Is reverted with 'ClusterIsLiquidated' when trying to register to a liquidated cluster
- ✓ Is reverted with 'OperatorDoesNotExist' when one of the operators has been removed
- ✓ Is reverted with 'OperatorDoesNotExist' when multiple operators have been removed

SSVClusters function `bulkRemoveValidator()`

- ✓ Removes multiple validators, updates cluster state **and** emits correct events
- ✓ Updates operatorEthVUnits on bulk register/remove even when cluster EB snapshot is **not** set
- ✓ Decrements stored EB snapshot vUnits when set **and** removing a subset of validators
- ✓ Clears stored EB snapshot vUnits when removing the last validators
- ✓ Removes 10 validators with 4 operators
- ✓ Removes 10 validators with 7 operators (60ms)
- ✓ Removes 10 validators with 10 operators (77ms)
- ✓ Removes 10 validators with 13 operators (81ms)
- ✓ Is reverted with 'ValidatorDoesNotExist' when no **public** keys are provided
- ✓ Is reverted with 'ValidatorDoesNotExist' when trying to remove non-existent validators
- ✓ Is reverted with 'IncorrectClusterState' when provided cluster data is stale **or** mismatched
- ✓ Is reverted with 'ClusterDoesNotExists' when attempting to remove from a missing cluster
- ✓ Bulk removes multiple validators from active legacy SSV cluster **and** decrements operator/DAO counts

by N (55ms)

- ✓ Reverts bulk removal atomically when one validator in batch is invalid
- ✓ Reverts SSV bulk removal atomically when one validator in batch is invalid (38ms)
- ✓ Keeps reactivate blocked for SSV clusters after bulk removing to zero validators

SSVClusters function `exitValidator()`

- ✓ Exits an existing validator **and** emits the correct event
- ✓ Does **not** change operatorEthVUnits **or** stored cluster EB snapshot when exiting
- ✓ Is reverted with 'IncorrectValidatorStateWithData' when validator was **not** registered
- ✓ Calling exitValidator twice on the same validator succeeds both times without reverting
- ✓ Is reverted with 'IncorrectValidatorStateWithData' when operator ids do **not** match the validator

Validator register/remove with non-zero ETH operator fees

- ✓ registers with 4 operators at different fees **and** deducts $\text{sum}(\text{fees}) * \text{blocksDelta}$ from cluster balance (39ms)
- ✓ second registration after N blocks settles val1 fees; burn rate doubles when val2 is active (43ms)
- ✓ removeValidator settles accumulated fees **and** operator snapshot balance matches expected earnings
- ✓ bulkRegisterValidator deducts fees proportional to bulk validator count

SSVClusters function `registerValidator()`

- ✓ Registers a **new** validator, creates **new** cluster with the expected data **and** emits correct events
- ✓ Initializes ETH defaults for legacy SSV operators **and** keeps them after registration
- ✓ Legacy SSV operators with zero SSV fee initialize ETH snapshot but keep ethFee=0 on registration
- ✓ Updates operatorEthVUnits even when cluster EB snapshot is **not** set
- ✓ Keeps stored EB snapshot unset when registering into an existing cluster without an explicit EB

snapshot

- ✓ Increments stored EB snapshot vUnits when cluster EB snapshot is set
- ✓ Registers a **new** validator with 7 operators
- ✓ Registers a validator into an existing cluster with 7 operators
- ✓ Registers a validator without additional deposit with 7 operators
- ✓ Registers a **new** validator with 10 operators (52ms)
- ✓ Registers a validator into an existing cluster with 10 operators
- ✓ Registers a validator without additional deposit with 10 operators
- ✓ Registers a **new** validator with 13 operators (52ms)
- ✓ Registers a validator into an existing cluster with 13 operators
- ✓ Registers a validator without additional deposit with 13 operators
- ✓ Is reverted with 'InvalidPublicKeyLength' when **public** key is empty **or** has invalid length
- ✓ Is reverted with 'PublicKeysSharesLengthMismatch' **if** there is a mismatch between **public** keys **and**

shares

- ✓ Is reverted with 'ValidatorAlreadyRegistered' **if** trying to register already existing key
- ✓ Is reverted with 'InvalidOperatorIdsLength' **if** the length is **not** allowed one for clusters
- ✓ Is reverted with 'UnsortedOperatorsList' **if** the list of operator ids is **not** sorted
- ✓ Is reverted with 'OperatorsListNotUnique' **if** the list of operator ids has duplications

- ✓ Is reverted with 'ClusterIsLiquidated' when trying to register to a liquidated cluster
- ✓ Is reverted with 'OperatorDoesNotExist' when one of the operators has been removed
- ✓ Revert on removed operator is atomic **and** does **not** partially initialize earlier operators
- ✓ Is reverted with 'OperatorDoesNotExist' when multiple operators have been removed
- ✓ Is reverted with 'ExceedValidatorLimitWithData' when registering a validator that pushes operator over the limit
- ✓ Succeeds registering a validator after removing one to bring operator back below the limit

SSVClusters function `removeValidator()`

- ✓ Removes an existing validator, updates cluster state **and** emits correct events
- ✓ Updates operatorEthVUnits on register/remove even when cluster EB snapshot is **not** set
- ✓ Removes a validator with 7 operators (46ms)
- ✓ Removes a validator with 10 operators (63ms)
- ✓ Removes a validator with 13 operators (62ms)
- ✓ Is reverted with 'ValidatorDoesNotExist' when validator was **not** registered
- ✓ Is reverted with 'IncorrectClusterState' when provided cluster data is stale **or** mismatched
- ✓ Is reverted with 'ClusterDoesNotExists' when attempting to remove from a missing cluster
- ✓ Is reverted with 'ValidatorDoesNotExist' when removing a validator twice
- ✓ Removes validator from active legacy SSV cluster **and** verifies operator counts **and** cluster hash (52ms)
- ✓ Keeps SSV cluster blocked operations after removing last SSV validator
- ✓ Removes validator from liquidated legacy SSV cluster **and** verifies operator counts
- ✓ Handles remove -> liquidateSSV -> remove flow with expected SSV operator/DAO count deltas
- ✓ Removes from SSV, migrates to ETH, removes from ETH, then adds to ETH without storage cross-contamination
- ✓ SSV remove path leaves orphaned EB snapshot untouched (defensive behavior)
- ✓ Processes SSV **and** ETH removals in the same block without storage/counter collision (69ms)
- ✓ Removes validator from SSV cluster with non-zero fees **and** verifies balance deduction
- ✓ Keeps explicit EB snapshot consistent across updateClusterBalance **and** remove
- ✓ Clears remaining explicit EB vUnits when removing the last validator
- ✓ removing one validator keeps deviation but decrements DAO baseline exactly
- ✓ removing the last validator clears DAO deviation for explicit-EB cluster

SSVViews dedicated coverage

- ✓ getBalance **and** getEffectiveBalance **return** expected values for active ETH cluster (228ms)
- ✓ liquidated clusters are reported as liquidated **and** balance/EB getters revert (40ms)
- ✓ isLiquidatable respects exact minimum-collateral boundary
- ✓ getBurnRate scales with EB vUnits (64 ETH == 2x of implicit 32 ETH) (65ms)
- ✓ getOperatorEarnings exposes ETH earnings while SSV earnings stay zero in ETH-only state
- ✓ ETH-only (post-migration-equivalent) views **return** ETH values **and** zero SSV values
- ✓ getOperatorEarnings returns both ETH **and** SSV earnings when both snapshots are funded (40ms)
- ✓ SSV-only clusters **return** positive SSV balance/burn rate while ETH getters **return** zero
- ✓ getEffectiveBalance returns implicit 32 ETH for active SSV cluster with no explicit EB snapshot
- ✓ getEffectiveBalance returns explicit EB for active SSV cluster when vUnits are set

Mainnet Governance Config Validation

Config file (deployments/params-candidate.json)

- ✓ exists **and** is readable from process.cwd()
- ✓ contains all required fields
- ✓ fee fields are non-negative integer strings
- ✓ quorumBps is an integer in [1, 10000]
- ✓ cooldownDuration is a positive integer
- ✓ minBlocksBetweenUpdates is a positive integer
- ✓ defaultOracleIds is an **array** of 4 distinct valid oracle ids
- ✓ minOperatorEthFee <= defaultOperatorEthFee <= maxOperatorEthFee

Packability

- ✓ Confirms raw mainnet values are **not** packable (remainder ≠ 0 mod 100,000)
- ✓ Confirms all deployment config values are packable (divisible by 100,000)
- ✓ All packable config values survive pack/unpack round-trip
- ✓ Is reverted with MaxPrecisionExceeded when packing a non-packable value
- ✓ Packs minimumLiquidationCollateralEth (940,000,000,000,000) without precision loss

Liquidation threshold math

- ✓ liquidationThresholdPeriod (35,800) is above the **system** minimum (21,480 blocks)
- ✓ Liquidation threshold is dominated by minimumLiquidationCollateral floor (51ms)

Operator fee boundaries

- ✓ defaultOperatorEthFee (1,778,800,000) is within [minOperatorEthFee, maxOperatorEthFee]
- ✓ Accepts operator fee at minOperatorEthFee (1,065,200,000)
- ✓ Accepts operator fee at maxOperatorEthFee (5,326,300,000)
- ✓ Is reverted with FeeTooLow when declaring fee one packable step below minimum
- ✓ Is reverted with FeeTooHigh when declaring fee one packable step above maximum

Cluster burn rate

- ✓ Computes correct burn rate for 1, 4, **and** 13 validators

- ✓ Deducts networkFeeEth × N_BLOCKS from cluster balance after N blocks (46ms)

Cooldown duration

- ✓ Is reverted with NothingToWithdraw before cooldown expires (604,800 seconds)
- ✓ Can claim after 604,800 seconds (7 days) elapse
- ✓ Stores cooldownDuration as 604,800 seconds (not blocks)

EB update frequency

- ✓ Stores minBlocksBetweenUpdates

Quorum

- ✓ 2 votes out of 4 should NOT reach quorum (50% < 75%)
- ✓ 3 votes out of 4 should reach quorum (75% >= 75%)

Liquidation collateral

- ✓ Is reverted when liquidating a cluster with balance above minimumLiquidationCollateral (47ms)
- ✓ Liquidates cluster when balance drops below minimumLiquidationCollateral

Long-running clusters (1 year simulation)

- ✓ Fee indices remain within uint64 bounds after 1 year (~2,628,000 blocks) (44ms)
- ✓ Balance accounting remains correct after 1 year (51ms)

SSVPackedLib and SSVCoreTypes

SSVCoreTypes constants

- ✓ PACKED_ETH_ZERO is 0
- ✓ PACKED_SSV_ZERO is 0
- ✓ VERSION_SSV is 0
- ✓ VERSION_ETH is 1
- ✓ VERSION_UNDEFINED is type(uint8).max (255)
- ✓ DEFAULT_OPERATOR_ETH_FEE is 1778_800_000
- ✓ DEDUCTED_DIGITS is 10_000_000
- ✓ ETH_DEDUCTED_DIGITS is 100_000

PackedETHLib

pack / unpack

- ✓ Packs a valid ETH value
- ✓ Unpacks a packed ETH value back to original
- ✓ Pack then unpack is identity for aligned values
- ✓ Packs zero
- ✓ Unpacks zero
- ✓ Packs the maximum uint64 * ETH_DEDUCTED_DIGITS value
- ✓ Reverts with MaxValueExceeded when value exceeds uint64 range
- ✓ Reverts with MaxPrecisionExceeded when value is not aligned
- ✓ Reverts with MaxPrecisionExceeded for ETH_DEDUCTED_DIGITS - 1

raw

- ✓ Returns the raw uint64 value
- ✓ Returns 0 for zero

comparison operators

- ✓ eq returns true for equal values
- ✓ eq returns false for different values
- ✓ neq returns true for different values
- ✓ neq returns false for equal values
- ✓ gt returns true when a > b
- ✓ gt returns false when a == b
- ✓ gt returns false when a < b
- ✓ gte returns true when a > b
- ✓ gte returns true when a == b
- ✓ gte returns false when a < b
- ✓ lt returns true when a < b
- ✓ lt returns false when a == b
- ✓ lt returns false when a > b
- ✓ lte returns true when a < b
- ✓ lte returns true when a == b
- ✓ lte returns false when a > b
- ✓ Comparisons work with zero

arithmetic operators

- ✓ add returns the sum of two packed values
- ✓ add with zero
- ✓ sub returns the difference of two packed values
- ✓ sub to zero
- ✓ sub reverts on underflow
- ✓ add reverts on overflow

PackedSSVLib

pack / unpack

- ✓ Packs a valid SSV value
- ✓ Unpacks a packed SSV value back to original
- ✓ Pack then unpack is identity for aligned values
- ✓ Packs zero

- ✓ Unpacks zero
- ✓ Packs the maximum uint64 * DEDUCTED_DIGITS value
- ✓ Reverts with MaxValueExceeded when value exceeds uint64 range
- ✓ Reverts with MaxPrecisionExceeded when value is **not** aligned
- ✓ Reverts with MaxPrecisionExceeded for DEDUCTED_DIGITS - 1

raw

- ✓ Returns the raw uint64 value
- ✓ Returns 0 for zero

comparison operators

- ✓ eq returns true for equal values
- ✓ eq returns false for different values
- ✓ neq returns true for different values
- ✓ neq returns false for equal values
- ✓ gt returns true when a > b
- ✓ gt returns false when a == b
- ✓ gt returns false when a < b
- ✓ lt returns true when a < b
- ✓ lt returns false when a == b
- ✓ lt returns false when a > b
- ✓ Comparisons work with zero

arithmetic operators

- ✓ **add** returns the sum of two **packed** values
- ✓ **add** with zero
- ✓ **sub** returns the difference of two **packed** values
- ✓ **sub** to zero
- ✓ **sub** reverts on underflow
- ✓ **add** reverts on overflow

ETH vs SSV scaling factor differences

- ✓ Same wei value produces different **packed** values for ETH vs SSV
- ✓ ETH allows finer granularity than SSV
- ✓ DEFAULT_OPERATOR_ETH_FEE is packable as ETH

_safeUint64

- ✓ passes through zero
- ✓ passes through value within uint64 range
- ✓ passes through max uint64
- ✓ reverts on max uint64 + 1
- ✓ reverts on max uint128
- ✓ reverts on realistic overflow scenario (operator earnings delta)

1423 passing (2m)
294 pending

Code Coverage

The client substantially expanded the test suite, increasing it from 728 to 1717 non-failing tests (+992, +136%) and adding broader integration, invariant, boundary, and edge-case coverage.

Coverage Report				
File Path	Line %	Statement %	Uncovered Lines	Partially Covered Lines
contracts/abstract/SSVReentrancyGuard.sol	100.00	100.00	-	-
contracts/libraries/SSVCoreTypes.sol	100.00	100.00	-	-
contracts/libraries/SSVReentrancyGuardLib.sol	100.00	100.00	-	-
contracts/libraries/storage/SSVStorageEB.sol	100.00	100.00	-	-

Coverage Report				
contracts/libraries/storage/SSVStorageReentrancy.sol	80.00	66.67	17	-
contracts/token/CSSVToken.sol	100.00	83.33	-	17, 22
contracts/token/SSVToken.sol	100.00	100.00	-	-
contracts/libraries/ProtocolLib.sol	100.00	88.37	-	114-116, 131-133
contracts/libraries/storage/SSVStorageProtocol.sol	100.00	100.00	-	-
contracts/libraries/storage/SSVStorageStaking.sol	100.00	100.00	-	-
contracts/libraries/CoreLib.sol	100.00	72.22	-	46-48, 69-71, 90
contracts/whitelisting/BasicWhitelisting.sol	66.67	66.67	20-21	-
contracts/libraries/storage/SSVStorage.sol	100.00	100.00	-	-
contracts/libraries/SSVPackedLib.sol	100.00	100.00	-	-
contracts/SSVProxy.sol	100.00	100.00	-	-
contracts/libraries/ValidatorLib.sol	100.00	100.00	-	-
contracts/libraries/ClusterLib.sol	100.00	94.23	-	75, 104, 141-143, 346-348
contracts/libraries/OperatorLib.sol	100.00	91.84	-	205-207, 250-254, 298-301, 313-316, 322-324, 375-377, 413-415, 452
contracts/modules/SSVOperatorsWhitelist.sol	100.00	90.32	-	60-64
contracts/SSVNetworkViews.sol	94.23	94.23	80, 88, 200	-
contracts/modules/SSVOperators.sol	100.00	97.01	-	305, 314, 334, 337, 344-346
contracts/modules/SSVValidators.sol	100.00	98.29	-	251-253
contracts/modules/SSVStaking.sol	100.00	95.35	-	53-55, 103-105, 140-142
contracts/modules/SSVDAO.sol	100.00	100.00	-	-

Coverage Report				
contracts/modules/SSVViews.sol	90.96	88.35	204, 215-216, 268-291, 470-471, 520, 706	126, 177-180, 188-190, 215, 231-233, 269-271, 276-280
contracts/SSVNetwork.sol	98.84	98.72	345	-
contracts/modules/SSVClusters.sol	100.00	98.94	-	582, 593-595
contracts/upgrades/mainnet/SSVNetworkSSVStakingUpgrade.sol	100.00	88.89	-	17
Total	98.41	96.53		

Changelog

- 2026-02-26 - Initial report
- 2026-04-10 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the

use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

